

THERE’S MORE THAN ONE WAY: MODE COLLAPSE IN RLVR & MODEBENCH

Anonymous authors

Paper under double-blind review

ABSTRACT

A model can answer more reliably while producing a narrower range of successful outputs. With binary rewards, reinforcement learning with verifiable rewards (RLVR) gives a repeated success and a different success the same feedback. We introduce MODEBENCH, five executable domains that identify distinct verified outputs for the same prompt, and ReplayMaxRL, which combines learning from fresh samples with memory of verified successes. The same execution keys serve both measurement and replay: each retained key receives equal weight, even when it becomes rare in fresh rollouts. Across three model scales, adding replay to Dr.GRPO improves held-out correctness and sampled verified support. Replay also complements MaxRL across all three scales, with five paired seeds in every domain. Harder-level comparisons cover four complete domains, with PantryPlan remaining partial. Hosted-model evaluation also finds accurate answers concentrated on few verified modes. An idealized analysis explains why binary rewards can amplify common modes and how replay can counter that pressure. Together, these results distinguish reliable answering from the availability of verified alternatives.

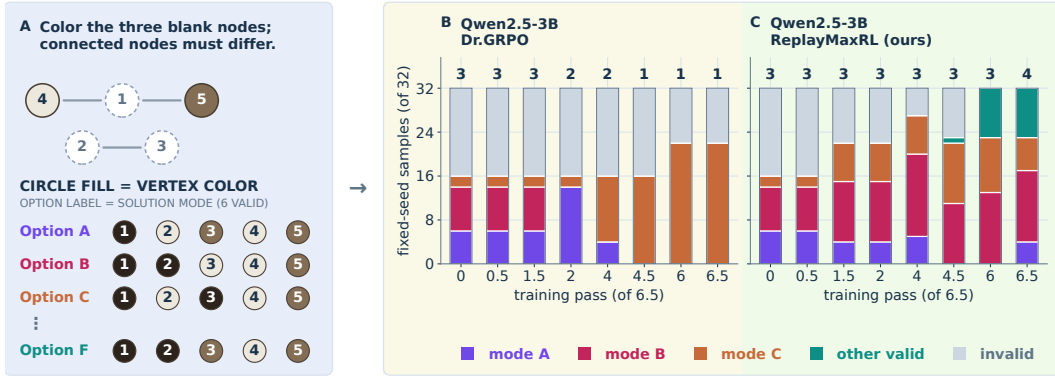


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; ReplayMaxRL produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Figure 5 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A language model can satisfy the same request in many ways: it can offer different explanations, generate different plans, or write different programs that meet the same requirements. These alternatives give users choices and may remain useful as preferences or constraints change. As a model becomes more successful, what happens to the diversity of successful outputs it can produce?

We study this question in reinforcement learning with verifiable rewards (RLVR), which improves a model using executable correctness feedback (Guo et al., 2025). Binary rewards treat a familiar correct solution and a different correct solution alike. Figure 1 illustrates why correctness and

verified alternatives must be measured separately. Our saved-output analysis directly finds increasing correct-key concentration in Graph and PantryPlan under Dr.GRPO and GRPO at all three scales (Appendix H.1). The raw-breadth precheck appears in Appendix H.2. The illustration compares different fresh objectives; the matched factorial separates their effects from replay.

MODEBENCH supplies the missing measurement: each validator returns a canonical execution key alongside correctness. Different strings may share a key; different accepted computations may reach the same answer. These identities also organize memory. ReplayMaxRL combines fresh MaxRL updates with replay of one verified exemplar per admitted key, assigning equal weight regardless of discovery frequency. Common successes therefore do not dominate the bank’s weights simply because they are sampled repeatedly.

The benchmark, conditional theory, and controlled comparisons study the same verified alternatives. Replay improves held-out success and sampled breadth in demonstrated settings and reduces conditional concentration in the Graph/Pantry comparisons under Dr.GRPO. Hosted observations establish the measurement’s broader relevance without identifying a training cause. Whether these alternatives improve later search or adaptation remains untested.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

For a prompt x with specification s_x , a validator maps an output y to failure $\perp$ or a canonical key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs count as the same mode exactly when their keys agree. Training sees only keys generated by the current policy; neither the full valid-key set $\mathcal{C}_x$ nor its size is supplied to the learner or replay bank.

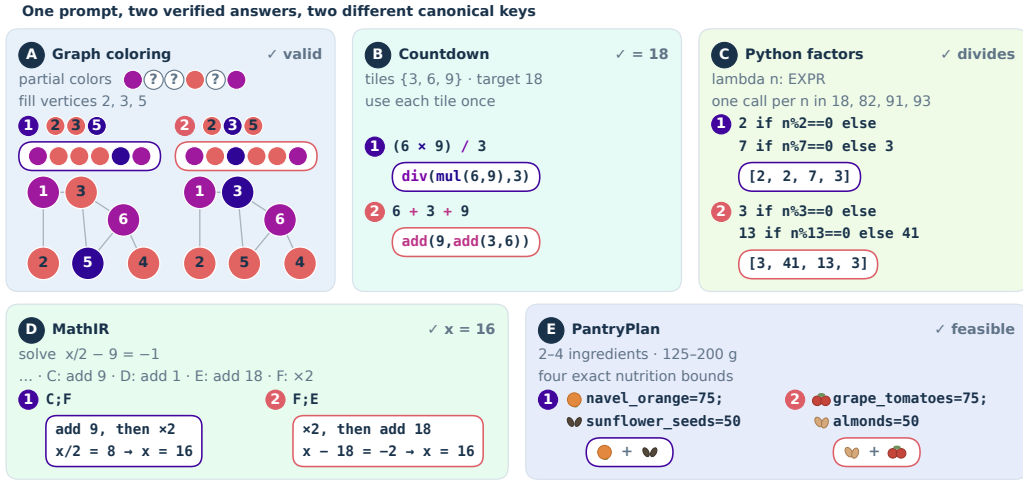


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

Figure 2 shows the five domains. Countdown and MathIR capture different routes to an answer, Python captures different tested behaviors, and Graph and PantryPlan capture different valid configurations. This distinction matters when interpreting breadth: an additional valid configuration and an additional computation route are both verified modes, but they are not the same kind of alternative. The keys measure executed outputs, not latent reasoning strategies or the usefulness of a solution in a later task. Appendix D gives the exact key and alias rules, with verbatim prompts in Appendix E.

2.1 SUCCESS AND SAMPLED BREADTH

Let $P = \Pr[V(Y, s_x) \neq \perp]$ and, when $P > 0$, let $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$. For K independent samples, D_K counts their distinct correct keys. The promptwise identities are

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K]. \quad (1)$$

The first endpoint measures success probability; the second measures an expected number of verified keys. We use $K = 8$ for the training endpoints. Their difference, *extra modes*, counts verified keys beyond the first success. Both raw breadth and extra modes can increase because correctness increases; neither by itself establishes a more balanced q .

Conditional collision measures that allocation directly:

$$C(q) = \sum_c q_c^2, \quad B_2 = 2P - P^2C(q), \quad (2)$$

where $B_2 = \mathbb{E}[D_2]$. Thus C is the probability that two independent correct outputs share a key. Greater C means greater concentration. We call an observed increase in this conditional concentration *mode collapse*; finite samples do not establish literal extinction of unobserved modes. At fixed P , majorization toward a more concentrated q lowers expected breadth, but a change in C alone does not imply majorization or a change in total support (Appendix B).

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

Levels retain each domain’s validator and canonical keys while changing problem structure. Level 1 is the original construction. Level 2 increases structural demands, such as hidden graph vertices and required Countdown operands, while preserving designated valid-mode-count histograms. Before training, frozen Qwen2.5-0.5B development pass@8 must lie in $[.10, .90]$ and below its Level-1 value in every domain.

The scale-calibrated extension uses Qwen2.5-3B, 7B, and 14B for Levels 3, 4, and 5, respectively. Each targets the same fixed Qwen2.5-0.5B Level-1 success rates; distinct@8 is measured but never fitted. Fresh held-out confirmation admits a level only if all five domains meet the registered pass@1 and pass@8 tolerances. Level 3 with its original prompts is admitted; a neutral Python revision and Levels 4–5 remain under calibration. These construction checks precede the replay comparison in Experiment 3. Native training evaluations use separate populations and parts of the protocol, so cross-level differences do not isolate difficulty (Appendix D.3).

3 REPLAYMAXRL: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

Figure 3 shows the two uses of verification: correctness rewards train on fresh samples, and canonical keys organize memory. Both paths use policy-generated responses; neither requires gold solutions or a precomputed catalogue of valid modes.

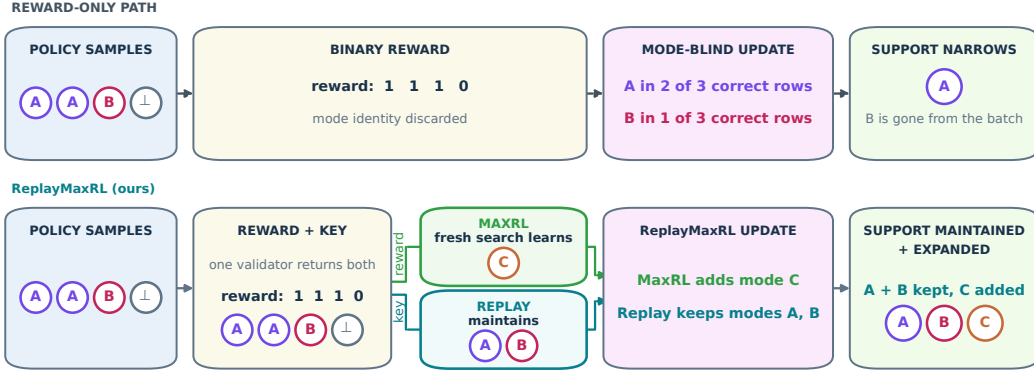


Figure 3: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores exemplars by canonical key and revisits retained keys with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

3.1 LEARNING FROM FRESH ROLLOUTS

MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026). With binary rewards, its centered group estimator weights a correct response more strongly when successes in the group are scarce. This provides a useful test of replay: memory should be evaluated alongside an objective designed for learning from sampled successes, as well as alongside Dr.GRPO (Liu et al., 2025).

Both fresh objectives still depend on what the policy currently samples. An absent mode contributes no direct score term, and an all-correct or all-incorrect group has zero centered task advantage. MaxRL changes the fresh learning signal without creating a persistent record of which correct behaviors have occurred. Appendix F.1 specifies the exact advantages and their finite-group interpretation.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Each training prompt owns an initially empty, bounded bank $\mathcal{B}_x$. When a new key is admitted, the bank retains one verified exemplar e_c . Repeated discoveries of that key do not add further entries. A deterministic schedule revisits nonempty prompt banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (3)$$

Each retained key receives the same share of replay, independent of how often it appeared in fresh rollouts. A mode discovered once therefore continues to receive a signal alongside a mode discovered repeatedly. Token normalization sets the loss scale for each exemplar; it does not make exemplar likelihood equal to the total probability of generating its canonical key.

We add this loss to the fresh MaxRL update. Replacing MaxRL with Dr.GRPO yields ReplayDr.GRPO. Replay exemplars stay outside the fresh reward group, so the fresh objective and replay can be varied separately. The controls execute the same bank and scoring code with zero replay gradient; their learned banks and workloads can differ as policies diverge. Banks contain only responses to training prompts. Held-out evaluation therefore tests what training changes in the policy, rather than retrieving answers from the bank. Appendix F gives admission, scheduling, masking, and loss scaling.

3.3 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

Consider one isolated prompt, finitely many categories with at least two correct and one incorrect outcome, finite initial logits, and group size $G \geq 2$. With independent canonical-outcome logits,

common length normalization, and Euclidean infinitesimal on-policy updates without a competing regularizer, the GRPO, Dr.GRPO, and binary MaxRL mean flows satisfy, after a positive time change,

$$\frac{dq_c}{d\tau} = q_c(q_c - C), \quad \frac{dC}{d\tau} = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (4)$$

A unique initial winner attracts the conditional mass (Theorem C.3). Aggregating strings into keys does not preserve neural gradient geometry; this does not certify AdamW/PPO or variable-length GRPO trajectories in the implemented neural models.

For the categorical replay surrogate, bank mass P_B and its conditional allocation q_B give the identity

$$-\frac{1}{|B|} \sum_{c \in B} \log p_c = -\log P_B + \log |B| + \text{KL}(U_B \| q_B). \quad (5)$$

The objective rewards banked mass and penalizes imbalance. The actual exemplar loss is not an exact key-probability objective. In the stated categorical flow, a fixed bank and constant positive replay dose give banked modes a probability floor; unbanked modes may vanish (Theorem C.6). Neither this floor nor a bounded exemplar surrogate guarantees visibility in eight draws or held-out transfer. Appendix A maps each claim to its basis and scope.

4 EXPERIMENTAL DESIGN

We compare Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B on all five Level-1 domains. Each run trains for eight passes with five registered seeds per comparison. Evaluation averages four eight-sample draws on 128 held-out prompts, with matched requests across methods. Complete paired blocks receive nominal 95% intervals; incomplete intersections retain their exact seed counts and are descriptive. Missing or ambiguous endpoints are not imputed. The shared recipe and source-integrity rules are in Appendices D.4 and M.1.

The comparisons answer three questions. First, does adding canonical replay to Dr.GRPO improve correctness and verified support across scales? Supporting controls test reward normalization, on-policy redistribution, ordinary success memory, and semantic exploration. UCPO and sparse RLEP-Dr are compared at 0.5B and 1B only; fixed Semantic-MaxEnt is defined in Appendix F.5. Second, does replay still help when the fresh objective is MaxRL? The central factorial crosses Dr.GRPO/MaxRL with replay off/on at all three scales, yielding Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL. Third, does the same pattern hold on the harder Level 2? This factorial uses Qwen2.5-0.5B, with four complete domain comparisons and partial PantryPlan results. Appendix Table 6 gives the exact coverage.

5 RESULTS

Mode collapse is measurable during training. All twelve Graph/Pantry Dr.GRPO/GRPO initial-to-final contrasts increase correct-key collision, with nominal intervals above zero. Both stream orientations agree in eleven; Falcon Graph GRPO is the exception. ReplayDr.GRPO lowers collision in all six matched Graph/Pantry contrasts, with negative intervals and both orientations agreeing. MaxRL replay has negative primary contrasts there, but Falcon and 3B Graph have an orientation exception. These show mitigation, not restoration of the initial distribution (Appendix Figure 18).

For $R \geq 2$ correct outputs with counts n_c , we estimate $\hat{C} = \sum_c n_c(n_c - 1)/[R(R - 1)]$ and weight jointly eligible prompts equally within seed. Under the audited runtime mapping, four eight-output groups yield eleven nominal streams; we take each stream’s earliest output and check disjoint 5/6-stream orientations. Distinct IDs do not prove iid sampling, so these selected-population means remain descriptive. Python has no jointly eligible Dr.GRPO/GRPO initial prompts; some MathIR conditions already have collision one. Replay’s eligible-prompt correctness can decrease while full-test `pass@8` rises. Full-test endpoints retain their intact eight-output groups. Appendix H.1 reports coverage, all 135 contrasts, and the retrospective source-census extension.

5.1 EXPERIMENT 1: REPLAY IMPROVES SUCCESS AND SAMPLED SUPPORT

Replay improves success and sampled support across scales. Figure 4A shows positive block-mean changes in both primary endpoints for every model and domain. Raw `distinct@8`

increases in all 74 admissible seed pairs. This consistency matters: the result spans valid configurations, computation routes, and tested program behaviors, rather than depending on a single task or favorable seed. The excluded Falcon Countdown run and its four-seed comparison remain explicit in the figure and Appendix M.1.

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

	Δ pass@8						Δ distinct@8					
	Graph	Countdown	Python	MathIR	Pantry	Average	Graph	Countdown	Python	MathIR	Pantry	Average
Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

	Δ pass@8						Δ distinct@8					
	Graph	Countdown	Python	MathIR	Pantry	Average	Graph	Countdown	Python	MathIR	Pantry	Average
Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04

Figure 4: **Canonical replay improves correctness and verified breadth.** (A) ReplayDr.GRPO minus Dr.GRPO at all three scales. (B) Qwen2.5-0.5B alternatives minus Dr.GRPO. Columns measure probability points and expected verified-key counts, respectively. The green Average weights domains equally within seed. A dagger marks four paired seeds in Falcon Countdown and its Average; other cells use five. Only complete blocks receive nominal 95% intervals. Purple denotes ReplayDr.GRPO; bold entries mark column maxima in Panel B.

Countdown provides a concrete interpretation: replay more than triples the number of sampled verified computation routes at Qwen2.5-0.5B while also raising correctness. Python shows why the two metrics must be read together: its correctness improves substantially, but the gain in breadth beyond the first correct output remains uncertain. More successful samples alone do not demonstrate a more balanced distribution over correct outputs. The appendix reports the exact means and uncertainty for these examples (Appendix G.2).

The controls also distinguish the value of balancing keys from simply reusing successes. At Qwen2.5-0.5B, ReplayDr.GRPO has the largest mean correctness gain among the direct alternatives in every domain (Figure 4B). Replacing equal key weights with weights proportional to fresh discovery frequency reduces extra modes on average, while the aggregate correctness contrast remains inconclusive. The advantage is concentrated in Graph, Countdown, and PantryPlan; it is not a uniform benefit across tasks. This ablation supports key balancing as a contributor to breadth (Appendix G.6, Figure 17).

5.2 EXPERIMENT 2: REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Replay adds value beyond MaxRL at all three scales. The two replay comparisons in Figure 5 separate memory from the choice of fresh objective. At 0.5B, 1B, and 3B, replay improves the domain-averaged correctness and raw mode count under both Dr.GRPO and MaxRL. Thus the replay benefit persists when fresh learning already uses MaxRL’s sampling-aware objective. The comparison identifies the effect of adding the replay update; it does not establish which individual stored modes account for the held-out gains.

The effect is not identical across domains or scales. ReplayMaxRL improves both endpoints in every 0.5B domain. At 1B, mean correctness improves in every domain, but Graph’s mean raw-support effect is negative and inconclusive. The 3B MaxRL comparison is now complete: all 50 endpoints cover five domains, both methods, and seeds 70–74. Averaging domains equally within seed, correctness rises from .547 to .701 and verified modes from .744 to 1.107. All five domains have positive mean effects on both endpoints; Graph and Python correctness inter-

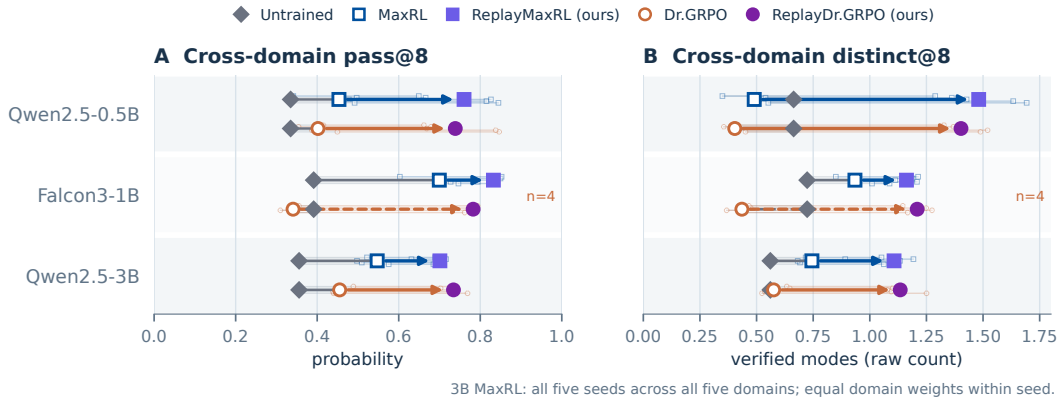


Figure 5: **Replay complements both fresh objectives at multiple scales.** Each row shows Dr.GRPO/ReplayDr.GRPO and MaxRL/ReplayMaxRL on both endpoints. Diamonds are initial models; open/filled markers are training without/with replay. Complete tracks average domains within seed; faint paths show seeds. All MaxRL tracks use five paired seeds in all five domains, including 3B. Falcon Dr.GRPO uses four common seeds; the other Dr.GRPO tracks use five. Appendix Figure 9 reports each domain’s correctness and verified-mode results at every scale.

vals still include zero. Appendix Figure 8 shows domain-specific effects; Appendix G.2 gives the aggregates; Appendix G.4 shows both metrics throughout training.

5.3 EXPERIMENT 3: REPLAY WITHIN A SECOND TASK CONSTRUCTION

Replay improves average endpoints within both levels. Figure 6 compares the four domains with complete factorials at Qwen2.5-0.5B: Graph, Countdown, Python, and MathIR (80/80 endpoints). Their aggregate moves toward greater correctness and sampled support with replay under either fresh objective, at both levels. The levels share mode definitions but use separate test populations; the within-level replay comparisons control prompts and fresh objectives. PantryPlan remains outside this aggregate because its Level-2 factorial is incomplete (10/20 endpoints).

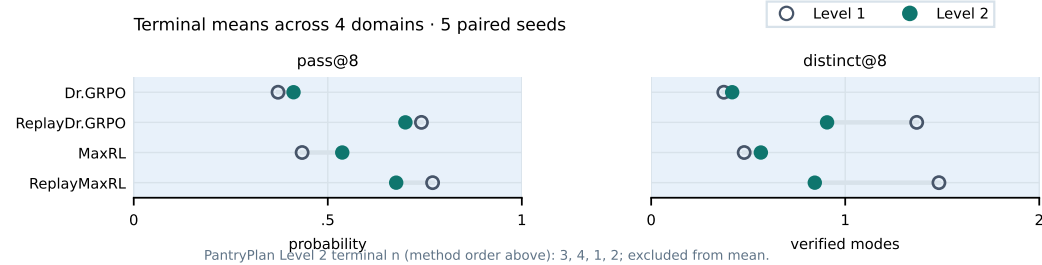


Figure 6: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2. Terminal means average Graph, Countdown, Python, and MathIR equally within each of five paired seeds. Both levels and all four methods share these seeds. Construction and admission belong to the benchmark design (Section 2.2); different test populations and parts of the protocol prevent an isolated difficulty interpretation. Partial PantryPlan arm counts remain separate (Appendix G.3).

Graph supplies the clearest domain-level evidence: both replay variants improve correctness and raw support substantially on the harder problem. The other completed domains’ correctness and raw-support intervals include zero. We therefore interpret the aggregate as evidence that replay can remain useful on these harder tasks; the cross-level comparison does not isolate a causal difficulty effect or establish universal improvement. Appendix G.3 reports the Graph estimates, and Appendix G.3 gives all fresh-objective and replay-interaction contrasts. Appendix Figure 10 uses the common four-arm intersection: five seeds in each complete domain, one descriptive Pantry seed. Sparse Pantry collision contrasts increase under replay, so the concentration benefit is not universal.

Prompt guidance and sampling budget affect the observed breadth. Two Qwen2.5-0.5B follow-ups cover selected Level-2/3 Python, MathIR, and Pantry problems. Removing strategy guidance, including an executable Python example, substantially lowers Python correctness (Appendix K). Collecting 64 fresh responses per problem and wording finds persistent concentration in most trained MathIR conditions, but additional discovery in Pantry and neutral-prompt Python replay (Appendix L). Eight draws can therefore understate available alternatives; prompt-driven correctness changes alone do not establish redistribution among correct solutions.

5.4 HOSTED MODELS: CORRECTNESS DOES NOT DETERMINE BREADTH

The distinction measured by MODEBENCH also appears beyond the training comparisons. We evaluate seven hosted deployments on all five domains and three levels, without additional training. Table 1 averages `pass@8` and `distinct@8` across domains at each level. Across deployments and levels, `pass@8` ranges from 98.0% to 100%, while the mean number of verified modes ranges from 1.36 to 2.31. Appendix Figure 22 gives the domain breakdown.

Model	Level 1		Level 2		Level 3	
	<code>pass@8</code> (%)	# modes	<code>pass@8</code> (%)	# modes	<code>pass@8</code> (%)	# modes
GPT-5.6 Sol	98.0	1.80	98.4	1.50	99.2	1.45
Claude Opus 5	98.3	1.43	99.7	1.44	99.8	1.36
GPT-5.4	98.1	2.11	98.4	1.72	99.5	1.68
Grok 4.3	99.5	1.97	100.0	1.57	100.0	1.55
Kimi K3	98.4	2.31	100.0	2.05	100.0	1.90
Claude Opus 4.8	98.8	1.64	100.0	1.67	99.8	1.51
DeepSeek V4 Pro	98.4	2.01	99.1	1.84	99.7	1.62

Table 1: **Hosted success and output diversity, averaged by level.** Each level averages five domains equally, using 128 prompts per domain and eight responses per prompt. `pass@8` is the fraction with any verified response; # modes is mean `distinct@8` over all prompts, including failures.

A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. All runs requested medium reasoning. Appendix J reports original-protocol results, prompt comparisons and uncertainty.

These exploratory comparisons describe the evaluated deployment and prompt configurations; they do not identify a training cause or a model-size effect. Hosted collision pools correct pairs, with population weights proportional to P^2 at a common draw budget; training collision weights eligible prompts equally. These are different aggregate estimands. Higher levels need not lower correctness for every deployment, and the display is not a common-prompt ranking.

A matched 480-prompt subset tests five temperatures with GPT-5.6 Sol (Figure 7). Non-default temperatures require reasoning `none` on this deployment. Raising T from 0 to 1.5 increases normalized empirical `pass@8` from 65.6% to 79.8% and mean `distinct@8` from .673 to 1.125. At $T = 2$, these are 79.4% and 1.146; paired 95% intervals for $T = 2$ minus $T = 1.5$ include zero on both axes.

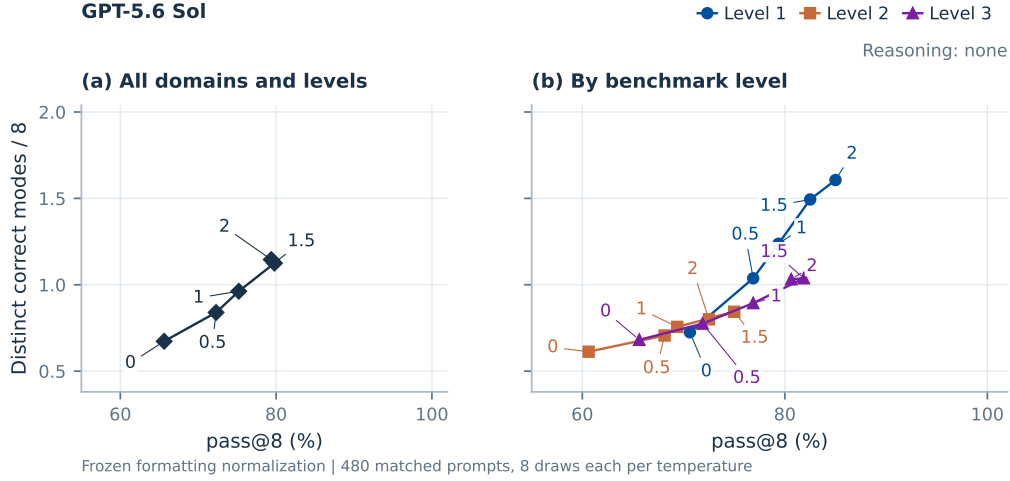


Figure 7: **Temperature traces $\text{pass}@8$ and verified breadth.** GPT-5.6 Sol, with 480 fixed prompts and eight draws each at $T \in \{0, 0.5, 1, 1.5, 2\}$. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical $\text{pass}@8$ is the fraction with at least one correct draw; $\text{distinct}@8$ averages unique correct modes, including zeroes. Connected points use reasoning none and the frozen formatting normalizer. Appendix J.3 gives pointwise uncertainty intervals for every benchmark level and all five measured temperatures.

6 RELATED WORK

RLHF and reasoning RL can reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). MODEBENCH measures verified output identities; semantic clustering instead infers equivalence from language (Farquhar et al., 2024). Sampling-based selection motivates preserving proposal alternatives (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024); our experiments do not measure their downstream utility.

Outcome bonuses, set prediction, distribution matching and uniform-correct objectives also encourage broader support (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026). MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); we ask whether memory adds value. Replay can mitigate forgetting (Lin, 1992; Rolnick et al., 2019), and RLEP reuses successful reasoning trajectories (Zhang et al., 2025). Our memory balances execution keys independently of discovery frequency, connecting replay to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates correctness from the availability of alternative verified outputs. Replay-MaxRL uses their execution identities as memory: fresh updates improve the policy on new samples, while replay keeps stored successes involved in training. Matched comparisons show that replay can improve correctness and sampled breadth, including beyond MaxRL.

Our claims concern sampled, task-defined outputs. Replay cannot retain a mode before discovering and storing it, and improved held-out breadth does not show that every stored exemplar survives. A fixed-bank audit finds improving central scores alongside a declining tail (Appendix H.4); exact mode probabilities remain unverified. The idealized theory does not certify neural training, and the experiments do not show that breadth causes the accuracy gains. Whether retaining alternatives helps models meet changing requirements or user preferences remains an open question.

REFERENCES

- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025. URL <https://arxiv.org/abs/2505.22617>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.
- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.

- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6820–6829. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017. URL <https://arxiv.org/abs/1701.06548>.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou. RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.

APPENDIX ORGANIZATION

The supplement opens with the claim–evidence map (Appendix A), then gives measurement identities and the complete conditional theory (Appendices B and C). Benchmark construction, full prompts, and the implemented algorithm follow. Controlled results (Appendix G) precede longitudinal diagnostics (Appendix H) and the separate hosted observations. Prompt interventions and reproducibility close the scientific record. All original proof statements and proofs are retained. The main keeps its original figure sequence; additional concentration, factorial, weighting, and within-level effect plots support the corresponding appendix analyses.

A CLAIMS AND THEIR EVIDENTIAL BASIS

The training and hosted studies share a measurement question: how is correct probability allocated among verified alternatives? They supply different kinds of evidence. The table states the basis of each central claim and the inference it supports; motivating downstream benefits remain hypotheses.

Claim	Identifiable basis	Scope of the inference
Success and breadth differ	Identity and concavity: Lemma B.1	At fixed correctness, concentration in majorization order lowers expected breadth. More correct samples can also raise the raw mode count.
Uniform categorical replay balances mass	Identity: Equation (8)	Likelihood replay already contains a balancing term. This neither requires a separate balance loss nor equates exemplar scores with key probabilities.
Binary reward can induce collapse	Conditional theorem: Theorem C.3	Independent canonical-outcome logits, common length normalization, Euclidean mean flow, and a unique initial winner; no neural-optimizer guarantee.
Replay remains available when fresh gradients vanish	Conditional score calculation: Appendix C.3	Equal-reward groups have zero centered task update; bank scoring need not. An available gradient does not ensure improvement after a shared-parameter update.
Persistent replay protects banked modes	Conditional theorem and corollary: Theorem C.6, Corollary C.7	Fixed bank, fixed positive dose, and the stated mean flow yield probability floors. Uniformity needs full coverage and equal categorical weights; unbanked modes may vanish.
Exemplar loss can bound key probability	Conditional lemma: Lemma C.8	A bounded joint potential and complete-response likelihood under the same sampling law protect those prompts. Neither held-out transfer nor visibility in eight draws follows.
Training can concentrate correct-key outputs	Retrospective longitudinal comparison: Appendix H.1	Graph/Pantry Dr.GRPO and GRPO show increased collision at each scale. Eligibility and stream sensitivities limit the observed population; exact support extinction is not identified.
Replay can reduce conditional concentration	Retrospective matched replay comparison: Appendix H.1	ReplayDr.GRPO reduces collision in Graph/Pantry at each scale. Correctness can also change on those eligible prompts; this is not an all-prompt effect.
Replay improves measured endpoints	Controlled comparisons: Figure 8	Paired replay additions isolate the implemented intervention within the reported model/domain cohorts. All five 3B MaxRL blocks use five paired seeds; individual-mode retention is not identified.
Equal key weights can aid breadth	Controlled weighting comparison: Appendix G.6	Uniform versus fresh-frequency weighting changes extra sampled modes in the reported cohorts; the average correctness contrast is inconclusive.
Replay can help on Level 2	Controlled within-level comparisons: Figure 10	Four complete domains support within-level replay effects. Separate terminal test splits prevent a pure cross-level difficulty-effect interpretation.
Fixed exemplars show heterogeneous score changes	Exploratory longitudinal finding: Appendix H.4	Central scores improve while a tail declines. No matched no-replay bank arm identifies a causal retention effect or exact key probabilities.
Accurate hosted outputs can be concentrated	Exploratory deployment comparison: Appendix J	Observed correctness and breadth describe the stated protocols. They do not identify RLVR as the cause, a model-size effect, or training-time collapse.
Temperature changes observed breadth	Exploratory matched-prompt comparison: Appendix J.3	Four temperature settings describe one deployment and reasoning profile. The best observed point does not establish an optimal sampling temperature in general.

Table 2: **What each central claim rests on.** Identities concern the sampling law, theorems specify conditional dynamics, and experiments support the stated intervention or exploratory observation. All reported intervals are nominal and unadjusted.

The before/after verifier-training precheck is a longitudinal checkpoint comparison (Appendix H.2). Its extra-mode declines are measured changes in the registered runs, not proof of zero probability for lost keys or a universal decrease in raw breadth across domains.

B METRIC AND OBJECTIVE DETAILS

Correctness and solution support are distinct properties of a policy. For a prompt x with reference specification s_x , the validator maps a response to either failure or a canonical key:

$$V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}.$$

Here $\mathcal{C}_x$ is the set of valid solution keys and $\perp$ denotes failure. The learner observes a key only for a response it generates; neither the full set $\mathcal{C}_x$ nor its size is revealed.

For $Y \sim \pi_\theta(\cdot | x)$, the policy induces both a total probability of verified success and a distribution over how that probability is allocated:

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c | x) = \Pr[V(Y, s_x) = c | V(Y, s_x) \neq \perp].$$

The conditional distribution is defined only when $P_\theta^+(x) > 0$; when $P_\theta^+(x) = 0$, all three sampling metrics below are zero. The first quantity measures correctness. The second describes the distribution of verified behaviors conditional on being correct. We call concentration of p_θ^+ onto fewer keys *mode collapse*, particularly when P_θ^+ remains stable or increases.

B.1 MEASURING SUCCESS AND SUPPORT

For K independent samples $Y_{1:K}$, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

MODEBENCH reports three averages with different units. $\text{distinct@}K = \mathbb{E}[D_K]$ is an expected count in $[0, K]$ of unique verified keys; $\text{pass@}K = \Pr[D_K > 0]$ and $\text{mean@}K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$. For every draw, $F_K \leq \mathbf{1}[D_K > 0] \leq D_K$; hence $\text{mean@}K \leq \text{pass@}K \leq \text{distinct@}K$. The inequality is valid, but equal numerical changes are not commensurate: .02 in $\text{pass@}8$ is two probability points, whereas .02 in $\text{distinct@}8$ is .02 expected verified keys.

Lemma B.1 (Success and sampled breadth). *Fix a prompt, a finite or countable valid-key set, and $K \geq 1$ independent draws from the same fixed sampling law. Let $\mu_c = \Pr[V(Y, s_x) = c]$ and $P = \sum_c \mu_c$. Then*

$$S_K := \text{pass@}K = 1 - (1 - P)^K, \quad B_K := \text{distinct@}K = \sum_c [1 - (1 - \mu_c)^K]. \quad (6)$$

For a finite valid-key set of size $m \geq 1$ and $P > 0$, write $q_c = \mu_c/P$. At fixed P , B_K is Schur-concave in q : if q majorizes q' (its decreasingly sorted partial sums are at least those of q'), then $B_K(P, q) \leq B_K(P, q')$. In particular,

$$1 - (1 - P)^K \leq B_K(P, q) \leq m[1 - (1 - P/m)^K]. \quad (7)$$

For $K \geq 2$, the lower bound is attained only by concentration on one key, and the upper bound only by uniform q . For $K = 1$, $B_1 = P$ for every q . At fixed q , increasing P cannot decrease B_K .

Proof. All K draws fail with probability $(1 - P)^K$. For each key c , its appearance indicator has expectation $1 - (1 - \mu_c)^K$; summing indicators gives the second identity, also for countably many keys by nonnegativity. For fixed $P > 0$ and $K \geq 2$, the function $f(v) = 1 - (1 - Pv)^K$ is strictly concave on $[0, 1]$. Transferring mass from a larger coordinate to a smaller one without reversing their order increases $\sum_c f(q_c)$; successive such transfers give the majorization assertion. The point mass and uniform vector are its extreme cases, giving the bounds and equality conditions. Finally, $\partial B_K / \partial P = K \sum_c q_c (1 - Pq_c)^{K-1} \geq 0$, which proves monotonicity in correctness. $\square$

These are promptwise identities, before averaging over prompts. In particular, applying the nonlinear formula to a mean correctness generally changes the result. They explain the shared measurement in the training and hosted studies: success depends on total correct mass; sampled breadth also depends on its allocation. More breadth alone proves neither greater conditional diversity nor better success, and neither metric establishes downstream utility.

Raw `distinct@K` measures sampled coverage, not the total number of positive-probability modes. For an uninformed reference, suppose a finite space has N actions and n_c actions map to valid key c . Uniform independent action sampling then gives $\sum_c [1 - (1 - n_c/N)^K]$ expected verified keys. The simpler expression $m_x [1 - (1 - 1/N)^K]$ applies when the m_x valid actions have distinct keys, as in our reported finite-action references. Open-ended program and derivation spaces have no canonical nominal “uniform” generator.

The registered terminal co-primary endpoints are `pass@K`, the probability of finding any correct output, and raw `distinct@K`, the number of verified keys found. We report both separately for each model–domain–level block. The derived contrast `distinct@K – pass@K` = $\mathbb{E}[D_K - 1[D_K > 0]]$ is an interpretive decomposition: it estimates the number of verified modes beyond the first. It is computable without knowing the full valid set $\mathcal{C}_x$ and remains coupled to the number of successful draws. We use it as an interpretive decomposition rather than a fully success-matched diversity measure or a third primary endpoint. Because every headline evaluation fixes $K = 8$, reporting `distinct@8/8` would add no information beyond a linear rescaling; we retain the raw count and label its units explicitly.

B.2 WHY BALANCE WITHIN THE REPLAY BANK

Uniform sampling over keys prevents discovery frequency from determining replay frequency. To explain its relationship to the evaluation metric, we first analyze an equal-weight categorical surrogate, without length normalization. A stored response’s probability need not equal its key’s total probability. For $C = |\mathcal{B}_x|$ banked keys, write $p(c)$ for their probabilities and q_B for their total mass. Then

$$-\frac{1}{C} \sum_{c \in \mathcal{B}_x} \log p(c) = -\log q_B + \log C + \text{KL}(U_C \| p(c) \mid c \in \mathcal{B}_x)). \quad (8)$$

The objective rewards greater banked verified mass and a more balanced conditional distribution, aligning with `pass@K` and `distinct@K`. The combined update need not improve both at every step. Indeed banked-key coverage contributes $\sum_c [1 - (1 - p(c))^K]$ to expected `distinct@K`; at fixed q_B and C , concavity makes it largest under uniform allocation, while $1 - (1 - q_B)^K$, the probability of any banked key, depends only on total mass. This deliberate objective–metric alignment motivates treating held-out `pass@K` as co-primary. Uniformity also maximizes the smallest conditional banked-key mass, making equal retention explicit. Banks contain only training-prompt responses; evaluation responses never enter training.

C MODE COLLAPSE AND VERIFIED REPLAY

Under the explicit categorical assumptions below, centered binary-reward estimators amplify a pre-existing imbalance among correct modes, whereas persistent verified replay supplies a loss barrier for banked modes. The complete-response argument then states what would be needed to transfer that protection to a language model. These results concern idealized update rules; they do not certify the implemented AdamW/PPO trajectories.

C.1 CATEGORICAL ABSTRACTION AND MEAN UPDATES

Fix one isolated prompt and a finite category set $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$, with $m \geq 2$ correct execution modes and $n \geq 1$ incorrect categories. Assign one independently optimized logit to each category, including one logit per canonical correct mode, and set $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$. Thus P measures correctness and q the conditional distribution among correct modes. Logits are finite initially, $0 < P(0) < 1$, and explicit reference KL is zero. The collapse result uses Euclidean infinitesimal expected updates in these logits, without optimizer preconditioning, momentum, or another regularizer.

A language model induces probabilities over canonical outcomes, but this identification of probabilities does not identify its update geometry. Aggregating several response strings into a mode does not generally commute with gradient flow in response logits. One independent logit per canonical outcome is therefore an explicit abstraction, not a reduction proved for the neural policy or its validator equivalence classes.

Draw a group $A_1, \dots, A_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[A_i \in \mathcal{C}]$, and let $M = \sum_i R_i$. At the on-policy point where the PPO ratio equals one, consider the common-length estimator

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(M) \left(R_i - \frac{M}{G} \right) \nabla_z \log p_{A_i}. \quad (9)$$

For the Dr.GRPO abstraction, $w(M) = 1$ after absorbing the shared positive length scale. Reward-standardized GRPO with equal or common response-length normalization takes $w(M)$ to be the reciprocal group reward standard deviation on mixed groups (with a fixed numerical stabilizer if used). Standard variable-length GRPO also weights each response by its own inverse length (Shao et al., 2024; Liu et al., 2025); that factor cannot generally be absorbed into $w(M)$ and is outside the equivalence proved here. In either included case, $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. We define the entire update as zero on all-equal groups without evaluating an undefined reciprocal standard deviation; likewise, $w(M)M(G-M)$ below is defined as zero when $M = 0$ or $M = G$. All advantages and normalization weights are detached in the score estimator.

Lemma C.1 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected update in Equation (9) is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(M)M(G-M)]}{G^2 P(1-P)} > 0. \quad (10)$$

For the Dr.GRPO abstraction, $c_G(P) = (G-1)/G$. Group centering and reward standardization rescale the expected gradient without changing its direction under these assumptions.

Proof. Condition on the complete binary reward vector, whose sum is M . The score identities give

$$\mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 0] = \nabla_z \log(1-P).$$

There are M correct rows with centered reward $(G-M)/G$ and $G-M$ incorrect rows with centered reward $-M/G$. Therefore the conditional expectation of the unaveraged sum in (9) is

$$w(M) \frac{M(G-M)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(M) \frac{M(G-M)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $M \sim \text{Binomial}(G, P)$ yield (10). For $w \equiv 1$, $\mathbb{E}[M(G-M)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1) P^j (1-P)^{G-2-j}.$$

The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. This establishes the smooth, positive extension to $P = 0, 1$ used in the subsequent flow and convergence arguments.

The next lemma specializes the practical-estimator analysis of Tajwar et al. (2026, Section 4.3 and Appendix D, Theorem 5) to our categorical notation; we include the derivation for completeness.

Lemma C.2 (Binary MaxRL has the same correctness-gradient direction). *Under the same categorical and common-length assumptions, let MaxRL assign $A_i^{\text{MaxRL}} = GR_i/M - 1$ when $M > 0$ and zero when $M = 0$, as in Section 3.1. For $\hat{g}_G^{\text{MaxRL}} = G^{-1} \sum_i A_i^{\text{MaxRL}} \nabla_z \log p_{A_i}$ at the on-policy point, the expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1-P)^{G-1}}{P} > 0, \quad (11)$$

with continuous endpoint values $c_G^{\text{MaxRL}}(0) = G-1$ and $c_G^{\text{MaxRL}}(1) = 1$.

Proof. Condition on $M = m > 0$. Each successful row has advantage $(G - m)/m$ and each failed row has advantage -1 . Using the same conditional score identities as above, the expected unaveraged sum is

$$(G - m)(\nabla_z \log P - \nabla_z \log(1 - P)) = \frac{G - m}{P(1 - P)} \nabla_z P.$$

After the outer factor $1/G$ and averaging over $M \sim \text{Binomial}(G, P)$,

$$\mathbb{E}[(G - M)\mathbf{1}\{M > 0\}] = G(1 - P) - G \Pr(M = 0) = G[(1 - P) - (1 - P)^G].$$

Substitution gives Equation (11). Its endpoint values follow by continuity (or the finite geometric-series identity), which also gives the stated continuous extension at both boundaries. $\square$

The same finite-series identity gives the potential used below:

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1 - P)^k}{k}. \quad (12)$$

In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, $\text{best}@k$ equals $\text{pass}@k$, so this is a harmonic mixture of sampling objectives for $k = 1, \dots, G - 1$. The upper limit is $G - 1$, not G : the implemented centered estimator drops the entire update on all-failure groups. Subtracting an unconditional score baseline even on those groups instead gives the order- G estimator. This distinction concerns the exact finite-group expectation at the on-policy point; it does not assert unbiasedness of subsequent clipped or adaptive-optimizer updates.

C.2 WINNER-TAKE-ALL COLLAPSE IN THE CATEGORICAL MEAN FLOW

Consider $\dot{z} = c_G(P) \nabla_z P$, with any of the smooth positive coefficients established above: Dr.GRPO, common-length reward-standardized GRPO, or the specified centered binary MaxRL estimator. This infinitesimal expected-update model removes sampling noise and retains independently trained Euclidean logits. The conditional dynamics reduce to the standard frequency-dependent replicator system with fitness $f_c(q) = q_c$ (Hofbauer & Sigmund, 1998). The result is a specialization to the group estimators, not a new general theorem about replicator dynamics. Related correctness-indifferent collapse analyses include Sinha et al. (2026); Lochab et al. (2026).

Theorem C.3 (Standard replicator specialization for binary-reward RL). *Under the independent-logit mean-flow assumptions above, start from finite logits with $0 < P(0) < 1$. If the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \rightarrow 1, \quad q_{c^*}(t) \rightarrow 1, \quad q_c(t) \rightarrow 0 \quad (c \neq c^*).$$

Thus these categorical mean flows converge to a single correct execution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward. This is an asymptotic statement within the specified update geometry, not a claim of finite-time support loss or a convergence guarantee for the stochastic neural optimizers used in our experiments.

Proof. The coefficient c_G is smooth and bounded on $[0, 1]$, and the softmax reward gradient is bounded. Thus the logit flow exists for all finite times and cannot develop infinite logits at finite time. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P) p_c(1 - P), \quad \dot{z}_a = -c_G(P) p_a P.$$

Also $\dot{P} = c_G(P) \|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s)) P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (13)$$

The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ; normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly uniform correct distribution is therefore a special initial condition. The theorem concerns limiting concentration: finite softmax logits already give positive probabilities at every finite time, so finite-time positivity alone is not a retention result.

Dependence on optimization geometry. For a smooth parameterization $z = z(\theta)$ with Jacobian $J_\theta = \partial z / \partial \theta$, Euclidean parameter flow instead induces $\dot{z} = c_G(P) J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization metric. Equal binary reward alone does not force this winner selection: the categorical natural-gradient reward flow can be represented by $\dot{z}_a = c_G(P)(R_a - P)$, which gives identical velocities to correct logits and preserves their ratios. Here $R_a = \mathbf{1}[a \in \mathcal{C}]$. Parameter count alone does not specify the optimization geometry. The empirical scale comparisons also change pretrained models and learning-rate schedules; they neither measure the geometry nor identify the cause of cross-scale differences. The theorem supplies a mechanism to test in controlled training; it cannot identify the training history or cause of concentration in a hosted model.

C.3 A REPLAY SIGNAL WHEN FRESH GROUPS SUPPLY NONE

Lemma C.4 (Fresh-group starvation and replay signal). *Fix the categorical policy above with correctness probability P , and draw $G \geq 2$ fresh categories independently. The probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

Every all-equal group has zero fresh task component for Equation (9) and the specified centered MaxRL estimator. A nonzero fresh task component therefore requires a mixed group, so its probability is at most h_G . This gives no lower bound on gradient magnitude and says nothing about other loss terms, clipping, or an optimizer step carrying momentum across successive updates.

For a nonempty fixed verified bank $\mathcal{B} \subseteq \mathcal{C}$ with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ and $\rho_w > 0$. Extend w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p),$$

which vanishes only at $p = \bar{w}$. In particular, if $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$, even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes and fixed $\rho_w > 0$, as $p \rightarrow \mathbf{e}_b$ at one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$. The replay norm therefore stays positive as fresh task-signal opportunities vanish.

Proof. The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$. A mixed group requires both a correct and an incorrect sample; the union bound gives the two bounds

GP and $G(1-P)$. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$, proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k-1)/k^2 = 1 - 1/k,$$

which gives the strictly positive limiting norm for $k \geq 2$. $\square$

The displayed replay vector averages the fixed bank. A stochastic exemplar draw has this vector only in expectation, and a nonzero logit coordinate can be annihilated by a degenerate neural Jacobian. Replay also need not increase the next mixed-group probability: $h_G(P)$ is largest at $P = 1/2$. The claim is that a banked target can supply a separate verified signal when the fresh task gradient is unavailable.

A bounded advantage based on fresh samples has a different boundary behavior. The next bound includes group-dependent detached scores and distinguishes sampled updates from gradients of an exactly evaluated entropy functional.

Lemma C.5 (Bounded sampled scores have no probability-independent boundary force). *In the same independent-logit model, condition on the training history, draw $A_1, \dots, A_G$ independently from p , and let a_i be any detached, possibly group-dependent advantages with $|a_i| \leq B$. To interpret the bound uniformly as $p_b \rightarrow 0$, assume a finite B independent of that limit. At the on-policy point define*

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (\mathbf{e}_{A_i} - p).$$

For every category b ,

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.

Proof. Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{A_i = b\} - p_b| = 2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

The bound compares the isolated logit-gradient components. It does not prove inevitable collapse, rule out retention by a bounded estimator, or guarantee that replay dominates the full update. An unsampled mode can still change through softmax normalization or shared parameters. Exact entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero; the lemma is not an impossibility result for entropy regularization. Nor does the absence of a mode from a finite sample establish zero probability.

C.4 FIXED-BANK REPLAY AND QUALITATIVE RETENTION

Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (14)$$

This is ordinary cross-entropy against the distribution that assigns mass $1/k$ to each banked mode and zero elsewhere. It is not conditioned on the bank: decreasing (14) can raise total bank mass as well as redistribute mass within the bank.

In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting; it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P) \nabla_z P - \rho \nabla_z R_{\mathcal{B}}(z). \quad (15)$$

Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas C.1 and C.2, c_G has a finite positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G - 1)/G$ and MaxRL has $c_G = [1 - (1 - P)^{G-1}]/P$. Define

$$F_{\mathcal{B}}(z) = \rho R_{\mathcal{B}}(z) - \Psi(P(z)). \quad (16)$$

Then (15) is exactly gradient flow $\dot{z} = -\nabla_z F_{\mathcal{B}}$.

Theorem C.6 (Verified replay prevents banked-mode extinction). *Suppose logits are finite at time T and thereafter the nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective replay dose ρ . Under (15), define*

$$C_T = R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho}.$$

Then for every $b \in \mathcal{B}$ and every $t \geq T$,

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (17)$$

Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including asymptotically, even when the bank does not cover every correct mode. This is stronger than finite-time positivity, which already follows from finite softmax logits without replay.

Proof. The smooth vector field has bounded velocity because c_G , $\nabla_z P$, and $\nabla_z R_{\mathcal{B}}$ are bounded, so finite initial logits give a flow for every finite $t \geq T$. Along this flow,

$$\frac{d}{dt} F_{\mathcal{B}}(z(t)) = -\|\nabla_z F_{\mathcal{B}}(z(t))\|_2^2 \leq 0.$$

Since $P(t) \leq 1$ and Ψ is increasing,

$$R_{\mathcal{B}}(z(t)) \leq R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho} = C_T.$$

Every $-\log p_b$ is nonnegative and their sum is $kR_{\mathcal{B}}$. Therefore $-\log p_b(t) \leq kC_T$ for every banked mode, which is exactly Equation (17) for every time after T . $\square$

The floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not the replay coefficient alone.

A common nonnegative, locally integrable rate $\eta(t)$ multiplying both terms of (15) preserves the energy bound, since $\dot{F}_{\mathcal{B}} = -\eta(t)\|\nabla F_{\mathcal{B}}\|_2^2 \leq 0$. The convergence statements below additionally require $\int_T^\infty \eta(t) dt = \infty$ and then follow by reparameterizing optimization time. Infinitely many replay visits alone establish neither this condition nor a fixed replay-to-fresh weighting. Different time-varying relative weights are outside the fixed-potential argument.

Full coverage and the categorical target.

Corollary C.7 (Conditional full-coverage categorical limit). *Assume the conditions of Theorem C.6 and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Let $u_c = 1/m$ on $\mathcal{C}$. Then the categorical mean flow converges to*

$$P(t) \longrightarrow 1, \quad q(t) \longrightarrow u.$$

Consequently

$$\min_{c \in \mathcal{C}} q_c(t) \longrightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \longrightarrow \log m,$$

and conditional correct-mode collapse is impossible.

Proof. For full coverage,

$$R_C = \log m - \log P + \text{KL}(u||q). \quad (18)$$

The potential is bounded below by $-\Psi(1)$, so its descent gives $\int_T^\infty \|\nabla F_C\|_2^2 dt < \infty$. The finite-group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded. Thus the Hessian of F_C and the velocity are bounded; the gradient along the trajectory is uniformly continuous. Square integrability then implies $\nabla F_C \rightarrow 0$; recurring gradients of fixed positive size would otherwise contribute energy over intervals of a uniform minimum length.

Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness of this limit establish convergence of the probability vector. $\square$

Positive weights and the implemented length normalization. Uniformity of the target is separate from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w = -\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho_w}.$$

If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w||q)$, and the independent-logit limit is $P \rightarrow 1$, $q \rightarrow w$. The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate equations forces $P = 1$, then each equation forces $p_c = w_c$.

A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$, so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-bank or stochastic neural training.

This distinction matters for the implemented mean-token exemplar loss. Even if each key were represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

Its effective categorical replay coefficient would be $\rho_w = \rho_{\text{LM}}A$. The full-coverage limit would therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual language model, $\pi_\theta(e_b | x)$ is only one contribution to the total key probability. Equal averaging of per-token exemplar scores does not imply uniform learned key probabilities; the categorical uniformity corollary is not that claim.

Full support is an additional premise, not a consequence of replay. In Python factors, prompts admit 16–3,600 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects only admitted banked modes; it supplies no probability floor for an unbanked key.

C.5 COMPLETE RESPONSES AND FINITE EVALUATION VISIBILITY

The learner scores a particular response string, while evaluation groups strings by execution key. Event containment provides a conditional bridge: one complete verified response is one way of producing its key. This bridge also makes the loss’s length factor explicit. Here $p_\theta(b | x) = \Pr_{Y \sim \pi_\theta(\cdot|x)}[V(Y, s_x) = b]$ is the unconditional probability of producing key b , rather than the probability conditional on correctness.

Lemma C.8 (Retention from a joint exemplar potential). *Fix finitely many verified complete-response exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j | x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$, every protected exemplar satisfies

$$p_\theta(b_j | x_j) \geq \pi_\theta(e_j | x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

Proof. The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative, so $-\log \pi_\theta(e_j | x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key b_j , which proves the stated lower bound on every protected key’s probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \Psi_x(1)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them. Its bound applies only to the listed exemplar–prompt pairs. Shared parameters do not extend it to held-out prompts; transfer to those prompts remains an empirical question.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or a score under another decoding law does not supply this bound without a separate event or likelihood comparison. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K \delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero (Clopper & Pearson, 1934). For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

C.6 SCOPE RELATIVE TO ENTROPY, ADMISSION, AND NEURAL OPTIMIZATION

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q||u), \quad R_u(p) = \log m - \log P + \text{KL}(u||q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$

have the unique global optimum $P = 1, q = u$ in probability space on the simplex with $P > 0$, for positive coefficients. This is a boundary limit for finite-logit softmax policies. The optimum follows from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets (Pereyra et al., 2017; Agarwal et al., 2021): replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions (Mei et al., 2020); our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes. In particular, the fixed-weight argument does not cover a target whose weights continue to change with fresh observation counts.

For a fixed joint potential, a discrete bound $F(\theta_{n+1}) \leq F(\theta_n) + \epsilon_n$ with $\epsilon_n \geq 0$ and $\sum_{n \geq n_0} \epsilon_n = E < \infty$ would extend Lemma C.8, using $D = F(\theta_{n_0}) + J_{\max} + E$ for steps $n \geq n_0$. This follows by telescoping and bounds retention, not convergence. The noisy optimizer would need a separate argument establishing such control of the joint loss’s upward drift. A positive replay coefficient or small nominal learning rate does not establish it. Adam moments, clipping, momentum, prompt scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not established the required joint-energy or update-error assumptions. The mathematical contribution is the stated categorical mechanism and conditional exemplar argument; empirical support and longitudinal exemplar scores remain separate measurements.

D BENCHMARK AND EVALUATION PROTOCOL

MODEBENCH defines correctness and identity through the same executable validator: invalid outputs receive reward zero and no key; valid outputs receive reward one and a canonical execution key. Neither the complete valid-key catalogue nor its size is supplied to training, replay, scheduling, or stopping. Table 3 specifies the five domains and which surface forms merge. The public dataset preserves each domain’s split roles and construction identity. This paper’s training comparisons use Levels 1 and 2; the separate Level-3 release does not imply completed Level-3 training results.

Table 3: **MODEBENCH.** Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt, exact over the released multi-answer split. Every domain trains on 384 prompts and evaluates on a fixed 128-prompt split; the exact prompt each policy receives is reproduced verbatim in Appendix E for a direct, byte-level audit of the evaluation interface.

Domain	Generated answer / check	Executed key	Aliases that merge	valid modes
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19

D.1 SPLIT IDENTITIES AND EXECUTABLE VALIDATION

Every domain uses 384 training and 128 disjoint evaluation prompts. Python materialization independently executes at least two functions with different valid keys for every row; MathIR executes every certified route through the same interpreter used for model outputs.

Table 4: Deterministic materialization identities. Hashes bind the released rows to their executable checks; the full hashes and construction records accompany the data.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96
Gold catalogue supplied to policy	no	no
Training-bank initialization	empty	empty

PantryPlan. The deterministic v2 release has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8 to 45; the 128 evaluation prompts contain 2,328 feasible supports, averaging 18.1875. Training and evaluation row hashes are a4883c5e...a757f and 5d81ba8d...5923. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

D.2 PYTHON EXECUTION CONTRACT

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean `and/or`, unary plus/minus/not, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and other identifiers are rejected. Task specifications contain two to eight distinct integers in $[4, 1000]$ and at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm; the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield $\perp$ and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For every frozen input n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor:  $d_1, \dots, d_m$` ; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

D.3 MATCHED LEVEL-2 CONSTRUCTION

Level 2 changes problem structure while preserving the verifier and canonicalizer. Its 384/128/128 training/development/evaluation rows have disjoint identities across the Level-1 and Level-2 splits. Training histograms match the native Level-1 training sets. Development and evaluation histograms match designated Level-1 development and confirmation reserves in Graph, Countdown, Python, and MathIR; Pantry uses native references, with its 64-row development histogram doubled to 128 rows.

These construction reserves are not the native Level-1 terminal test sets. The displayed terminal evaluations have different valid-key histograms between levels in Graph, Countdown, and Python. Cross-level changes therefore combine task structure and test population; they do not isolate difficulty at fixed test-set support. Replay comparisons remain paired within each level on the same prompts, seeds, fresh objective, and training budget.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

Before treatment training, frozen Qwen2.5-0.5B development evaluation admitted a Level-2 domain only when `pass@8` lay in $[\cdot10, \cdot90]$ and was strictly below its construction-reference Level-1 value under identical decoding. The matched-level identity manifest records support histograms, row hashes, construction seeds, disjointness checks, and these admission receipts.

Scale-calibrated Levels 3–5. These levels use frozen Qwen2.5-3B, 7B, and 14B, respectively, against the same fixed Qwen2.5-0.5B Level-1 reference. Recipes are selected on development problems before fresh held-out confirmation. Admission requires absolute `pass@1` and `pass@8` differences of at most $\cdot04$ and $\cdot08$, respectively, in every domain, with four independent groups of eight samples per prompt. `Distinct@8` is reported but is not a fitting objective. The interface uses native chat, temperature 1, top- p 1, and 192 generated tokens: boxed-direct answers for Graph and the established hybrid solver with legal-syntax guidance for the other domains. This is an empirical match to fixed measured targets, not a two-sided equivalence test. The admitted Level-3 V3 release uses its original prompts; Levels 4 and 5 remain under calibration. Their development tiers are candidate recipes, not additional benchmark levels.

A subsequent Python Level-3 interface removes the strategy hints (Appendix K); its difficulty is not certified by that admission. A separate dataset revision is being calibrated under this neutral interface, varying numeric cases while preserving the exact Level-2 valid-mode-count histogram in each split. Development fitting precedes fresh held-out confirmation against the original fixed Level-1 success rates, with the same $\cdot04$ and $\cdot08$ tolerances. The historical reference retains its original wording, so this is a numerical match across specified interfaces. Existing results remain attached to the original datasets and prompts; the revised neutral dataset has no admitted result yet.

D.4 TRAINING AND EVALUATION SETTINGS

Table 5: **Matched run contract.** Settings and systems changes are shared by methods within each model–domain–seed comparison, including the zero-weight replay control.

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .

Base model	Pinned revision
Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfd
Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

Responses are independent within each evaluation draw. For each training seed, metrics average over all 128 prompts and four draws; raw responses and draw-level records are retained. The co-primary terminal outcomes are `pass@8` and `raw distinct@8`; `mean@8` is diagnostic. Evaluation never supplies training data, bank entries, checkpoint selection, or a stopping rule. Complete paired five-seed blocks receive nominal pointwise 95% Student- t intervals; partial primary intersections report exact counts and descriptive means without intervals. Intervals condition on the frozen prompts and evaluation requests. Domain averages are explicitly labeled, and no missing terminal result is replaced by a shallower checkpoint. Source admission and curve gaps follow Appendix M.1; comparison-specific intersections and the replay-weighting bootstrap are specified with their results.

E EXACT DOMAIN PROMPTS

Each domain uses one system message and one user message, with no few-shot examples. The same template renders training and evaluation for every method. These are the first rows of the frozen evaluation splits, including the exported chat-template wrapper. The checked generator `ops/check_paper_domain_prompts.py` reproduces them from the datasets. A two-space continuation indent marks display wrapping: joining it to the previous line with one space recovers the original string. The PantryPlan listing retains its real newlines.

E.1 GRAPH COLORING

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/graph_coloring_modebench_v2`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3), (3,5),
(3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no edge has the
same color on both endpoints. A partial coloring is 2??1?2, where ? means uncolored.
Keep every shown digit fixed. There are exactly 3 question marks. Return exactly one
```

```

final answer and no explanation: exactly 3 digits, each one 1, 2, or 3, for the missing
positions from left to right, inside \boxed{<|im_end|>
<|im_start|>assistant

```

E.2 COUNTDOWN

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/exact_countdown_easy3_probe.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use each
given number exactly once. You may use +, -, *, /, and parentheses. Return exactly one
final answer and no explanation: the expression inside \boxed{<|im_end|>
<|im_start|>assistant

```

E.3 PYTHON FACTORS

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/python_factor_modebench_v1.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external Python tool
will call it once for each n in [18, 82, 91, 93]. For every call, return an integer d
satisfying 1 < d < n and n % d == 0. Different valid return vectors are different
solution modes. You may use integer literals, n, +, -, *, //, %, comparisons, Boolean
operators, and conditional expressions; calls, imports, attributes, containers, and
other names are forbidden. Return exactly the one-line lambda inside \boxed{<|im_end|>
explanation.<|im_end|>
<|im_start|>assistant

```

E.4 MATHIR ACTION MENU

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/mathir_action_menu_v1.

```

<|im_start|>system
Return only the final answer inside \boxed{<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed by
exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c. Action
menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E means
sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations, repeated
states, and paths that do not finish with x isolated are rejected. Return only the IDs
separated by semicolons inside the answer box; for example formatting only: \boxed{A;C}.
Do not return x's numeric value.<|im_end|>
<|im_start|>assistant

```

E.5 PANTRYPLAN

Template qwen_pantry_support_mask; 384 training and 128 evaluation
rows in var/data/pantry_plan_modebench_v2.

```

<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the corresponding
Pantry row in printed order. A trusted environment will choose exact quantities on
precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:

```

```

- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal, protein=21.5g,
  fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal, protein=0.74g,
  fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal, protein=0.941g,
  fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant

```

F REPLAY ALGORITHM

For each prompt x , $\mathcal{B}_x$ stores at most M verified policy exemplars, one per canonical key; the serialized cursor τ selects banks for replay. The same update supports either fresh objective.

F.1 FRESH TASK ADVANTAGES

For a group of G fresh responses with verified binary rewards r_i and $m = \sum_i r_i$, Dr.GRPO uses $A_i = r_i - m/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} Gr_i/m - 1, & m > 0, \\ 0, & m = 0. \end{cases}$$

The centered MaxRL estimator (Tajwar et al., 2026) gives failures advantage -1 in mixed groups and weights each success more strongly when successes are scarce. All-correct and all-failure groups have zero task advantage. Lemma C.2 gives its harmonic pass@ k mean objective. These advantages apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

F.2 STATE AND OPTIMIZER UPDATE

Algorithm 1 One ReplayMaxRL optimizer update

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \tau)$

Ensure: Atomically updated $(\theta', \mathcal{B}', \tau')$

- 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward r_i
- 4: Set $c_i \leftarrow \tilde{c}_i$ iff $r_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 5: **end for**
- 6: Compute fresh task advantages A_i^o from $\{r_j\}_{j=1}^G$; for ReplayMaxRL use the formula in Appendix F.1
- 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
- 8: $(z, \tau') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \tau)$ by a fixed round-robin cursor
- 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
- 10: For each retained exemplar e_c , compute $s_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} s_c$
- 12: **else**
- 13: $L_{\text{replay}} \leftarrow 0$
- 14: **end if**
- 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$
- 17: Checkpoint $(\theta', \mathcal{B}', \tau')$ as one atomic state

F.3 ADMISSION, CAPACITY, AND DETERMINISTIC SCHEDULING

A bank is keyed by the SHA-256 digest of the exact prompt-token sequence. Only loss-active rows with positive reward and a nonempty validator key qualify. The full group is validated against the pre-group bank before any new key is committed. For each new key, the lexicographically smallest normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is therefore invariant to rollout-row order. The bank keeps one exemplar per key, up to $M = 16$, with no eviction or replacement. Later keys may enter diagnostics but not replay support when capacity is full. Prompt-hash collisions or changed tokens under an existing hash are hard errors.

At each update, the scheduler sorts nonempty prompt-bank hashes and selects one by a fixed round-robin cursor; singleton banks are eligible. The cursor is independent of reward, evaluation, desired support, or entropy, and is serialized with the bank. Stored prompt and response tokens are materialized unchanged into a right-padded teacher-forcing batch. Prompt and padding positions are masked. Each exemplar contributes its mean response-token log probability, averaged uniformly across keys in the selected bank before applying λ_{replay} and the exact $(G - 1)/G^2$ rollout scaling in Algorithm 1. Neither exhaustive support nor evaluation outputs enter this loss.

F.4 MATCHED CONTROLS AND RECOVERABLE STATE

Controls share validation, bank updates, scheduling, materialization, and the score pass, with the replay score-gradient vector replaced by exact zeros. Treatment activates only that derivative. Policies can subsequently differ in samples, bank contents, selected banks, exemplar lengths, and realized computation. Logs retain bank/key counts, replayed token counts, raw and applied losses, score-gradient summaries, cursor phase, and the exact-zero switch. Model and optimizer state, bank counts and exemplar tokens, replay and data cursors, and request-stream state are checkpointed atomically; the loader validates schema, capacity, scheduler and estimator constants, token types, prompt hashes, key/exemplar consistency, and cursor bounds before accepting a resume.

F.5 FIXED SEMANTIC MAXENT COMPARATOR

Fixed Semantic MaxEnt is an entropy-inspired fresh-rollout comparator. For validator-positive canonical outcomes, an empirical predictor combines successful policy history and successful leave-

one-out peers. If K keys have total count N , unit pseudocounts and one reserved unseen bucket give

$$\hat{p}(a) = \frac{n(a) + 1}{N + K + 1}, \quad \hat{p}(\text{UNSEEN}) = \frac{1}{N + K + 1}. \quad (19)$$

The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}(a_i), C\} - \mathbb{E}_{\hat{p}}[\min\{-\log \hat{p}(a), C\}]}{C}, \quad \eta = .10, C = 5. \quad (20)$$

Invalid and unsuccessful rows receive zero. The centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome present in a fresh rollout, but supplies no direct reinforcement to an absent remembered exemplar. Its heterogeneous empirical effects therefore remain a comparison with exploration rather than a retention guarantee.

For the true conditional distribution $q_\theta = p_\theta(\cdot \mid C)$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (21)$$

This motivates the direction of redistribution, not an unbiased-gradient claim for the empirical estimator. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{A \sim p} [\mathbf{1}\{A \in C\} (s_A - b) \nabla_z \log p_A] = P \mathbb{E}_{c \sim q} [s_c \nabla_z \log q_c] + (\mathbb{E}_q[s] - b) \nabla_z P.$$

A historical baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla H(q)$. For a single correct key, conditional entropy is zero, yet the implemented unseen bucket can give that key negative semantic advantage. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

G EMPIRICAL RESULTS AND COMPARISONS

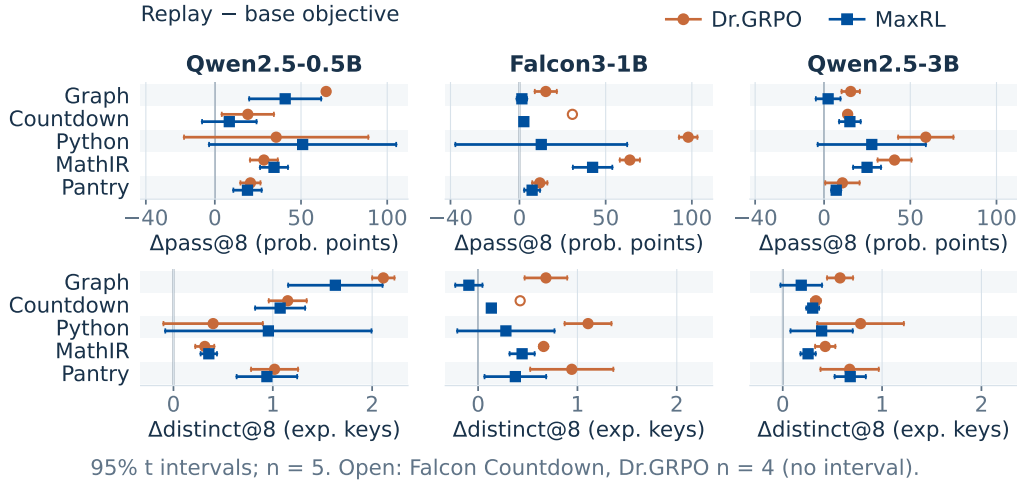


Figure 8: **Canonical replay complements both fresh objectives.** Each point compares replay with its matched no-replay control at pass eight, within domain and scale. The two endpoints have different units: success probability points and expected verified-key counts. All 75 MaxRL pairs are complete; Falcon Countdown uses four Dr.GRPO pairs and all other Dr.GRPO blocks use five. Nominal intervals accompany complete blocks only. Domain variation is retained; these comparisons identify the implemented replay addition rather than a universal benefit or factorial interaction.

G.1 COMPARISON REGISTRY

The primary comparisons cross fresh objective (Dr.GRPO or MaxRL) with verified replay, first across three model scales on Level 1 and then at Qwen2.5-0.5B on Level 2. Table 6 separates these comparisons from the supporting controls and replay-weighting ablation. Terminal populations use the September 11 frozen census. The registered Qwen2.5-0.5B weighting aggregate retains its original frozen population; its endpoint values are unchanged in that census.

Table 6: **Comparison scope and available terminal evidence.** A complete block has five registered paired seeds. Primary partial intersections retain their exact counts and receive no interval. UCPO and sparse RLEP-Dr are reported only at the two smaller model scales; their denominators cover all five benchmark domains and five registered seeds at each selected scale.

Comparison	Scope	Terminal coverage	Question
Dr.GRPO / ReplayDr.GRPO	Level 1, all three scales	74/75 pairs; 14/15 complete blocks	Verified memory beyond fresh reward
MaxRL / ReplayMaxRL	Level 1, all three scales	75/75 pairs; 15/15 complete blocks	Memory beyond a stronger fresh objective
Four-method factorial	Level 2, Qwen-0.5B	4/5 complete domains; Pantry pairs: two Dr.GRPO, one MaxRL	Within-level replay replication
GRPO	Level 1, all three scales	75/75 runs	Reward-standardization control
UCPO	Level 1, 0.5B and 1B	50/50 runs	On-policy advantage redistribution
Sparse RLEP-Dr	Level 1, 0.5B and 1B	47/50 runs	Success memory without uniform key balancing
Fixed Semantic MaxEnt	Level 1, registered scale/domain blocks	10/15 complete blocks	Fresh-rollout semantic exploration
Uniform / frequency replay	Level 1, nine registered blocks	44/45 treatment cells; 8/9 complete blocks	Contribution of replay-key balancing

G.2 TERMINAL COMPARISONS ACROSS SCALES

Within each model–domain–seed comparison, methods share initialization, data identities, the eight-pass horizon, evaluation requests, verifier, and replay scheduling code. The replay control runs the same bank and scoring path with zero replay derivative. Resulting policies can build different banks. The admissible intersections are defined in Appendix M.1.

Table 7: **Matched results after eight training passes.** Columns compare Dr.GRPO with ReplayDr.GRPO after exactly eight training passes. Entries are means over the displayed paired-seed intersection; n @pass gives its size and training pass. P, M, and D denote $\text{pass}@8$, $\text{mean}@8$, and $\text{distinct}@8$. All source-admissible terminal pairs are included: Falcon Countdown has $n = 4$, other blocks $n = 5$ (Appendix M.1). No missing endpoint is imputed. Pale green marks a ReplayDr.GRPO mean that is strictly higher than its matched Dr.GRPO mean.

Model	Domain	n @pass	Dr.GRPO			ReplayDr.GRPO		
			P	M	D	P	M	D
Qwen2.5-0.5B	Graph coloring	5@8.0	.323	.323	.325	.969	.404	2.438
	Countdown	5@8.0	.480	.465	.487	.672	.597	1.637
	Python factors	5@8.0	.172	.172	.172	.528	.507	.571
	MathIR	5@8.0	.512	.464	.513	.796	.688	.829
	PantryPlan	5@8.0	.522	.522	.522	.729	.545	1.539
Falcon3-1B	Graph coloring	5@8.0	.656	.300	1.034	.809	.334	1.717
	Countdown	4@8.0	.325	.267	.415	.632	.343	.838
	Python factors	5@8.0	.000	.000	.000	.979	.722	1.107
	MathIR	5@8.0	.095	.068	.097	.736	.292	.756
	PantryPlan	5@8.0	.633	.633	.633	.751	.560	1.576
Qwen2.5-3B	Graph coloring	5@8.0	.682	.360	1.060	.836	.349	1.637
	Countdown	5@8.0	.609	.569	.628	.746	.505	.962
	Python factors	5@8.0	.069	.069	.069	.659	.603	.852
	MathIR	5@8.0	.278	.168	.279	.686	.580	.706
	PantryPlan	5@8.0	.638	.558	.841	.744	.546	1.514

Dr.GRPO retention contrasts. Both co-primary block means are positive in all 14 complete five-seed comparisons, and raw $\text{distinct}@8$ increases in all 74 admissible pairs. At Qwen2.5-0.5B, Countdown $\text{pass}@8$ rises from .480 to .672 and raw $\text{distinct}@8$ from .487 to 1.637; the extra-mode effect is +.959 (paired 95% interval $[+.863, +1.054]$). Python correctness rises from .172 to .528, with an extra-mode effect of +.043 ($[-.077, +.164]$); MathIR rises from .512 to .796, with extra modes +.030 ($[+.001, +.060]$). These intervals are nominal, unadjusted Student- t estimates over five paired training seeds. Extra modes remain coupled to correctness. Countdown keys identify computation routes, but these held-out endpoints do not identify which previously discovered routes survived training.

Figure 9 resolves the domain-specific results behind the main cross-domain display. All five 3B MaxRL domain comparisons are complete: both arms have seeds 70–74 at the registered terminal endpoint (50/50 runs). The Dr.GRPO pair uses five seeds except Falcon Countdown, which uses four. These populations remain separate: a factorial interaction requires the common four-arm intersection, not differences of aggregates with different seeds.

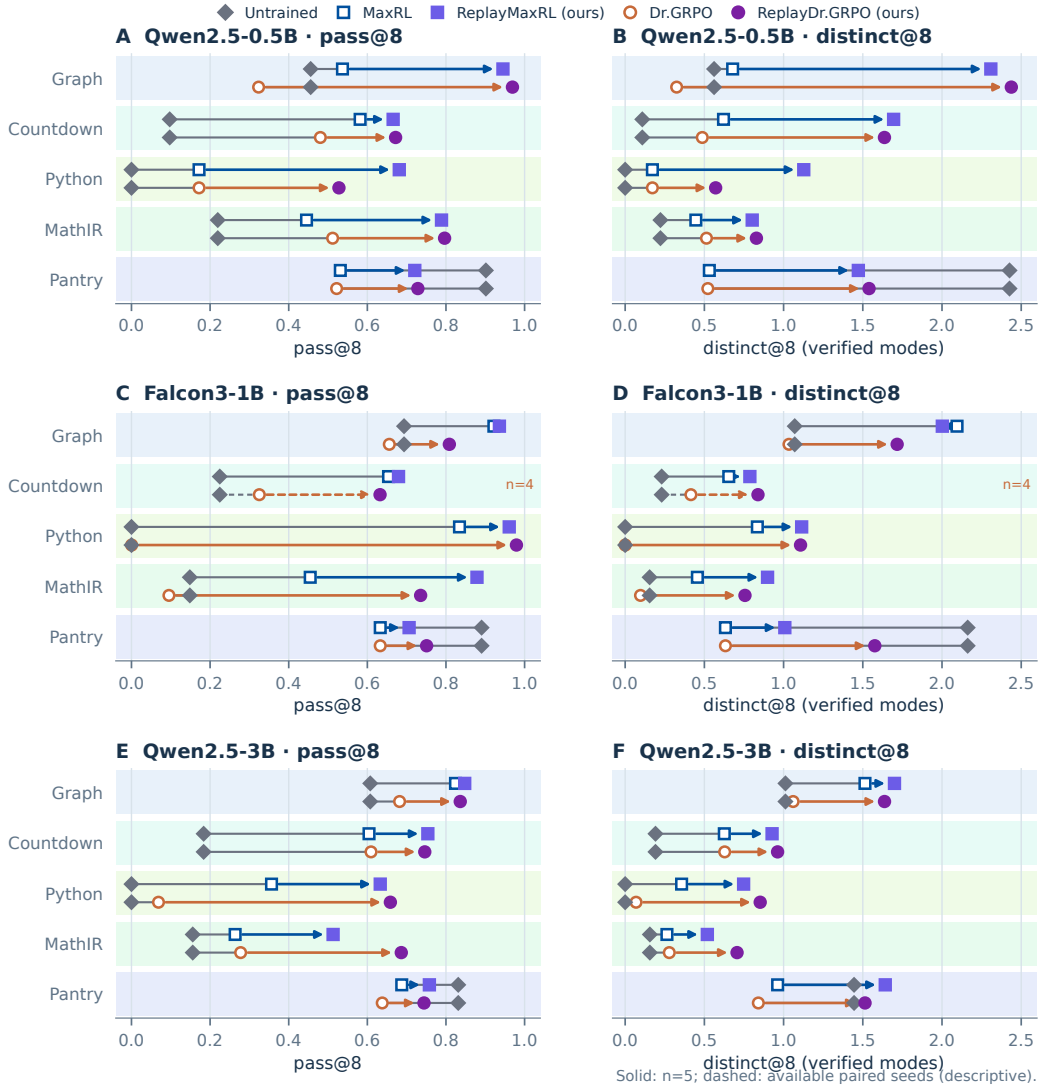


Figure 9: **Per-domain results behind the cross-domain averages in Figure 5.** Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show correctness and raw support. MaxRL–ReplayMaxRL pairs retain five terminal seeds throughout the two smaller models and Qwen2.5-3B: every MaxRL domain uses $n = 5$. Solid connectors identify these complete comparisons. Dr.GRPO–ReplayDr.GRPO uses four admissible seeds for Falcon Countdown and five elsewhere. Domain tints match Figure 2. Replay arrows, trained-arm markers, and untrained references use the same encoding as Figure 5. The E118 model index groups the archived exports by model and domain.

Write $P = \text{pass@8}$, $D = \text{distinct@8}$, and $B = D - P$. The derived B counts verified modes beyond the first correct outcome in each sampled set and remains coupled to correctness. The endpoint tables report exact paired terminal effects. A dash denotes no observed pair; partial means retain $n/5$ without an interval. Complete-block intervals are nominal 95% estimates, without multiple-comparison or repeated-look adjustment. All underlying arm means and seed effects remain in the adjacent machine-readable records.

Table 8: **ReplayMaxRL effects across model scales at pass 8.** All effects are ReplayMaxRL minus MaxRL on the displayed paired seeds. Brackets give paired Student- t intervals for ΔB in complete blocks. The machine-readable report also retains both arm means, all per-seed effects, and intervals for P and D .

Model	Domain	Replay arm	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	ReplayMaxRL	5/5	+0.408	+1.630	+1.221 [+0.926, +1.517]
Qwen-0.5B	Countdown	ReplayMaxRL	5/5	+0.084	+1.074	+0.990 [+0.859, +1.121]
Qwen-0.5B	Python	ReplayMaxRL	5/5	+0.509	+0.955	+0.446 [-0.156, +1.048]
Qwen-0.5B	MathIR	ReplayMaxRL	5/5	+0.343	+0.356	+0.013 [+0.008, +0.017]
Qwen-0.5B	PantryPlan	ReplayMaxRL	5/5	+0.189	+0.941	+0.751 [+0.520, +0.982]
Falcon-1B	Graph	ReplayMaxRL	5/5	+0.014	-0.093	-0.107 [-0.220, +0.005]
Falcon-1B	Countdown	ReplayMaxRL	5/5	+0.025	+0.133	+0.108 [+0.082, +0.134]
Falcon-1B	Python	ReplayMaxRL	5/5	+0.127	+0.280	+0.153 [-0.272, +0.578]
Falcon-1B	MathIR	ReplayMaxRL	5/5	+0.424	+0.443	+0.019 [+0.006, +0.031]
Falcon-1B	PantryPlan	ReplayMaxRL	5/5	+0.073	+0.375	+0.302 [+0.034, +0.569]
Qwen-3B	Graph	ReplayMaxRL	5/5	+0.023	+0.186	+0.163 [+0.021, +0.305]
Qwen-3B	Countdown	ReplayMaxRL	5/5	+0.150	+0.302	+0.152 [+0.119, +0.185]
Qwen-3B	Python	ReplayMaxRL	5/5	+0.277	+0.391	+0.115 [+0.080, +0.149]
Qwen-3B	MathIR	ReplayMaxRL	5/5	+0.249	+0.255	+0.006 [+0.001, +0.011]
Qwen-3B	PantryPlan	ReplayMaxRL	5/5	+0.070	+0.679	+0.609 [+0.467, +0.751]

At Qwen2.5-0.5B, ReplayMaxRL adds 1.221, .990, and .751 extra modes on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph instead has a negative mean raw-support effect (-.093, interval [-.229, +.043]), and its correctness effect is inconclusive (+.014, [-.016, +.044]). Positive cross-domain effects therefore do not imply a breadth gain in every domain. All five Qwen2.5-3B blocks are complete. Their extra-mode effects are +.163, +.152, +.115, +.0059, and +.609 for Graph, Countdown, Python, MathIR, and PantryPlan; every nominal interval excludes zero, although MathIR’s breadth gain is small. Correctness intervals exclude zero for Countdown, MathIR, and PantryPlan; Graph and Python remain inconclusive.

Cross-domain factorial summaries. Averaging five domains equally within each paired seed, ReplayMaxRL minus MaxRL gives +.307 on `pass@8` and +.991 on `raw distinct@8` at Qwen2.5-0.5B (unadjusted 95% intervals [+ .208, +.406] and [+ .815, +1.167]), and +.133/+ .228 at Falcon3-1B ([+ .043, +.223] and [+ .094, +.361]). Both use five paired seeds. The Dr.GRPO replay effects on these two endpoints are +.337/+ .999 at Qwen2.5-0.5B, +.442/+ .774 at Falcon3-1B, and +.279/+ .559 at Qwen2.5-3B. The Falcon Dr.GRPO average uses four common admissible seeds without an interval; the other two use five. These are post-hoc descriptive cross-domain summaries. At Qwen2.5-3B, MaxRL/ReplayMaxRL arm means are .547/.701 on correctness and .744/1.107 on raw support. Equal domain averages within each of seeds 70–74 give replay effects of +.154 ([+ .086, +.222]) and +.363 ([+ .285, +.441]), respectively. The common four-arm intersection contains fourteen complete five-seed blocks and Falcon Countdown at four seeds. Factorial interactions require that intersection rather than differences between aggregates using different seeds.

G.3 TERMINAL COMPARISON ON THE HARDER LEVEL

Figure 6 uses Graph, Countdown, Python, and MathIR: each has all five seeds at step 3,072 for both levels and all four methods. Means average those four domains equally within each seed, then the five seed means. This is a descriptive 20-cell comparison, not a five-domain estimate; the levels have separate held-out prompts, with the population differences in Appendix D.3. At Level 2, PantryPlan has three Dr.GRPO, four ReplayDr.GRPO, one MaxRL, and two ReplayMaxRL terminal endpoints (10/20). Dr.GRPO/replay pairs are seeds 43 and 46; MaxRL/replay has seed 43, the only common four-arm seed. Independently available arm means do not estimate paired effects.

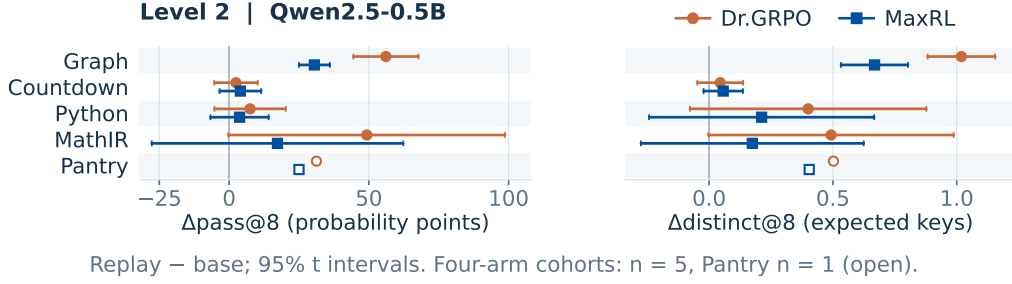


Figure 10: **Replay effects within Level 2.** Replay and control are compared on the same prompts and training seeds, with separate Dr.GRPO and MaxRL effects. Four domains have complete factorials; PantryPlan retains its partial counts and descriptive effects. Construction and admission are described in Section 2.2; terminal absolute values appear in Figure 6. Differences between levels combine task, population, and protocol changes, so they do not identify a causal effect of difficulty alone.

Table 9: **Replay effects on the harder Level-2 tasks at pass 8.** MaxRL and Dr.GRPO identify the fresh objective; each effect is its replay variant minus the corresponding objective without replay. Each row uses its own pairwise intersection, which can exceed the four-method intersection. Brackets give paired Student-*t* intervals for ΔB only when all five registered pairs are available.

Model	Domain	Objective	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Dr.GRPO	5/5	+0.561	+1.019	+0.458 [+0.394, +0.522]
Qwen-0.5B	Graph	MaxRL	5/5	+0.305	+0.668	+0.363 [+0.280, +0.446]
Qwen-0.5B	Countdown	Dr.GRPO	5/5	+0.025	+0.045	+0.020 [-0.014, +0.055]
Qwen-0.5B	Countdown	MaxRL	5/5	+0.040	+0.057	+0.017 [+0.001, +0.032]
Qwen-0.5B	Python	Dr.GRPO	5/5	+0.075	+0.400	+0.325 [-0.228, +0.878]
Qwen-0.5B	Python	MaxRL	5/5	+0.037	+0.212	+0.175 [-0.295, +0.645]
Qwen-0.5B	MathIR	Dr.GRPO	5/5	+0.493	+0.493	+0.000 [+0.000, +0.000]
Qwen-0.5B	MathIR	MaxRL	5/5	+0.173	+0.175	+0.002 [-0.003, +0.006]
Qwen-0.5B	PantryPlan	Dr.GRPO	2/5	+0.234	+0.367	+0.133
Qwen-0.5B	PantryPlan	MaxRL	1/5	+0.250	+0.404	+0.154

Level-2 Graph provides the clearest replay effects on correctness and extra modes under both objectives. Countdown’s ReplayMaxRL extra-mode effect is small (+.017, interval [+0.001, +0.032]), while its correctness and raw-support intervals include zero. Python and MathIR have positive mean correctness and raw-support effects with intervals including zero; MathIR’s extra-mode effects are near zero. Pantry’s two-seed Dr.GRPO replay contrast is +.234 on *P*, +.367 on *D*, and +.133 on *B*. Its one-seed MaxRL replay contrast is +.250, +.404, and +.154, respectively. Neither partial contrast receives an interval.

For each common seed and endpoint *Y*, the replay interaction is

$$I = (Y^{\text{ReplayMaxRL}} - Y^{\text{MaxRL}}) - (Y^{\text{ReplayDr.GRPO}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises *P* relative to Dr.GRPO by +.233 ([+.140, +.325]), while the replay interaction on *P* is −.256 [−.329, −.183]). This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 10: **Fresh-objective effects and replay interactions on Level 2.** All three contrasts within a domain use the same four-arm seed intersection. The interaction is (ReplayMaxRL – MaxRL) – (ReplayDr.GRPO – Dr.GRPO). Each entry gives its mean and, only for a complete five-seed block, a paired unadjusted 95% Student-*t* interval. Archived four-arm exports: E119 model index.

Domain	<i>n</i>	Contrast	ΔP	ΔD	ΔB
Graph	5	MaxRL – Dr.GRPO	+.233 [+ .140, +.325]	+.322 [+ .162, +.482]	+.089 [+ .019, +.159]
Graph	5	ReplayMaxRL – ReplayDr.GRPO	–.023 [–.062, +.016]	–.029 [–.198, +.140]	–.006 [–.142, +.130]
Graph	5	Replay $\times$ MaxRL interaction	–.256 [–.329, –.183]	–.351 [–.446, –.255]	–.095 [–.186, –.003]
Countdown	5	MaxRL – Dr.GRPO	–.014 [–.073, +.045]	–.014 [–.073, +.045]	+ .000 [+ .000, +.000]
Countdown	5	ReplayMaxRL – ReplayDr.GRPO	+ .002 [–.008, +.012]	–.002 [–.047, +.044]	–.004 [–.042, +.035]
Countdown	5	Replay $\times$ MaxRL interaction	+ .016 [–.047, +.078]	+ .012 [–.068, +.092]	–.004 [–.042, +.035]
Python	5	MaxRL – Dr.GRPO	+ .000 [+ .000, +.000]	+ .000 [+ .000, +.000]	+ .000 [+ .000, +.000]
Python	5	ReplayMaxRL – ReplayDr.GRPO	–.037 [–.232, +.157]	–.188 [–.988, +.613]	–.150 [–1.014, +.713]
Python	5	Replay $\times$ MaxRL interaction	–.037 [–.232, +.157]	–.188 [–.988, +.613]	–.150 [–1.014, +.713]
MathIR	5	MaxRL – Dr.GRPO	+ .283 [–.193, +.759]	+ .283 [–.193, +.759]	+ .000 [+ .000, +.000]
MathIR	5	ReplayMaxRL – ReplayDr.GRPO	–.036 [–.158, +.085]	–.035 [–.158, +.088]	+ .002 [–.003, +.006]
MathIR	5	Replay $\times$ MaxRL interaction	–.320 [–.864, +.225]	–.318 [–.860, +.224]	+ .002 [–.003, +.006]
PantryPlan	1	MaxRL – Dr.GRPO	+ .041	+ .061	+ .020
PantryPlan	1	ReplayMaxRL – ReplayDr.GRPO	–.021	–.037	–.016
PantryPlan	1	Replay $\times$ MaxRL interaction	–.062	–.098	–.035

Level-2 Graph endpoint detail. The development admission comparison lowers frozen-model pass@8 from .438 to .148. At pass 8, Dr.GRPO/MaxRL reach .202/.434, versus .763/.739 with their replay variants. The paired correctness gains are +.561 and +.305 (unadjusted 95% Student-*t* intervals [+ .445, +.677] and [+ .250, +.360], five seeds each). The weakest replay run (.711) exceeds the strongest non-replay run (.477). Replay adds 1.019/.668 raw distinct@8, yielding 1.243/1.214 under the two fresh objectives. These observations show improved correctness and sampled support on harder Graph; they do not establish that broader support causes the correctness gain. Cross-level prompts are distinct, and the four-domain Level-2 aggregate excludes the incomplete Pantry factorial.

G.4 ACCURACY AND VERIFIED MODES THROUGHOUT TRAINING

The following curves include every primary method and model scale used in the main paper. Each objective pair uses its domain-specific terminal seed intersection as one fixed cohort throughout training. A checkpoint is plotted only when all members have complete, valid evaluations for both methods. Missing or conflicting checkpoints leave gaps; seed counts never shrink to extend a mean. Four fixed-seed draws of eight samples on the same 128 held-out prompts define each evaluation. Accuracy and mode curves use identical seeds and checkpoints, without smoothing or outcome-based selection.

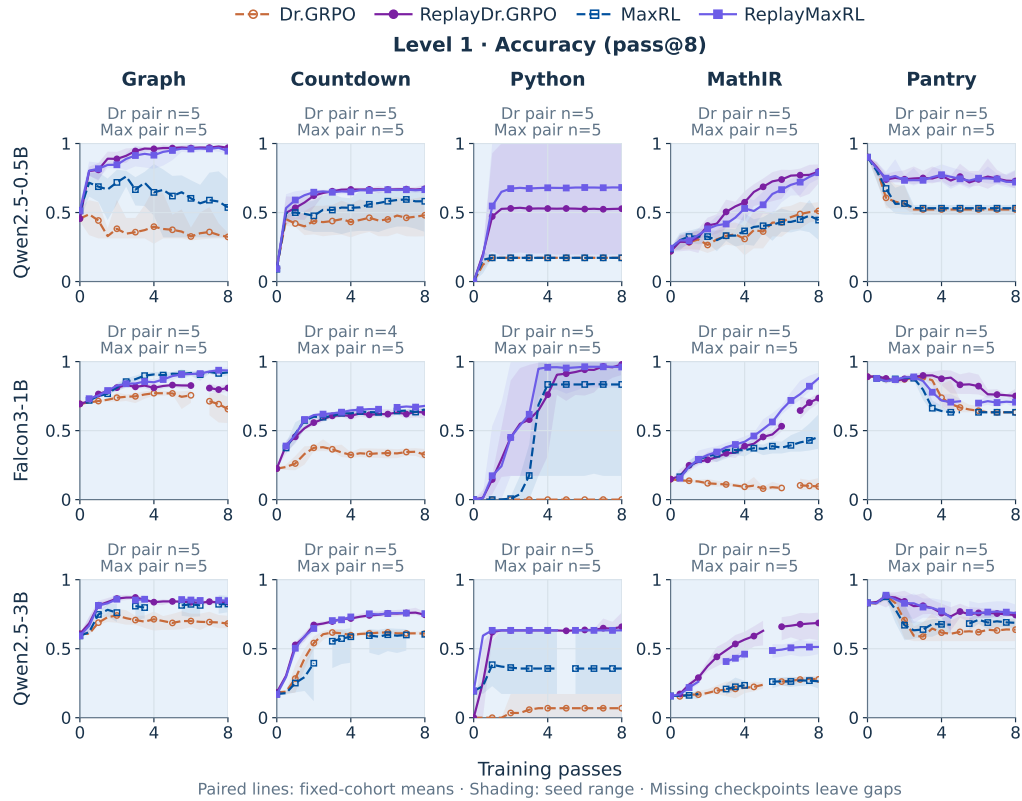


Figure 11: **Accuracy throughout training at all three model scales.** Rows are Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns are the five domains. Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL use the same method colors as Figure 5. Lines are fixed-cohort means of pass@8; bands are seed ranges, not confidence intervals. Counts identify each objective’s terminal paired seeds. Qwen2.5-3B MaxRL uses all five seeds in every domain; Falcon Countdown Dr.GRPO/replay uses four. Gaps indicate missing or conflicting checkpoints.

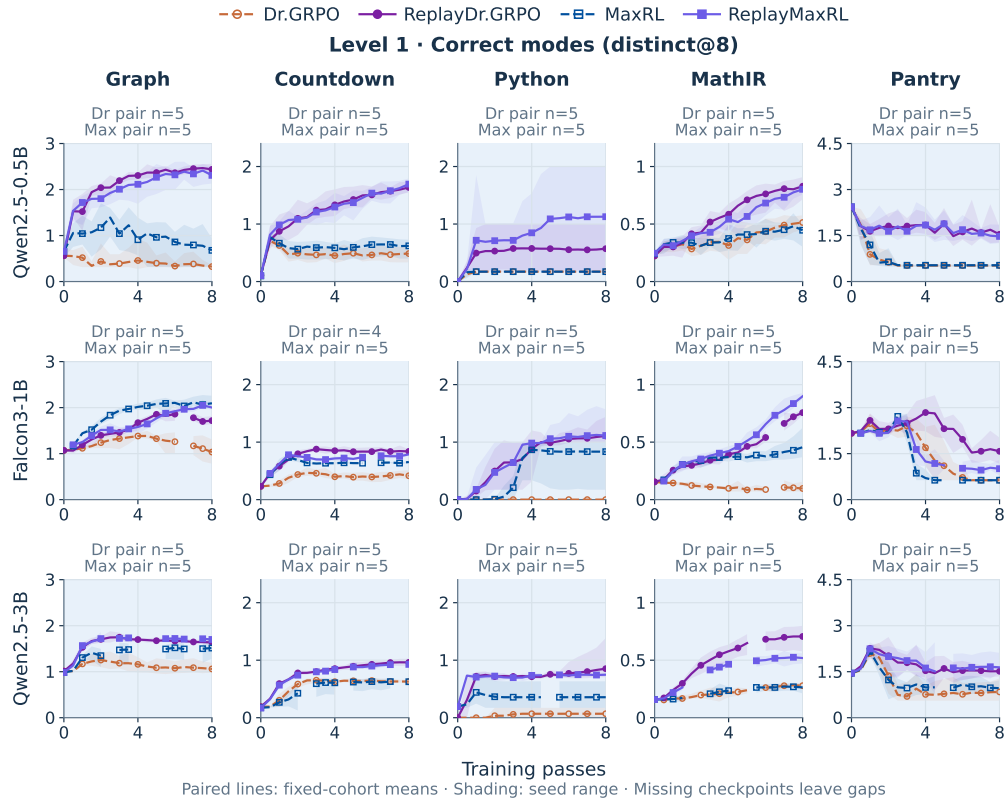


Figure 12: **Verified modes throughout training at all three model scales.** The same model rows, methods, seeds, and checkpoints as Figure 11, now measuring `distinct@8`. Lines are descriptive fixed-cohort means and bands show seed ranges. A count of one has no band. Domain-specific vertical scales are shared across model rows; absolute heights across different domains should be read against their labeled axes. Missing evaluations are never carried forward or joined across a gap.

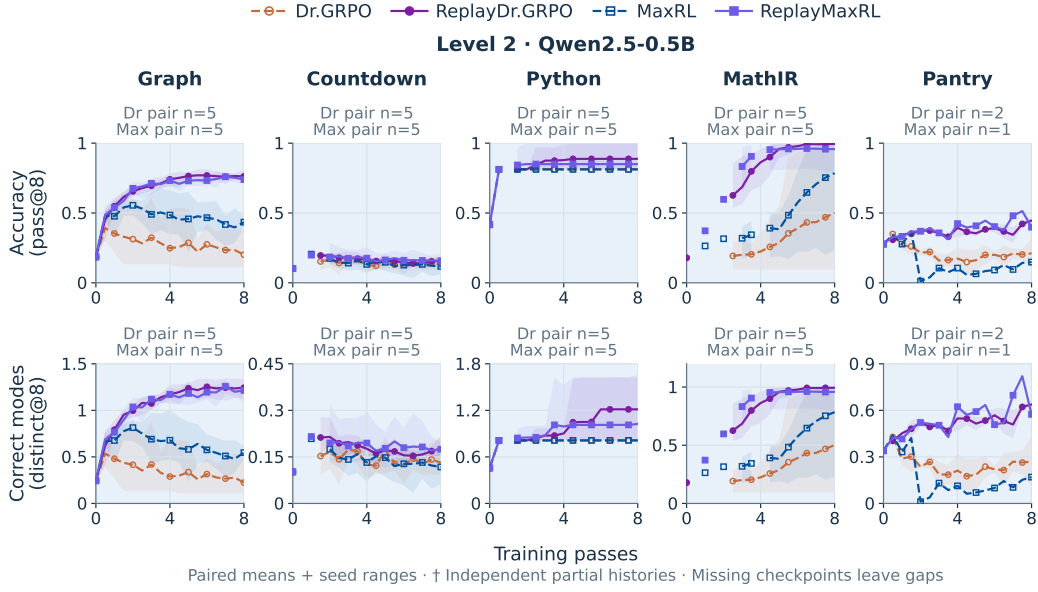


Figure 13: **Accuracy and verified modes during Level-2 training.** Qwen2.5-0.5B is the model tested at this level. Rows show `pass@8` and `distinct@8` for all four primary methods. Graph, Countdown, Python, and MathIR contain five terminal paired seeds per objective. PantryPlan uses two terminal Dr.GRPO/replay pairs and one MaxRL/replay pair. Additional unpaired histories are marked separately; they do not contribute to paired means. One-seed curves have no band. Both metrics follow the same evaluation rules, with gaps at missing checkpoints and no extrapolation.

G.5 SUPPORTING COMPARATORS

GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse RLEP-Dr replays verified successes without uniform canonical-key weighting. Fixed Semantic MaxEnt supplies a fresh-rollout exploration incentive (Appendix F.5). These are controls for alternative explanations of the retention results.

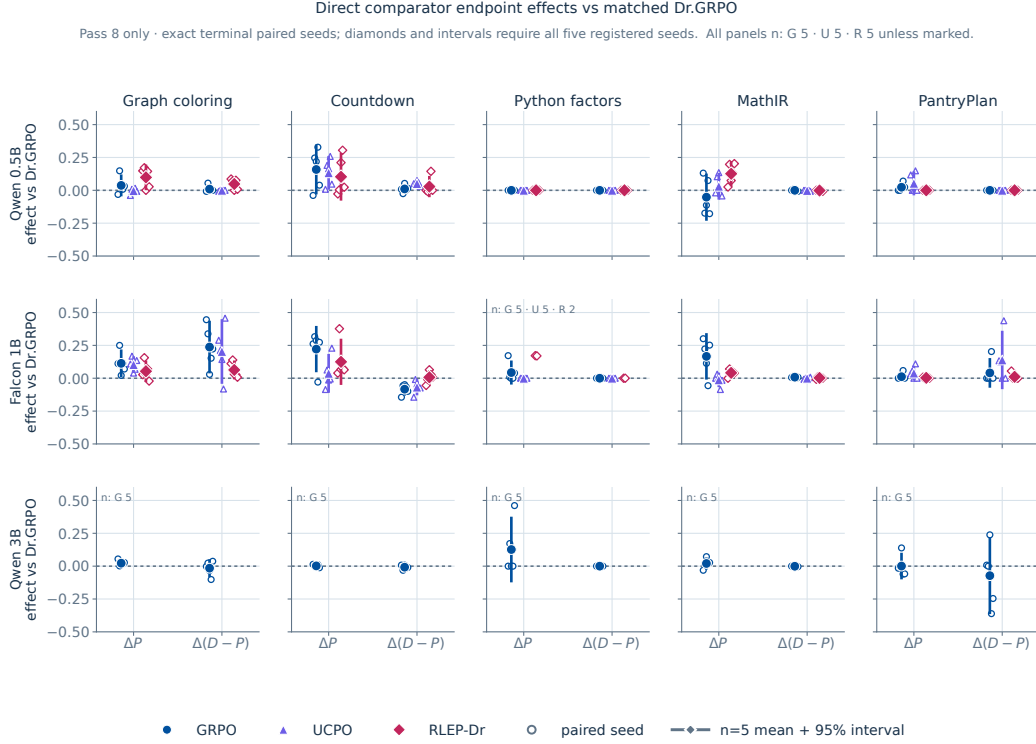


Figure 14: Direct alternatives produce local rather than uniform final gains. Each effect subtracts the same-seed Dr.GRPO control from the comparator after eight passes. Effects are shown for $\text{pass}@8$ and extra verified modes beyond the first success. Circles are paired seeds; diamonds and 95% Student- t intervals appear only for complete five-seed blocks. Exact n is printed, and unsupported comparisons remain blank rather than being extrapolated or imputed at any seed.

The alternatives give local rather than uniform benefits. Qwen-0.5B Countdown UCPO raises P by $+.133$ ($[+.005, +.262]$) and B by $+.051$ ($[+.031, +.071]$), whereas its MathIR effects are inconclusive. MathIR RLEP-Dr raises P by $+.126$ ($[+.030, +.222]$) with neutral extra modes ($-.002$, $[-.006, +.003]$); its Countdown effects and Falcon Pantry effects are inconclusive. The full five-seed 3B plain-GRPO blocks likewise have intervals including zero on both correctness and extra modes. RLEP’s frozen offline pool was collected at temperature .7, top- $p = .95$; all reported evaluations use the common temperature-1, top- $p = 1$ contract. An offline pool versus an online bank remains an algorithmic difference.

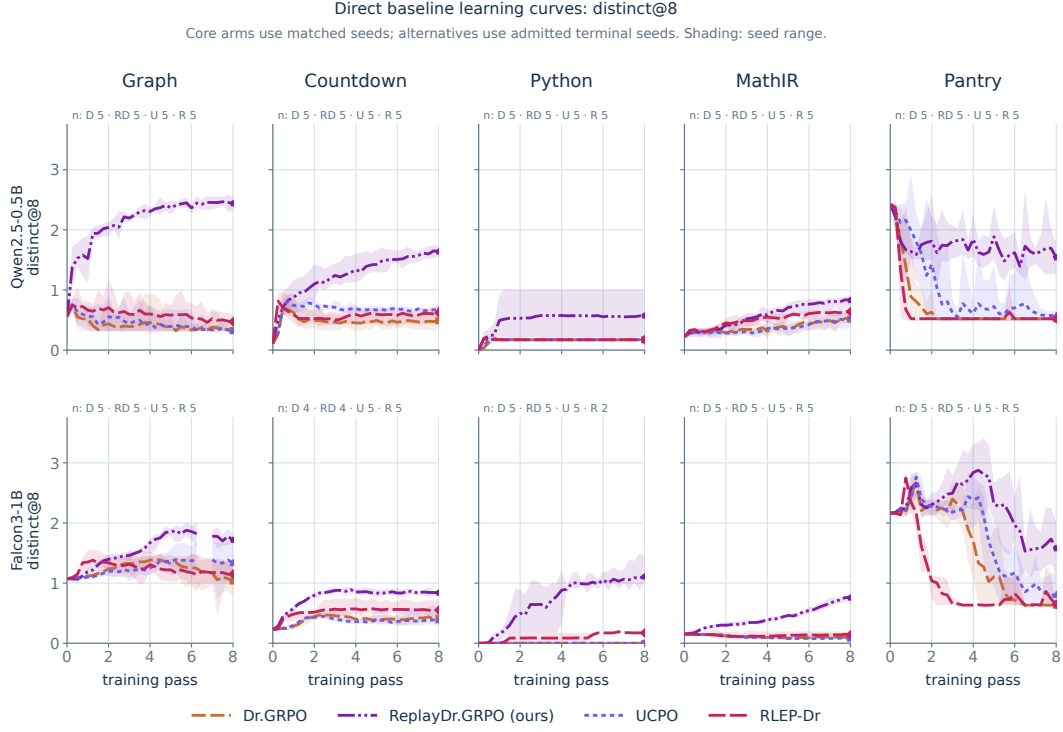


Figure 15: **Verified-mode trajectories for the supporting comparators.** Rows show Qwen2.5-0.5B and Falcon3-1B; columns are the five domains. UCPO and sparse RLEP-Dr are compared with Dr.GRPO and ReplayDr.GRPO at these two smaller scales. Labels give exact method-specific seed counts. Lines are descriptive distinct@8 means and bands are seed ranges. The corresponding accuracy curves use identical populations in Figure 16.

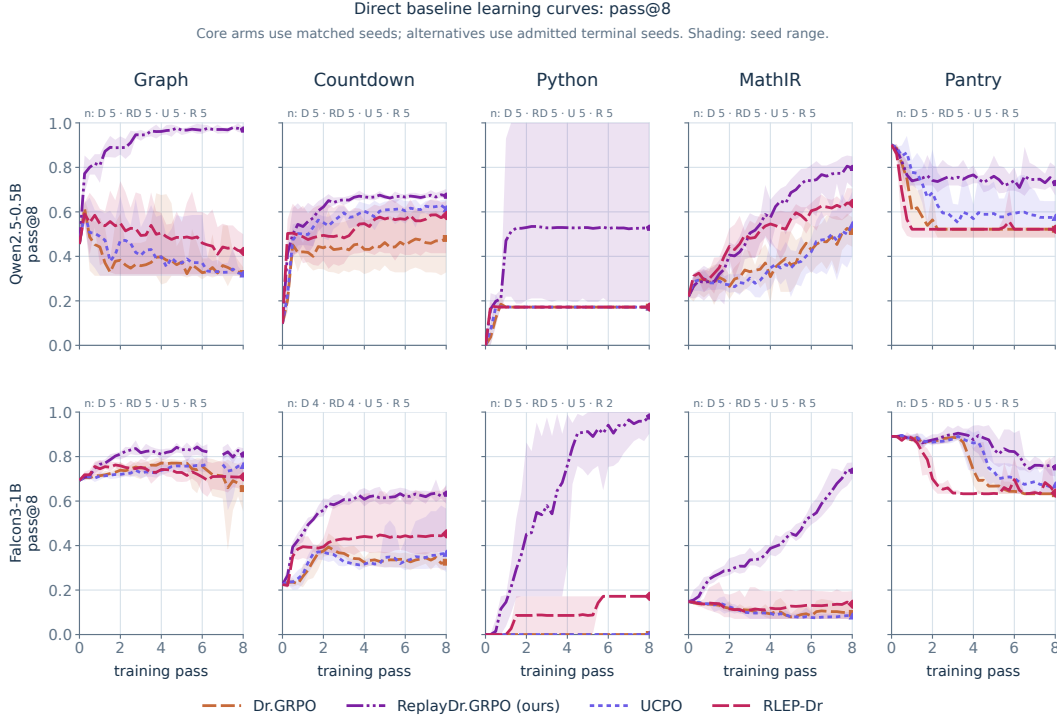


Figure 16: **Accuracy trajectories for the supporting comparators.** The same model rows, domains, methods, terminal seeds, and registered checkpoints as Figure 15, now measuring pass@8. Bands show the seed range and the vertical axes span $[0, 1]$ throughout. MaxRL and ReplayMaxRL at all three scales are compared on both metrics in Figures 11 and 12.

G.6 UNIFORM VERSUS FRESH-FREQUENCY REPLAY

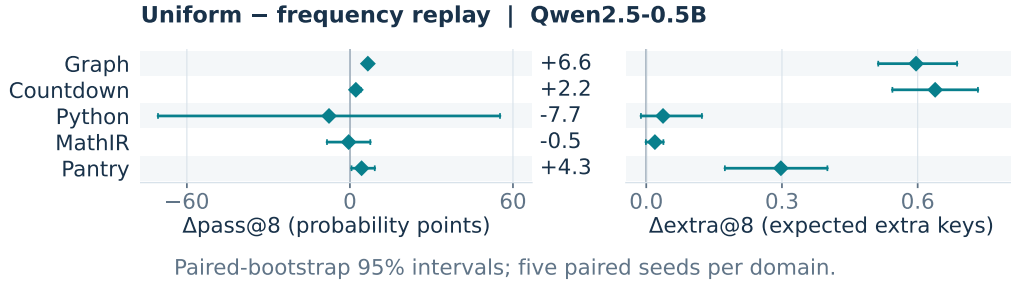


Figure 17: **Equal key weights can improve breadth beyond the first success.** Uniform minus fresh-frequency weighting of verified replay. Success and extra modes are separate endpoints; a positive extra-mode effect does not hold correctness fixed. The displayed model, domains, seeds, and uncertainty use the frozen weighting comparison. The domain variation and inconclusive success contrast limit the claim to the demonstrated breadth contribution.

The weighting ablation retains the same bank construction and update code, changing only the within-bank target: uniform weights over retained keys versus weights proportional to validator-positive fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below favor uniform weighting. The registered co-primary contrasts are ΔB and ΔP ; the five-domain Qwen2.5-0.5B aggregate first averages domain effects within each matched seed, giving equal domain weight.

Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired training seeds and use linearly interpolated 2.5th and 97.5th percentiles. A resampled seed carries its complete domain vector when estimating the five-domain aggregate. The endpoints and

aggregate were preregistered; the exact seed-resampling and percentile conventions are analysis choices. These nominal intervals quantify seed variability within the fixed benchmark, without resampling prompts or domains or adjusting across comparisons.

Table 11: **Uniform versus fresh-frequency replay at pass 8.** Effects are uniform minus frequency weighting. Brackets give the paired percentile bootstrap interval for ΔB over all 5^5 ordered resamples in complete blocks. Every observed treatment endpoint in this snapshot passes the persisted replay-weight telemetry audit. Partial blocks retain exact seed counts. Paired model exports.

Model	Domain	Weighting	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Uniform	5/5	+0.066	+0.662	+0.596 [+0.513, +0.687]
Qwen-0.5B	Countdown	Uniform	5/5	+0.022	+0.661	+0.639 [+0.544, +0.733]
Qwen-0.5B	Python	Uniform	5/5	-0.077	-0.040	+0.037 [-0.012, +0.123]
Qwen-0.5B	MathIR	Uniform	5/5	-0.005	+0.014	+0.019 [-0.001, +0.038]
Qwen-0.5B	PantryPlan	Uniform	5/5	+0.043	+0.341	+0.298 [+0.174, +0.400]
Falcon-1B	Graph	Uniform	5/5	+0.045	+0.278	+0.233 [+0.182, +0.320]
Falcon-1B	PantryPlan	Uniform	5/5	-0.021	-0.167	-0.146 [-0.564, +0.414]
Qwen-3B	Graph	Uniform	4/5	-0.006	+0.063	+0.070
Qwen-3B	PantryPlan	Uniform	5/5	+0.011	+0.219	+0.207 [+0.038, +0.377]

The registered Qwen2.5-0.5B five-domain effect is +.318 extra modes per prompt ([+.272, +.358]), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The correctness effect is +.010 ([-.119, +.139]), establishing neither a gain nor equivalence. Python’s correctness effects range from -.781 to +.828 across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes (+.233, [+0.182, +.320]), but its mean sampled correctness decreases by -.0152 ([-.0214, -.0090]); its P interval includes zero. Falcon Pantry is inconclusive: $\Delta B = -.146$ ([-.564, +.414]) and $\Delta P = -.021$ ([-.063, +.026]). Qwen2.5-3B Graph has a descriptive extra-mode effect of +.070 at four pairs. Pantry is complete at five pairs, with $\Delta B = +.207$ ([+.038, +.377]) and inconclusive correctness (+.011, [-.020, +.043]). All observed treatment endpoints pass the persisted replay-weight telemetry audit. The evidence supports a contribution from key balancing in the stated domains, not a uniform accuracy or breadth advantage across every model-domain block.

Evidence boundary. The primary effects are within-domain, within-scale paired comparisons. They do not establish a scaling law, a positive factorial interaction, or preservation of every previously discovered mode. Partial domains are reported as observed and do not support complete-block generality.

H COLLAPSE AND REPLAY MECHANISM CHECKS

H.1 CONDITIONAL CONCENTRATION FROM SAVED TRAINING OUTPUTS

This retrospective analysis distinguishes the distribution of correct keys from the probability of producing a correct output. If $P = 0$, C is undefined and $B_2 = 0$. For a fixed prompt with correct mass $P > 0$, write $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$ and define

$$C(q) = \sum_c q_c^2, \quad B_2 = 2P - P^2 C(q). \quad (22)$$

The first quantity is the probability that two independent correct outputs share a key. The second identity follows by expanding $B_2 = \sum_c [1 - (1 - Pq_c)^2]$. It makes explicit why sampled breadth can change through correctness or conditional concentration. A decrease in C alone does not imply majorization, larger total support, or recovery of any particular mode. In particular, observed collisions cannot establish that an unobserved key has exactly zero probability under the policy.

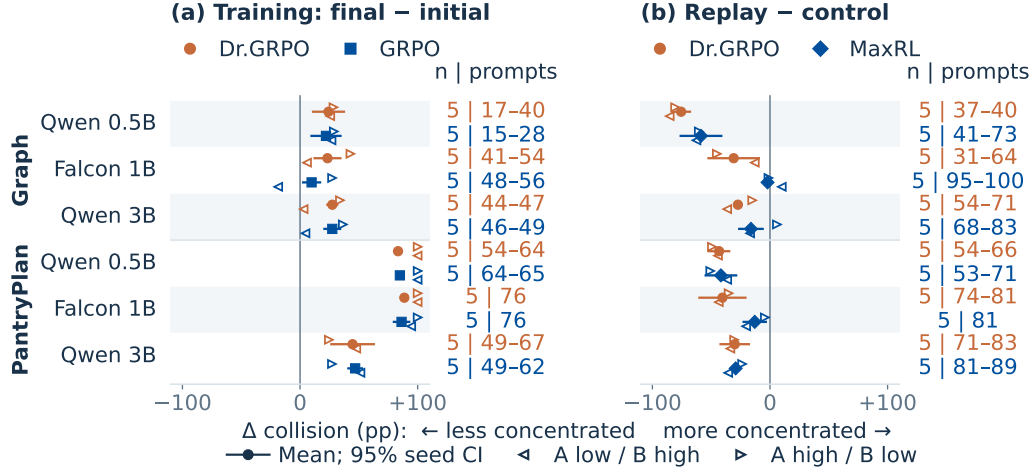


Figure 18: **Training concentrates verified outputs; canonical replay can mitigate it.** Left: final minus initial correct-key collision for Dr.GRPO and GRPO. Right: replay minus its matched terminal control, separately for Dr.GRPO and MaxRL. Graph and PantryPlan have substantial initial breadth. Dots and nominal 95% intervals average five seed contrasts; each contrast weights jointly eligible prompts equally. Smaller markers show the two disjoint-stream orientations; gutters give defined seed n and the eligible-prompt count range out of 128. The two panels have separate eligibility and are not one trajectory. Appendix H.1 reports all five domains, undefined cells, source assumptions, and correctness changes on the same eligible prompts.

Connection to the conditional theory. Under the assumptions of Theorem C.3, the time change in its proof gives $dq_c/d\tau = q_c(q_c - C)$. Consequently,

$$\frac{dC}{d\tau} = 2 \left(\sum_c q_c^3 - C^2 \right) = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (23)$$

The variance vanishes exactly when q is uniform on its positive support. Thus concentration is monotone along this ideal categorical flow, with strict increase whenever the positive coordinates are unequal. The identity provides a theoretical basis for the diagnostic; it does not establish monotonicity for the implemented neural optimizer or a corresponding monotonic decrease under finite-bank replay.

Estimator and identifiable population. For $R \geq 2$ correct samples with per-key counts n_c , use the pair statistic

$$\hat{C} = \frac{\sum_c n_c(n_c - 1)}{R(R - 1)}. \quad (24)$$

For independent samples from one fixed prompt and policy, conditioning on $R = r$ leaves r independent correct labels with distribution q . Every ordered pair therefore agrees with probability $C(q)$, proving $\mathbb{E}[\hat{C} \mid R = r] = C(q)$ for $r \geq 2$. This is a promptwise identity; it does not prove unbiasedness after joint selection using other conditions or dependent prompts. We leave $R < 2$ undefined, rather than assigning a concentration value to insufficient correct observations.

For each paired training seed s , let E_s contain exactly the prompts with a defined estimate in both conditions. The reported seed contrast is $|E_s|^{-1} \sum_{x \in E_s} (\hat{C}_{B,x} - \hat{C}_{A,x})$ when E_s is nonempty. We then weight defined training seeds equally, preserving all registered blocks and their missingness. Before/after contrasts use each method’s own initial checkpoint and step 3,072; replay contrasts compare matched terminal conditions. No other method substitutes for a missing initial evaluation. Five defined seeds receive nominal Student- t intervals; partial blocks are descriptive. These intervals describe the measured seed means and are not simultaneous guarantees or uncertainty over unseen prompts.

The hosted analysis uses the same promptwise collision concept but pools correct pairs. At a common sampling budget, the ratio of expected colliding and correct pair counts is $\sum_x P_x^2 C(q_x) / \sum_x P_x^2$. Its

finite-sample ratio is not generally unbiased for that target. The training contrast here weights eligible prompts equally. Changing correctness can change either eligibility or hosted pair weights, so the two aggregate values are not directly interchangeable.

Retrospective protocol and source coverage. The estimator and comparisons were recorded before the initial concentration contrasts, with the sampling-stream amendment also preceding those effects. The paper’s source census was then refreshed during the analysis. A separately recorded extension, declared after inspecting the initial results, includes all thirteen newly available terminal checkpoints and withdraws one newly conflicted initial checkpoint. It preserves the original results and source snapshot. The final collection contains 475 registered runs and 892 available checkpoint evaluations, with 74 Dr.GRPO and 75 MaxRL Level-1 replay pairs before conditional eligibility. Complete terminal coverage does not supply missing initial outputs or make every conditional contrast identifiable. Both analysis phases, exact source-prefix hashes, and the extension’s timing are retained with the result artifact for reproducibility.

Sampling-stream audit. Four saved eight-output groups do not provide 32 independently seeded responses. The recorded parent seeds increment by one; the audited vLLM V0 sampler increments each child’s seed by its output index. The resulting nominal stream ranges overlap, giving eleven distinct identifiers. For each prompt and checkpoint we retain the earliest draw/output position for every identifier, independently of correctness or key identity. Repeated positions sometimes disagree, so identical seed integers are not treated as a proof of identical outputs. Archived actor source is available for 150 of the 400 factorial cells and none of the 75 additional GRPO cells; historical dependency identity is not cryptographically established for every run. The stream mapping rests on the audited implementation and recorded neutral request contract, with the historical-runtime assumption stated explicitly.

The primary eleven-stream comparisons reuse nominal streams across conditions and remain descriptive after common eligibility. Two additional orientations compare the lower five identifiers in condition A with the upper six in B, then reverse the assignment. Each orientation retains its own eligible prompts and seed counts. Shared identifiers also recur across prompts; neither their separation across conditions nor these selected-population means establishes independent prompt observations. Independent fixed-law sampling is an explicit assumption of the estimator identity, not a conclusion of the source audit.

Original `mean@8`, `pass@8`, and `distinct@8` remain averages over the four intact groups, reconstructed from all saved verified keys. Cross-group overlap affects dependence, not linearity of those marginal averages. The complete result artifact also retains each intact-group collision comparison, a fixed-first-group check, fixed-across-seed prompt intersections, and four-condition change differences. A naive concatenated 32-position collision is retained solely as a reuse diagnostic. These views are not treated as independent replications or substituted for missing cells.

Findings and their limits. In Graph and PantryPlan, all twelve Dr.GRPO/GRPO before/after comparisons across the three scales show increased conditional concentration, with nominal intervals above zero. Correctness also increases on those same eligible prompts. Graph’s increases range from .098 to .274; PantryPlan’s range from .447 to .887. The two disjoint orientations agree in direction in eleven of these twelve blocks; Falcon Graph with GRPO is the exception. The evidence therefore supports observed training-induced concentration in these tasks, with a visible sampling sensitivity, rather than a universal claim about every RLVR run or exact disappearance of modes.

ReplayDr.GRPO lowers concentration relative to Dr.GRPO in all six Graph and PantryPlan comparisons: changes range from $-.756$ to $-.273$ in Graph and $-.433$ to $-.300$ in PantryPlan. Both disjoint orientations agree in direction in all six, and their primary nominal intervals lie below zero. These comparisons also have lower per-sample correctness on their jointly eligible prompts. They establish a change in the conditional correct-key distribution on that observable population; they do not hold correctness fixed or imply that every accuracy measure improves on the same prompts. Full-test `pass@8` and sampled breadth remain the separately reported primary outcomes of the replay experiment.

The other domains constrain the interpretation. Dr.GRPO/GRPO Python initial outputs provide no jointly eligible prompts at this budget, so their longitudinal collision contrast is unidentified. Some Countdown initial comparisons use only one or two eligible prompts per seed. Several MathIR conditions already have observed collision one, leaving little detectable change. MaxRL comparisons,

incomplete saved-output cohorts, and the two sampling orientations are reported explicitly in the following tables; we do not infer a universal concentration benefit from raw breadth gains.

In the completed 3B MaxRL replay cohort, Countdown, Python, and PantryPlan show lower concentration with nominal intervals below zero; MathIR’s measured contrast is zero. Graph’s primary contrast is negative, but its two disjoint orientations disagree in direction, as they do for Falcon Graph with MaxRL. At Level 2, both Graph replay contrasts are negative with agreement across orientations. Partial PantryPlan comparisons instead show increased collision: $+.490$ for ReplayDr.GRPO from one defined seed with sixteen eligible prompts, and $+1.000$ for ReplayMaxRL from one seed with one eligible prompt. These sparse observations provide no complete-block conclusion and remain visible rather than being dropped from a claim of uniformly beneficial replay.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.242	[+0.110, +0.373]	17–40	+0.257 (5)	+0.284 (5)
Graph	GRPO	5	+0.221	[+0.097, +0.344]	15–28	+0.268 (5)	+0.287 (5)
Graph	MaxRL	5	+0.145	[-0.033, +0.324]	18–40	+0.115 (5)	+0.174 (5)
Graph	ReplayDr	5	-0.507	[-0.558, -0.456]	45–49	-0.519 (5)	-0.465 (5)
Graph	ReplayMaxRL	5	-0.399	[-0.458, -0.339]	40–51	-0.436 (5)	-0.348 (5)
Countdown	Dr.GRPO	5	+1.000	[+1.000, +1.000]	1–2	– (0)	+1.000 (5)
Countdown	GRPO	5	+0.964	[+0.863, +1.065]	1–2	– (0)	+0.920 (5)
Countdown	MaxRL	4	+0.977	–	0–2	– (0)	+1.000 (4)
Countdown	ReplayDr	5	+0.467	[+0.117, +0.816]	1–3	– (0)	+0.713 (4)
Countdown	ReplayMaxRL	5	+0.420	[+0.359, +0.480]	1–2	– (0)	+0.493 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	0	–	–	0	– (0)	– (0)
Python	ReplayDr	0	–	–	0	– (0)	– (0)
Python	ReplayMaxRL	0	–	–	0	– (0)	– (0)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	5–8	+0.000 (5)	+0.000 (1)
MathIR	GRPO	5	+0.040	[-0.071, +0.151]	4–10	+0.000 (5)	+0.250 (2)
MathIR	MaxRL	5	+0.022	[-0.039, +0.084]	5–9	+0.000 (5)	+0.067 (5)
MathIR	ReplayDr	5	+0.059	[-0.046, +0.163]	8–11	+0.000 (5)	+0.367 (2)
MathIR	ReplayMaxRL	5	+0.063	[-0.009, +0.134]	9–11	+0.000 (5)	+0.250 (5)
Pantry	Dr.GRPO	5	+0.833	[+0.819, +0.847]	54–64	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.849	[+0.845, +0.853]	64–65	+1.000 (5)	+1.000 (5)
Pantry	MaxRL	5	+0.859	[+0.836, +0.881]	54–65	+1.000 (5)	+1.000 (5)
Pantry	ReplayDr	5	+0.413	[+0.345, +0.481]	78–89	+0.543 (5)	+0.538 (5)
Pantry	ReplayMaxRL	5	+0.446	[+0.324, +0.567]	80–88	+0.593 (5)	+0.550 (5)

Table 12: **Initial to final: Qwen2.5-0.5B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

H.2 BREADTH BEFORE AND AFTER VERIFIER-ONLY TRAINING

The problem precheck evaluates unmodified Dr.GRPO and GRPO at passes 0 and 8 on all three models and five domains: 150 complete runs, each with four fixed $K = 8$ draws. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. Correctness falls in 10 of the 30 domain–method–scale comparisons, while macro raw `distinct@8` ends above its initial value at Qwen2.5-3B. This distinguishes loss of multiple correct outcomes from the broader question of whether any correct outcome is sampled.

Initial evaluations are measured within each method. Qwen2.5-3B MathIR and Pantry E80-R1 seeds 71–74 carry slightly different initial requests from plain GRPO (.154 versus .162, and .832 versus .828 on P). Those cells therefore have separate initial references.

Graph and PantryPlan contain substantial initial breadth at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-breadth percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. ReplayDr.GRPO does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four admissible seeds. These observations concern sampled held-out support, not the survival histories of particular stored modes.

Domain	Objective	n	$\Delta \hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.233	[+0.124, +0.343]	41–54	+0.059 (5)	+0.428 (5)
Graph	GRPO	5	+0.098	[+0.026, +0.170]	48–56	-0.186 (5)	+0.273 (5)
Graph	MaxRL	3	-0.113	–	48–52	-0.083 (3)	+0.039 (3)
Graph	ReplayDr	5	-0.007	[-0.065, +0.050]	37–45	+0.048 (5)	+0.045 (5)
Graph	ReplayMaxRL	2	-0.062	–	51–52	-0.300 (2)	-0.015 (2)
Countdown	Dr.GRPO	5	+0.022	[-0.038, +0.082]	6–8	+0.000 (5)	– (0)
Countdown	GRPO	5	+0.180	[+0.128, +0.231]	6–11	+0.126 (5)	– (0)
Countdown	MaxRL	2	+0.182	–	11	+0.143 (2)	– (0)
Countdown	ReplayDr	4	+0.035	–	10–11	+0.158 (4)	– (0)
Countdown	ReplayMaxRL	1	+0.135	–	11	+0.143 (1)	– (0)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	0	–	–	0	– (0)	– (0)
Python	ReplayDr	0	–	–	0	– (0)	– (0)
Python	ReplayMaxRL	0	–	–	–	– (0)	– (0)
MathIR	Dr.GRPO	5	+0.105	[+0.088, +0.122]	6	– (0)	+0.098 (5)
MathIR	GRPO	5	+0.054	[-0.099, +0.206]	3–7	– (0)	+0.233 (5)
MathIR	MaxRL	1	+0.278	–	6	+1.000 (1)	+0.133 (1)
MathIR	ReplayDr	5	+0.086	[+0.026, +0.146]	6–7	– (0)	+0.111 (5)
MathIR	ReplayMaxRL	2	+0.167	–	5–6	+1.000 (2)	+0.000 (2)
Pantry	Dr.GRPO	5	+0.886	[+0.886, +0.886]	76	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.863	[+0.798, +0.928]	76	+0.945 (5)	+1.000 (5)
Pantry	MaxRL	1	+0.886	–	76	+1.000 (1)	+1.000 (1)
Pantry	ReplayDr	4	+0.437	–	75–83	+0.451 (4)	+0.618 (4)
Pantry	ReplayMaxRL	2	+0.746	–	76–78	+0.740 (2)	+0.978 (2)

Table 13: **Initial to final: Falcon3-1B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta \hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.274	[+0.232, +0.316]	44–47	+0.030 (5)	+0.339 (5)
Graph	GRPO	5	+0.274	[+0.207, +0.341]	46–49	+0.046 (5)	+0.364 (5)
Graph	MaxRL	5	+0.072	[-0.002, +0.145]	45–55	-0.208 (5)	+0.215 (5)
Graph	ReplayDr	5	-0.038	[-0.060, -0.016]	48–53	-0.227 (5)	+0.121 (5)
Graph	ReplayMaxRL	5	-0.145	[-0.198, -0.093]	46–52	-0.220 (5)	+0.165 (5)
Countdown	Dr.GRPO	5	+0.160	[+0.109, +0.211]	11–12	+0.200 (5)	-0.016 (5)
Countdown	GRPO	5	+0.182	[+0.182, +0.182]	11	+0.200 (5)	+0.000 (5)
Countdown	MaxRL	5	+0.056	[-0.046, +0.159]	9–11	+0.050 (5)	+0.000 (5)
Countdown	ReplayDr	5	+0.020	[-0.060, +0.099]	11–15	+0.169 (5)	-0.071 (5)
Countdown	ReplayMaxRL	5	-0.100	[-0.145, -0.055]	10–12	+0.000 (5)	-0.050 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	+0.000	[+0.000, +0.000]	2–9	+0.000 (2)	+0.000 (5)
Python	ReplayDr	0	–	–	0	– (0)	– (0)
Python	ReplayMaxRL	5	-0.023	[-0.050, +0.003]	9	+0.000 (5)	-0.040 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	11–14	+0.000 (5)	+0.000 (5)
MathIR	GRPO	5	+0.000	[+0.000, +0.000]	9–11	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	7–14	+0.000 (5)	+0.000 (5)
MathIR	ReplayDr	4	-0.012	–	14	+0.000 (4)	+0.000 (4)
MathIR	ReplayMaxRL	5	+0.000	[+0.000, +0.000]	12–15	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	+0.447	[+0.266, +0.627]	49–67	+0.478 (5)	+0.245 (5)
Pantry	GRPO	5	+0.469	[+0.411, +0.526]	49–62	+0.514 (5)	+0.271 (5)
Pantry	MaxRL	5	+0.431	[+0.297, +0.565]	59–69	+0.394 (5)	+0.246 (5)
Pantry	ReplayDr	5	+0.184	[+0.081, +0.288]	64–71	+0.087 (5)	+0.037 (5)
Pantry	ReplayMaxRL	5	+0.142	[+0.018, +0.266]	68–75	+0.046 (5)	-0.015 (5)

Table 14: **Initial to final: Qwen2.5-3B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.625	[+0.295, +0.955]	2–10	+0.633 (2)	+0.860 (5)
Graph	MaxRL	5	+0.452	[+0.387, +0.518]	6–10	+0.767 (4)	+0.484 (5)
Graph	ReplayDr	5	-0.204	[-0.282, -0.126]	9–11	+0.033 (5)	-0.372 (5)
Graph	ReplayMaxRL	5	-0.128	[-0.262, +0.006]	9–11	+0.333 (5)	-0.267 (5)
Countdown	Dr.GRPO	1	+0.000	–	0–1	– (0)	– (0)
Countdown	MaxRL	1	+0.000	–	0–1	– (0)	– (0)
Countdown	ReplayDr	5	+0.000	[+0.000, +0.000]	1	– (0)	– (0)
Countdown	ReplayMaxRL	3	+0.000	–	0–1	– (0)	– (0)
Python	Dr.GRPO	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	MaxRL	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	ReplayDr	5	+0.094	[-0.259, +0.448]	32	-0.224 (5)	+0.146 (5)
Python	ReplayMaxRL	5	+0.199	[-0.088, +0.485]	32	-0.070 (5)	+0.258 (5)
MathIR	Dr.GRPO	3	+0.000	–	0–5	– (0)	+0.000 (3)
MathIR	MaxRL	4	+0.000	–	0–6	– (0)	+0.000 (4)
MathIR	ReplayDr	5	+0.000	[+0.000, +0.000]	6	– (0)	+0.000 (5)
MathIR	ReplayMaxRL	4	+0.000	–	6	– (0)	+0.000 (4)
Pantry	Dr.GRPO	2	-0.867	–	1–3	-0.500 (1)	-1.000 (1)
Pantry	MaxRL	1	+0.000	–	1	– (0)	– (0)
Pantry	ReplayDr	2	-0.049	–	15–16	-0.075 (2)	-0.050 (2)
Pantry	ReplayMaxRL	1	+0.092	–	19	+0.097 (1)	-0.222 (1)

Table 15: **Initial to final: Qwen2.5-0.5B, Level 2.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.756	[-0.831, -0.680]	37–40	-0.853 (5)	-0.807 (5)
Graph	MaxRL	5	-0.587	[-0.759, -0.414]	41–73	-0.625 (5)	-0.605 (5)
Countdown	Dr.GRPO	5	-0.529	[-0.605, -0.452]	42–78	-0.529 (5)	-0.535 (5)
Countdown	MaxRL	5	-0.547	[-0.635, -0.459]	45–84	-0.578 (5)	-0.530 (5)
Python	Dr.GRPO	5	-0.090	[-0.341, +0.160]	22	-0.115 (5)	-0.062 (5)
Python	MaxRL	5	-0.266	[-0.577, +0.044]	22	-0.298 (5)	-0.272 (5)
MathIR	Dr.GRPO	5	-0.008	[-0.017, +0.001]	44–68	-0.006 (5)	-0.008 (5)
MathIR	MaxRL	5	-0.005	[-0.013, +0.003]	32–56	-0.007 (5)	-0.002 (5)
Pantry	Dr.GRPO	5	-0.433	[-0.518, -0.347]	54–66	-0.442 (5)	-0.494 (5)
Pantry	MaxRL	5	-0.417	[-0.547, -0.288]	53–71	-0.363 (5)	-0.508 (5)

Table 16: **Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.309	[-0.523, -0.094]	31–64	-0.129 (5)	-0.452 (5)
Graph	MaxRL	5	-0.021	[-0.040, -0.002]	95–100	+0.100 (5)	-0.013 (5)
Countdown	Dr.GRPO	4	-0.168	–	33–42	-0.139 (4)	-0.233 (4)
Countdown	MaxRL	5	-0.063	[-0.079, -0.047]	78–81	-0.062 (5)	-0.063 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	-0.093	[-0.350, +0.165]	22–128	-0.039 (5)	-0.090 (5)
MathIR	Dr.GRPO	5	+0.004	[-0.007, +0.015]	9–14	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.010, +0.009]	40–72	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.404	[-0.600, -0.209]	74–81	-0.435 (5)	-0.352 (5)
Pantry	MaxRL	5	-0.129	[-0.223, -0.035]	81	-0.203 (5)	-0.043 (5)

Table 17: **Replay minus control at the final checkpoint: Falcon3-1B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.273	[-0.299, -0.247]	54–71	-0.363 (5)	-0.151 (5)
Graph	MaxRL	5	-0.162	[-0.261, -0.062]	68–83	-0.169 (5)	+0.059 (5)
Countdown	Dr.GRPO	5	-0.137	[-0.183, -0.090]	70–75	-0.153 (5)	-0.139 (5)
Countdown	MaxRL	5	-0.105	[-0.126, -0.084]	56–76	-0.113 (5)	-0.095 (5)
Python	Dr.GRPO	2	-0.190	–	0–22	-0.221 (2)	-0.141 (2)
Python	MaxRL	5	-0.153	[-0.265, -0.041]	22–81	-0.203 (5)	-0.083 (5)
MathIR	Dr.GRPO	5	-0.010	[-0.022, +0.003]	23–34	-0.017 (5)	+0.003 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	21–27	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.300	[-0.420, -0.179]	71–83	-0.337 (5)	-0.305 (5)
Pantry	MaxRL	5	-0.292	[-0.319, -0.265]	81–89	-0.355 (5)	-0.237 (5)

Table 18: **Replay minus control at the final checkpoint: Qwen2.5-3B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.577	[-0.736, -0.418]	9–25	-0.599 (5)	-0.544 (5)
Graph	MaxRL	5	-0.405	[-0.472, -0.339]	28–42	-0.262 (5)	-0.416 (5)
Countdown	Dr.GRPO	4	-0.230	–	0–9	-0.281 (4)	-0.292 (4)
Countdown	MaxRL	4	-0.073	–	0–10	+0.000 (3)	-0.118 (4)
Python	Dr.GRPO	5	-0.209	[-0.563, +0.146]	104	-0.219 (5)	-0.234 (5)
Python	MaxRL	5	-0.106	[-0.397, +0.184]	104	-0.076 (5)	-0.121 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	12–122	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.004, +0.002]	29–125	+0.000 (5)	-0.001 (5)
Pantry	Dr.GRPO	1	+0.490	–	0–16	+0.656 (1)	+0.567 (1)
Pantry	MaxRL	1	+1.000	–	1	– (0)	– (0)

Table 19: **Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 2.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

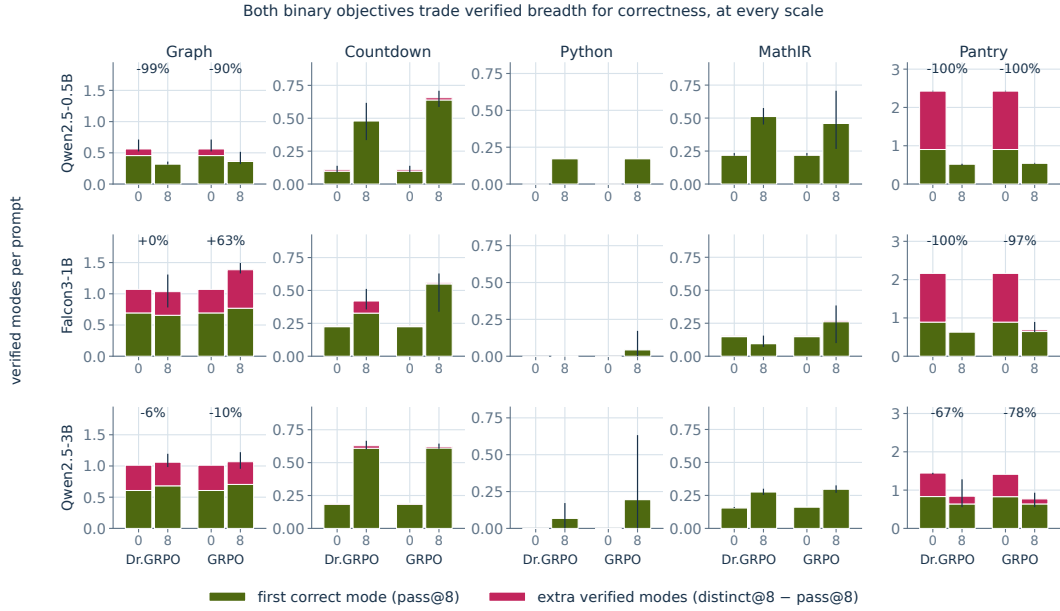


Figure 19: **Verifier-only training reduces average extra verified modes at each tested scale.** Bars split distinct@8 into the first correct mode and every verified mode beyond it, at training pass 0 and pass 8, for unmodified Dr.GRPO then GRPO in each panel. All 150 evaluations are complete at five seeds and four fixed $K = 8$ draws; whiskers are the seed range and pass 0 is measured per method. Badges give the change in extra verified modes, printed only where pass-0 breadth exceeds .05 of them; the other three domains start at .011 or less.

Table 20: **Problem precheck, five-domain macro-averages.** Both verifier-only objectives at pass 0 and pass 8, all three scales, five seeds and four fixed $K = 8$ draws per model–domain–method comparison. “Extra modes” is $\text{distinct@8} - \text{pass@8}$; “removed” is the fraction of that quantity lost between pass 0 and pass 8. Breadth loss is present at every scale and is far from uniform across scales, and at Falcon3-1B Dr.GRPO macro pass@8 falls as well.

Model	Dr.GRPO			GRPO		
	pass@8	extra modes	removed	pass@8	extra modes	removed
Qwen2.5-0.5B	0.335 → 0.402	0.329 → 0.002	99.4%	0.335 → 0.435	0.329 → 0.006	98.2%
Falcon3-1B	0.391 → 0.342	0.332 → 0.094	71.6%	0.391 → 0.454	0.332 → 0.135	59.4%
Qwen2.5-3B	0.356 → 0.455	0.205 → 0.120	41.5%	0.356 → 0.490	0.199 → 0.101	49.4%

Uniform-action references are meaningful for the enumerable Graph and Pantry representations. They are not defined for open-ended programs or derivations. Pantry’s bit-uniform reference exceeds every final model, so this domain is a support-concentration stress test rather than evidence of complete support coverage.

H.3 FRESH-GRADIENT DEGENERACY AND OBSERVED OUTPUT COLLAPSE

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Table 21: **Absolute references for raw distinct@8.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Frozen is the matched pass-0 model; the final two columns are pass-8 means. Dashes avoid pretending that an open-ended program or derivation space has a canonical uniform distribution.

Model	Domain	Uniform	Frozen	Dr.GRPO	ReplayDr.GRPO
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

Domain	Dr.GRPO	ReplayDr.GRPO
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

A raw-response audit of all five Qwen2.5-0.5B Python control seeds establishes observed output collapse directly. At pass 0, a draw has 7.99 unique raw completions per prompt on average. By steps 288–480, every seed produces 32 byte-identical temperature-1 samples per prompt, also identical to the logged greedy completion, on all 128 prompts. This persists through step 3,072. Terminal draws contain 1.00 unique completion per prompt in the control and 1.97 under replay. A collapsed completion can be incorrect: this is one observed completion per prompt, not one correct key everywhere. Aggregate endpoint equality or zero draw spread alone would not establish that conclusion, nor does this finite audit rule out low-probability tails.

These checks explain a vulnerability of the registered sparse binary-reward recipe. They do not establish dominance over methods that keep fresh on-policy signal active. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider-decoding evaluations remain relevant controls beyond the registered factorial.

H.4 FIXED-BANK EXEMPLAR SCORE TRAJECTORIES

The fixed-bank study tracks Qwen2.5-0.5B ReplayDr.GRPO on Graph for seeds 43–47. Before step 384’s fresh group, bank membership and fresh counts are frozen; admission then stops while uniform replay continues through 3,072 updates under the same training recipe. Every frozen exemplar is identified by prompt and canonical-key fingerprints, without selection by its score trajectory. Each scheduled replay visit records mean token log probability s and sequence log probability ℓ . Changes compare an identity’s first and final post-freeze visits; observation steps can differ across identities.

All 3,406 frozen identities across 1,575 seed-prompt pairs have finite, aligned scores at every scheduled visit: 8–9 visits per identity and 29,041 observations in total. The pooled median change is $+.038$ nat/token (95% interval $[+.025, +.048]$), but the 10th percentile is $-.256$ and 2.61% of identities lose more than $.5$ nat/token ($[1.54, 3.77]\%$). The worst intermediate change is -1.328 nat/token. On the sequence scale, the median is $+.154$ nat/sequence ($[+.104, +.198]$), while 21.99% lose more than $.5$ nat/sequence. Lengths range from 4 to 100 tokens, so the two threshold percentages have different meanings.

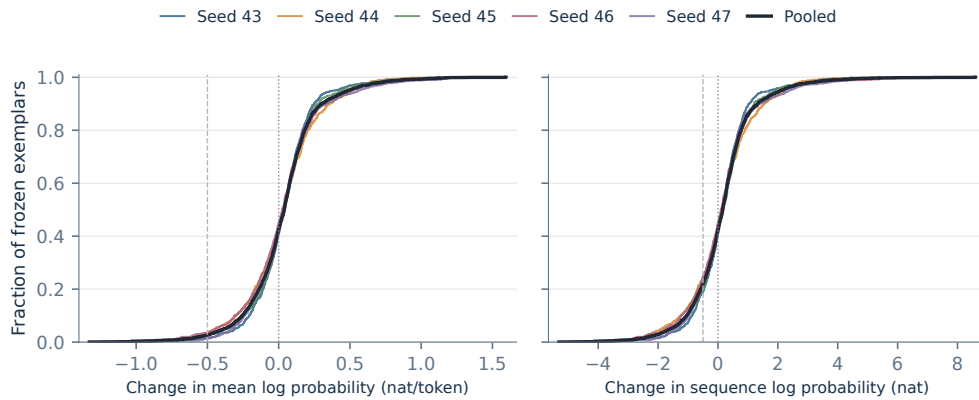


Figure 20: **Fixed-exemplar scores improve centrally but retain a declining tail.** Empirical cumulative distributions show first-to-final post-freeze score changes for every frozen identity, with one curve per seed and the pooled identity distribution. Mean token and sequence scores are separate panels; positive values indicate increased teacher-forced likelihood. Neither panel measures the exact probability of generating a canonical mode during policy sampling.

Pooled summaries weight identities equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap draws (random seed 121), resampling prompts within each original seed and then five seeds with replacement; repeated seeds reuse their within-draw prompt sample. Linear empirical quantiles and percentile intervals are analysis choices. The artifact `paper/results/e121_fixed_bank_survival.json` retains all seed summaries, identity trajectories, source hashes, and audit diagnostics.

These are teacher-forced score surrogates, not exact probabilities of sampling canonical modes. Visits do not cover every intervening update, and the declining tails exclude a claim that every exemplar improves. Without a matched no-replay fixed-bank arm, this study does not identify a causal replay effect. It neither numerically validates Theorem C.6’s probability floor nor certifies a finite-network guarantee. It adds longitudinal measurements of fixed exemplars to the held-out support evidence.

I INFERENCE-ONLY CONCENTRATION IN A HOSTED MODEL

This exploratory evaluation measures the current output distribution of GPT-5.6 Sol under the inherited MODEBENCH interface. It is separate from the matched training interventions: it adds no frontier-model training, replay comparison, or controlled model-size contrast. Only the completed, audited deployment enters these tables.

Protocol and provenance. The September 11, 2026 run uses the Azure deployment `gpt-5.6-sol`; saved provider headers identify snapshot `gpt-5.6-sol-2026-07-09`. Each of the five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. The service returns temperature 1.0 and `top-p .98`; neither setting was overridden. These settings and the hosted natural-language interface differ from the local training evaluation’s token-level grammar constraints.

Level 1 uses the native E118 evaluation splits retained in E124. Level 2 uses the frozen matched r5 construction; Level 3 uses the matched v3 construction admitted by the completed canonical v3 confirmation. Level 3 is an exploratory test extension here, not a third level of the paper’s training intervention. All cells use the multi-answer split. The Level-2/Level-3 support histograms match a separate fixed-reference Level-1 confirmation reserve, which differs from the native Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Exact prompts, dataset identities and hashes, requests, provider-exposed responses, grades, retry receipts, usage and verifier snapshots are retained in the experiment’s evidence directory. All

15,360 intended samples completed, with distinct response identifiers; 107 failed API attempts were retained and recovered by retries. Provider random-number independence and hidden reasoning contents cannot be audited from the exposed responses.

Table 22: **Hosted correctness and concentration under strict grading.** GPT-5.6 Sol, with medium reasoning. Each level weights five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is the percentage of within-prompt correct pairs sharing a mode, pair weighted within each domain. Brackets give pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8 in this paper (called pass@1 in the hosted report). Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise descriptive intervals.

Table 23: **GPT-5.6 Sol under strict executable grading.** Every cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is distinct@8. C is correct-pair collision (%), with 95% prompt-bootstrap interval); U is the uniform-correct-key collision reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Formatting sensitivity. Strict grades remain primary. A secondary typography normalizer was introduced after inspecting the first 15 responses. It fixes a frozen set of presentation conventions, including escaped Python/LaTeX syntax, and reuses the original executable validators without repairing calculations or program logic. It recovers 2,580 additional verified outputs; strict successes and their keys are retained. This is a post hoc diagnostic, not a replacement benchmark. All 3,072 primary Python responses passed an independent serial recheck with no grade

changes. One cold-start timeout in the secondary grading cache was reconciled with repeated serial verification; the original cache and correction audit remain preserved.

Table 24: **Post hoc formatting-normalized diagnostic.** Same samples and metric definitions as Table 23, with fixed presentation normalization before executable verification. The strict table remains primary; normalization was designed after the first 15 responses.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration and level. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized diagnostic gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not a monotonic law within domains. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses particularly clearly: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 correct pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). Graph was selected after the GPT run for this illustration because its correctness is high, support is certified and formatting normalization changes no scores; both tables report all domains.

Interface and claim limits. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Eight draws cannot establish that an unseen mode has zero probability. The evidence supports concentration under the specified prompts and deployment configuration; it identifies neither a training cause nor a causal difficulty or model-size effect.

Reproduction record. The source record is `artifacts/frontier_modebench_gpt56sol_20260911/summary.json`. Its hash, complete cell statistics and run settings are copied into `results/frontier_hosted_20260911.json`; adjacent generated table bodies supply the displayed values. The evidence directory also retains `completion_audit.json`, `prompt_contract_audit.json`, the original requests and responses, and `interpretation_notes.md`. Hosted-model results are separate from the training-checkpoint archive and its matched-seed comparisons.

J COMPARISON ACROSS HOSTED DEPLOYMENTS

Figure 22 compares verified output breadth across hosted deployments. Its main display applies the frozen formatting normalizer to every deployment and uses the complete revised-wording Opus 5 Python cohort at all three levels. Every response enters its accuracy and mode-count metrics; outputs are not selected by correctness or provider outcome. The other cells use their original prompts. This is a descriptive comparison across the stated task formulations, rather than a common-prompt model ranking.

The tables and Graph figure below preserve every original-protocol result, including failed draws and native refusals. The separate Python comparison reports the original and revised conditions under both grading rules. These records expose the effect of the prompt change without replacing the original evidence. Level 3 remains an exploratory test extension; no hosted-model training or replay intervention is performed. Original deployments enter the comparison only with a passing integrity audit and all 15,360 responses.

Completed hosted comparison. The admitted deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro: 7 complete runs, 107,520 responses, and the same 128 held-out prompts per domain and level with eight stateless samples per prompt. Only runs with a passing completion audit and the identical frozen prompt digest enter these tables. No training, model-side tools or conversation history is added.

Provider configurations. GPT-5.6 Sol: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 5: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. GPT-5.4: hosted API, requested reasoning setting medium, native output limit 8192. Grok 4.3: hosted API, requested reasoning setting medium, native output limit 8192. Kimi K3: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 4.8: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. DeepSeek V4 Pro: hosted API, requested reasoning setting medium, native output limit 8192.

Provider effort labels do not establish equal compute, and native token accounting differs. The saved native request and returned usage determine each configuration; provider defaults remain unspecified whenever native responses omit them.

The appendix retains strict executable grades alongside formatting-normalized scores. The main display uses the same frozen formatting rule for every cell and the complete separately collected revised Opus 5 Python condition; all other displayed cells retain the original request wording. The shared typography diagnostic was developed after the first 15 GPT-5.6 Sol responses and frozen before the prospective comparison runs. It retains its post hoc status for GPT-5.6 Sol. Inherited route preferences and the Level-1 PantryPlan interface change limit cross-level interpretation. Countdown has no certified finite-total-support uniform reference. These data establish neither a training cause nor a causal model-size or difficulty effect. All pair-collision intervals are pointwise; overlapping or nonoverlapping cell intervals are not a paired model-difference test.

Collection interruption: DeepSeek V4 Pro. 13 HTTP-200 bodies had no terminal finish reason or visible answer and were preserved as provider protocol failures. A later storage interruption left 192 logged physical request starts without a saved outcome or usage; their outcome and possible billing remain unknown. The audit registers these attempts separately from the 15,360 terminal draws. Only missing terminal draws were recollected; saved terminal responses, including truncations, were retained. Reported token totals are subtotals of known usage, not zero-cost claims for failed or interrupted attempts. Source-bound recovery sidecars preserve the original receipts and the exact interruption accounting.

J.1 A SEPARATE PYTHON PROMPT SENSITIVITY TEST

After the original Opus 5 Python refusals, synthetic tasks were used to select a direct arithmetic-expression formulation with no system message. We then collected eight independent draws for every original Python test task: all 384 prompts across three levels, with unchanged adaptive thinking, medium effort, 8,192 output tokens, allowed operators, public inputs and frozen graders. The

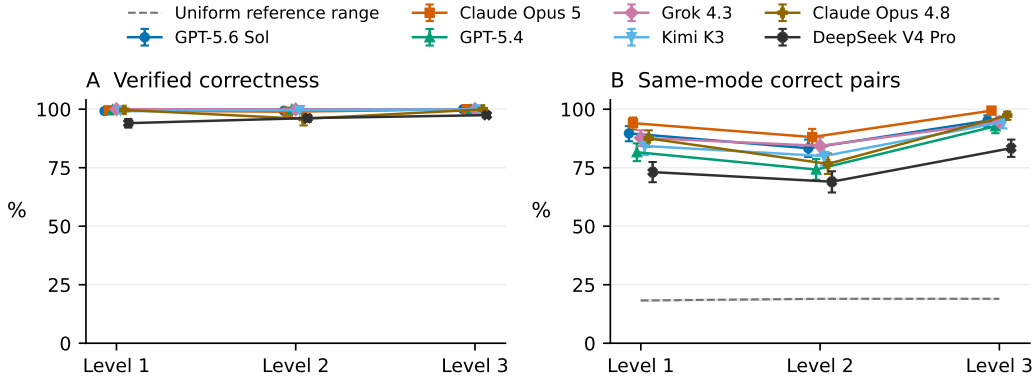


Figure 21: **Verified correctness and concentration in hosted Graph outputs.** Each deployment receives the same 128 prompts per level, with eight stateless samples per prompt. Intervals resample whole prompts. The gray band spans the models’ uniform-correct-key references, conditional on their observed correct draws. Graph was selected after the GPT-5.6 Sol run and before the prospective comparison runs. Level labels do not guarantee increasing difficulty for each model.

revised user instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed input. It does not prescribe a preferred divisor. Both task wording and system-message presence change; collection time also differs. This exploratory follow-up therefore supports a practical prompt repair without isolating its cause. Original refusals and all three residual refusals are retained. The original-protocol appendix tables retain all original Python responses; the main display uses this complete revised condition with an explicit caption note.

Table 25: **Opus 5 Python: original versus plain task wording.** Each row has 1,024 responses. R counts native refusals; A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). Strict and normalized results use the same frozen grader and typography rule. Original Level-2/3 collision is undefined because no prompt has two correct draws.

Condition	L	R	A	Strict		C	Normalized		
				D	D		A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4	
	2	1021	0.3	0.02	–	0.3	0.02	–	
	3	1024	0.0	0.00	–	0.0	0.00	–	
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9	
	2	0	71.8	1.31	84.7	100.0	1.38	85.7	
	3	3	76.2	1.38	82.7	99.7	1.41	84.1	

The new condition has no API failures or token-limit truncations. Strict grading accepts 2,282 responses; the frozen rule rescues 787 more by converting escaped percent signs (\%) to Python modulo (%). Thus every non-refused answer passes after this formatting-only correction. Normalized collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The residual three refusals occur on two prompts whose other identical requests are accepted. No retry-until-accepted selection is used.

The paired analysis resamples whole eight-draw prompt groups across conditions (20,000 replicates). Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2, 100.0] percentage points (pointwise 95% intervals). Output draws themselves are independent, and the different correct-pair populations preclude interpreting a raw collision difference as a matched-population prompt effect.

The saved sensitivity record binds the original and revised native receipts, request payloads, strict regrade audit and frozen normalization outputs.

Separate first-valid retry diagnostic. After completing the fixed 3,072-response revised-wording cohort, a separate condition retries only its three remaining refused slots, retaining the first

verified-valid response. All three succeed on their first additional request, and all reproduce a mode already seen for that prompt. This uses 3,075 requests to select 3,072 valid responses; its 100% accepted-set accuracy follows from selection and is not unconditional model accuracy. The selected set’s `distinct@8` is unchanged at 1.492, 1.375, 1.414 at Levels 1–3. Level-3 correct-pair collision changes from 84.0909% to 84.1518%; the other levels are unchanged. All initial and retry responses remain preserved. The main figure continues to use the complete fixed eight-draw condition, including its three refusals; these selected-validity results are only a separate diagnostic with variable request effort.

J.2 A MATCHED REQUESTED-TEMPERATURE SENSITIVITY TEST

A separate fixed subset contains eight prompts from each of the five domains at each of three levels: 120 prompts, with eight independent draws per prompt and temperature. Grok 4.3 and Kimi K3 each receive requested temperatures $T = 1.0$ and $T = 1.5$, giving 960 draws per condition and 3,840 in total. The two conditions share task rows and all non-temperature request settings; both temperatures are collected concurrently within each deployment. Every response is retained. This subset is a separate experiment and does not replace any full-cohort result in the main hosted display.

Provider API contracts support the requested controls and the deployments accept the requests, but native responses do not report effective sampling temperature. DeepSeek is excluded from this contrast because its documented thinking-mode interface ignores the temperature setting. These contrasts therefore concern the tested requested settings, not arbitrary temperatures or all deployments.

Tables average all five domains equally within each level; All equally averages all 15 domain-level cells. Confidence intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining each eight-draw group. They are pointwise and have no multiplicity adjustment. Correct-pair collision remains conditional on correct draws and can change its eligible prompt population; an undefined cell makes the corresponding macro undefined.

Observed sensitivity. For Grok, the observed breadth change is +0.050 [-0.058, +0.158] modes, and the collision change is -0.78 [-3.86, +2.12] percentage points. These intervals do not show a clear broadening at the tested setting. The per-response accuracy change is -2.29 [-3.85, -0.94] percentage points. This does not establish invariance to temperature in general.

Kimi returns 696/960 token-limited outputs at $T = 1.5$, compared with 0 at $T = 1.0$. Strict accuracy falls from 96.56% to 21.25%, while raw breadth falls from 1.933 to 0.983 modes. This is a severe token-limit and accuracy failure, not a clean increase in correct-output concentration. The five-domain collision mean is undefined at $T = 1.5$ because some cells have no eligible correct pairs. Neither model has provider-declared refusals in this experiment.

† marks an omitted interval when fewer than all 20,000 bootstrap replicates define the contrast. For Kimi collision, only 11,815 and 11,616 replicates are defined at Levels 1 and 2; the raw conditional percentile summaries remain in the source record. We do not present them as ordinary 95% intervals. Undefined cells are never replaced by zero or silently omitted.

Full domain-level estimates, counts, native outcomes and collision eligibility remain in the source-bound temperature analysis. Countdown has no finite-total-support uniform reference; finite-support sensitivity summaries use the other four domains. The paper record authenticates every included condition summary, native completion audit, frozen normalizer, paired analysis source and sampling-control review, with the complete fixed response cohort retained throughout.

J.3 GPT-5.6 SOL: TEMPERATURE, PASS@8 AND VERIFIED MODES

Figure 7 plots empirical `pass@8` against `distinct@8` for $T \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$, with reasoning none. Each condition retains the same 480 held-out prompts (32 per domain-level cell), with eight stateless responses per prompt: 3,840 responses per temperature and 19,200 in total. A prompt contributes one to `pass@8` if at least one of its eight answers is verified correct, and contributes its number of distinct correct canonical keys to `distinct@8`. Zero-success groups and all unsuccessful draws remain in the denominator. We compute success directly from the saved groups, not as $1 - (1 - p)^8$ from pooled per-response accuracy; task difficulty varies across prompts.

Table 26: **Requested-temperature sensitivity: Strict executable grades.** A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.50	-70.00 [-75.62, -63.75]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.12	-13.11 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.06	20.94	-78.12 [-83.75, -72.81]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.33	—	—
	All	A	96.56	21.25	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.25	—	—

Table 27: **Requested-temperature sensitivity: Frozen formatting-normalized grades.** A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.81	-69.69 [-75.31, -63.44]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.61	-12.62 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.38	20.94	-78.44 [-84.06, -73.12]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.18	—	—
	All	A	96.67	21.35	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.20	—	—

The expanded cohort retains all 120 original prompts and their 4,800 responses, adding 360 prompts and 14,400 responses. The extension follows the same outcome-independent row-hash ranking within each domain-level cell. Requests retain the frozen prompt bytes and verifier; only temperature varies in the sampling controls. New collection interleaves temperatures. Separate old-120 and new-360 analyses below assess cohort sensitivity; prompt composition and collection time differ between these cohorts.

The deployment rejected non-default temperatures with medium reasoning and rejected temperature 2.5. These curves use the supported reasoning `none` profile. Historical medium-reasoning measurements cover only the original 120 prompts and are omitted from the expanded-cohort tables and figure. This does not test temperature robustness of medium reasoning. A zero-temperature request does not establish deterministic outputs.

Each level averages five domains equally; All averages all 15 cells. Pointwise 95 percent intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining all eight draws. There is no multiplicity adjustment. A zero-width empirical interval reflects constant observed groups, not certain success on unseen prompts.

Table 28: **GPT temperature frontier: Strict executable grades.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). Brackets give pointwise 95% intervals; all temperatures use reasoning `none` and eight draws per prompt.

<i>T</i>	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	55.0 [48.8, 61.3]	0.569 [0.500, 0.637]	53.9 [47.7, 60.1]
	2	45.0 [39.4, 50.6]	0.456 [0.400, 0.512]	39.5 [34.4, 44.7]
	3	49.4 [42.5, 56.2]	0.519 [0.444, 0.594]	44.5 [38.3, 50.9]
	All	49.8 [46.2, 53.3]	0.515 [0.475, 0.554]	46.0 [42.6, 49.4]
0.5	1	70.6 [64.4, 76.9]	0.956 [0.850, 1.069]	57.5 [51.9, 63.0]
	2	50.0 [44.4, 55.6]	0.525 [0.463, 0.594]	38.6 [33.9, 43.4]
	3	54.4 [48.1, 60.6]	0.581 [0.506, 0.662]	43.0 [37.1, 48.9]
	All	58.3 [54.8, 61.9]	0.688 [0.640, 0.738]	46.4 [43.3, 49.5]
1.0	1	75.6 [69.4, 81.2]	1.181 [1.050, 1.312]	57.3 [52.0, 62.7]
	2	51.9 [45.6, 58.1]	0.581 [0.506, 0.662]	37.6 [33.2, 42.0]
	3	60.6 [54.4, 66.9]	0.713 [0.625, 0.806]	42.7 [37.3, 48.1]
	All	62.7 [59.2, 66.2]	0.825 [0.767, 0.885]	45.9 [42.9, 48.8]
1.5	1	80.6 [75.0, 86.2]	1.413 [1.269, 1.556]	56.6 [51.7, 61.4]
	2	58.1 [51.9, 64.4]	0.650 [0.569, 0.738]	38.4 [34.3, 42.7]
	3	65.6 [59.4, 71.2]	0.838 [0.731, 0.956]	43.0 [38.0, 48.2]
	All	68.1 [64.8, 71.5]	0.967 [0.900, 1.035]	46.0 [43.3, 48.8]
2.0	1	83.8 [78.1, 88.8]	1.519 [1.375, 1.663]	59.7 [54.8, 64.5]
	2	54.4 [48.1, 60.6]	0.600 [0.525, 0.675]	35.3 [31.4, 39.3]
	3	61.9 [56.2, 67.5]	0.819 [0.719, 0.925]	40.5 [35.8, 45.5]
	All	66.7 [63.3, 70.0]	0.979 [0.915, 1.044]	45.2 [42.6, 47.8]

Observed temperature frontier. At $T = 0.0$ and $T = 2.0$, normalized pass@8 is 65.62% and 79.38%, respectively, while distinct@8 is 0.673 and 1.146. Across these 5 measured temperatures, the highest observed aggregate pass@8 occurs at $T = 1.5$ and the highest distinct@8 occurs at $T = 2.0$. This finite exploratory sweep does not establish a population optimum or a continuous Pareto boundary, and level-specific curves differ.

Per-response accuracy changes from 62.11% to 55.94% between the endpoints. That diagnostic is distinct from pass@8: success within eight draws and the reliability of an individual draw need not change together as temperature varies.

The paired $T = 2.0$ minus $T = 0.0$ contrast is 13.8 [10.8, 16.7] pass@8 percentage points and 0.473 [0.417, 0.529] distinct correct modes, with pointwise 95% intervals. All prompt-level records, domain estimates, original per-response and collision analyses, and paired comparisons remain in the retained reports for both strict and normalized grading.

Retained outcomes. At $T = 0.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 0.5$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.5$, 0 of

Table 29: **GPT temperature frontier: Frozen formatting-normalized grades.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). Brackets give pointwise 95% intervals; all temperatures use reasoning none and eight draws per prompt.

T	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	70.6 [65.0, 76.2]	0.725 [0.662, 0.788]	70.5 [64.5, 76.2]
	2	60.6 [55.0, 66.2]	0.613 [0.556, 0.669]	54.5 [49.0, 59.9]
	3	65.6 [59.4, 71.9]	0.681 [0.606, 0.756]	61.4 [55.2, 67.6]
	All	65.6 [62.3, 69.0]	0.673 [0.635, 0.710]	62.1 [58.8, 65.5]
0.5	1	76.9 [71.2, 82.5]	1.038 [0.931, 1.150]	71.7 [66.4, 77.0]
	2	68.1 [62.5, 73.8]	0.706 [0.644, 0.769]	53.7 [48.8, 58.6]
	3	71.9 [65.6, 77.5]	0.775 [0.700, 0.850]	58.1 [52.3, 63.9]
	All	72.3 [69.2, 75.4]	0.840 [0.792, 0.890]	61.2 [58.2, 64.3]
1.0	1	79.4 [73.8, 84.4]	1.238 [1.106, 1.369]	70.4 [65.2, 75.5]
	2	69.4 [63.7, 75.0]	0.756 [0.681, 0.831]	51.6 [47.1, 56.2]
	3	76.9 [71.2, 82.5]	0.894 [0.806, 0.988]	55.2 [49.8, 60.7]
	All	75.2 [72.1, 78.3]	0.963 [0.904, 1.021]	59.1 [56.2, 62.0]
1.5	1	82.5 [77.5, 87.5]	1.494 [1.344, 1.644]	68.5 [63.7, 73.3]
	2	75.0 [69.4, 80.6]	0.844 [0.762, 0.931]	49.8 [45.5, 54.3]
	3	81.9 [76.2, 87.5]	1.038 [0.931, 1.156]	54.7 [49.6, 59.8]
	All	79.8 [76.7, 82.9]	1.125 [1.056, 1.196]	57.7 [54.9, 60.5]
2.0	1	85.0 [80.0, 90.0]	1.606 [1.456, 1.762]	69.6 [64.8, 74.3]
	2	72.5 [66.9, 78.1]	0.800 [0.731, 0.869]	45.7 [41.7, 49.8]
	3	80.6 [75.0, 86.2]	1.031 [0.931, 1.137]	52.5 [47.7, 57.5]
	All	79.4 [76.2, 82.5]	1.146 [1.079, 1.212]	55.9 [53.3, 58.6]

3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 2.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. These outcome counts retain every registered draw and are separate from the verifier-based correctness and canonical-key measurements above.

Sensitivity to the prompt expansion. The original and additional prompt cohorts retain separate paired temperature comparisons. Differences between cohorts may reflect prompt composition or collection time; they are not causal time effects.

Across all 480 prompts, the paired $T = 2$ minus $T = 1.5$ contrast is -0.4 [-2.5, 1.7] pass@8 percentage points and 0.021 [-0.029, 0.071] distinct modes (95% intervals). Original 120 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is -1.7 [-5.0, 0.8] pass@8 percentage points and -0.008 [-0.083, 0.067] distinct modes (95% intervals). Additional 360 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is 0.0 [-2.8, 2.8] pass@8 percentage points and 0.031 [-0.033, 0.094] distinct modes (95% intervals).

J.4 NATIVE PROVIDER OUTCOMES AND UNDEFINED CONCENTRATION

Provider-declared refusals and filtering are deployment outcomes, not evidence that the underlying model cannot solve the mathematical task. Empty refusals remain failed draws under the frozen verifier; no requests or grades are overwritten. Classification uses native metadata, not a semantic detector of refusals written as ordinary answer text.

Correct-pair collision is undefined when no prompt supplies two correct draws. The five-domain mean is therefore undefined if any domain has no eligible pairs; such entries remain – rather than zero or a mean over fewer domains.

GPT-5.6 Sol native completion states: completed: 15,360. Empty visible answers are reported separately from provider refusals.

Claude Opus 5 native completion states: end_turn: 12,838, refusal: 2,522. Empty visible answers are reported separately from provider refusals.

Claude Opus 5 returns provider-declared refusals for 409, 1021, 1024 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are cyber: 2454. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

Table 30: **Native service outcomes: GPT-5.6 Sol.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 31: **Native service outcomes: Claude Opus 5.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	409	0	409
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	1021	0	1021
2	MathIR	35	0	35
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	1024	0	1024
3	MathIR	33	0	32
3	PantryPlan	0	0	0

GPT-5.4 native completion states: completed: 15,355, incomplete: 5. Empty visible answers are reported separately from provider refusals.

Table 32: **Native service outcomes: GPT-5.4.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	1
2	Graph	0	0	0
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	1
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	2

Grok 4.3 native completion states: stop: 15,360. Empty visible answers are reported separately from provider refusals.

Table 33: **Native service outcomes: Grok 4.3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Kimi K3 native completion states: length: 15, stop: 15,345. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 native completion states: end_turn: 15,355, max_tokens: 5. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 returns provider-declared refusals for 0, 0, 0 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are not supplied. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

DeepSeek V4 Pro native completion states: length: 632, stop: 14,728. Empty visible answers are reported separately from provider refusals.

Table 34: **Native service outcomes: Kimi K3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 35: **Native service outcomes: Claude Opus 4.8.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	3
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	1

Table 36: **Native service outcomes: DeepSeek V4 Pro.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	59
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	4
1	PantryPlan	0	0	13
2	Graph	0	0	38
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	59
2	PantryPlan	0	0	80
3	Graph	0	0	24
3	Countdown	0	0	7
3	Python	0	0	0
3	MathIR	0	0	100
3	PantryPlan	0	0	246

Table 37: **Strict level summaries for completed hosted deployments.** Each level weights the five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is correct-pair collision (% with pointwise 95% whole-prompt intervals), pair weighted within each domain. Provider settings differ; these are descriptive outputs.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	—
Claude Opus 5	3	77.8	79.8	1.07	—
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Table 38: **GPT-5.6 Sol: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Table 39: **Claude Opus 5: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5

Table 40: **GPT-5.4: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–
1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–
2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–
3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 41: **Grok 4.3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 42: **Kimi K3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–
1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–
2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–
3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 43: **Claude Opus 4.8: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5

Table 44: **DeepSeek V4 Pro: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 45: **GPT-5.6 Sol: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Table 46: **Claude Opus 5: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5

Table 47: **GPT-5.4: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 48: **Grok 4.3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 49: **Kimi K3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 50: **Claude Opus 4.8: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5

Table 51: **DeepSeek V4 Pro: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

The result record `results/frontier_comparison_20260911.json` binds every source summary, completion audit, frozen prompt digest, raw-sample digest and primary graded sample digest. Pending deployments are omitted from both the figure and numerical comparisons. The figure was selected after the reference GPT run for its high strict correctness, certified support and lack of formatting corrections, and before collecting the prospective comparison runs. All five domains are retained in the tables, including disagreement across models.

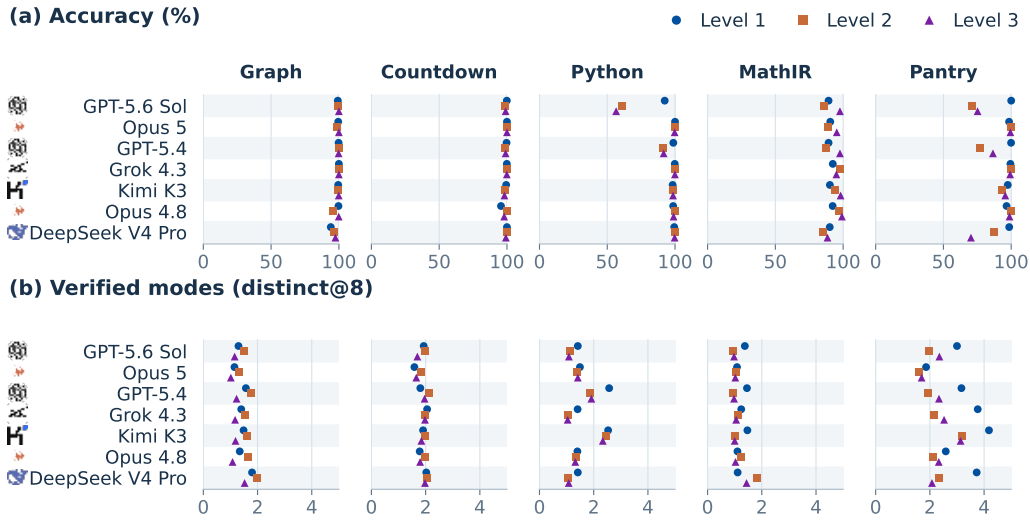


Figure 22: **Accurate hosted outputs can still cover few verified modes.** Columns show domains; markers distinguish Levels 1–3. Each point uses all eight responses to each of 128 prompts. Top: per-response accuracy. Bottom: `distinct@8`. A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. Table 1 reports per-level averages of `pass@8` and `distinct@8` across all five domains.

K MATCHED PROMPT-HINT ABLATION

K.1 REMOVING STRATEGY HINTS FROM UNCHANGED PROBLEMS

We freeze a paired follow-up on Python factors, MathIR, and Pantry at Levels 2 and 3. Within each of these six cells, the 32 smallest SHA-256 hashes of (20260911, level, domain, row index) select problems from the 128-row evaluation split, without consulting outcomes. Both arms generate eight fresh draws per identical problem. The original arm retains the published prompt. The neutral arm removes the Python suggestions to test small divisors with nested conditionals or dispatch on listed values, the MathIR ordered algebraic-isolation suggestion, and the Pantry preference for high-energy/protein, low-sodium ingredients such as seeds or oats. MathIR changes “those operations” to “the operations” to preserve the menu-ID instruction. The user problem, answer specification, executable verifier, and remaining format requirements are unchanged.

We report the paired neutral-minus-original effects on empirical $P_8 = \text{pass@8}$ and $D_8 = \text{distinct@8}$, with $B_8 = D_8 - P_8$ as additional verified breadth. All returned answers, including incorrect, empty, and truncated outputs, remain in their eight-draw groups; only verifier-accepted outputs create a verified key. Strict verification is primary; one unchanged, previously frozen formatting normalizer is a sensitivity analysis and preserves every strict success and canonical key. Python outputs in both arms undergo the same serial, warmed-verifier recheck, with original grades and any corrections retained in separate receipts.

Intervals use 20,000 whole-prompt bootstrap resamples, paired across arms and stratified by domain and level (seed 20260911). Cells contribute equally to reported macros. These are pointwise exploratory intervals for 32 problems per cell; crossing zero does not establish equivalence or absence of a prompt effect. A zero-width bootstrap interval records zero observed resampled variation, including all-tied P_8 outcomes; it is not precise evidence of population equivalence. Correct-pair collision and the certified-support uniform reference are secondary and retain their correct-pair denominators; the machine-readable report includes joint eligibility counts and undefined cells. Eight sampled draws do not identify total unseen support.

This appendix reports the complete local panel only. The separately registered frontier panel is not included here, and the overall local-plus-frontier experiment is incomplete.

For local models, we evaluate the initial Qwen2.5-0.5B model and fixed Level-2-trained DrGRPO and Replay DrGRPO checkpoints. Python and MathIR use matched training seeds 43–47; Pantry uses seeds 43 and 46. Level 3 is transfer evaluation. Seed means retain the same selected problem set across checkpoints. Five-seed intervals resample whole training seeds and paired prompts; Pantry reports both seed effects and intervals conditional on the two checkpoints. The native local syntax constraints and 192-token budget differ from hosted inference, so effects are interpreted within each model and interface.

Table 52: Local Qwen2.5-0.5B checkpoints under strict verification. Each arrow gives original $\rightarrow$ neutral. P_8 is empirical pass@8 , D_8 is verified distinct@8 , and $B_8 = D_8 - P_8$. Effects are neutral minus original; brackets are pointwise 95% intervals. Local rows average the displayed number of training seeds. Five-seed intervals include seed and paired-prompt resampling; two-seed Pantry intervals condition on the two fixed checkpoints.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531 [-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688 [-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000 [-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031 [+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00
DrGRPO ($s = 5$)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025 [-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00
DrGRPO ($s = 5$)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037 [-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00
DrGRPO ($s = 2$)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016 [-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02
DrGRPO ($s = 2$)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000 [-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02
DrGRPO ($s = 5$)	Python factors L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812 [-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00
DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000 [-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00
Replay DrGRPO ($s = 5$)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019 [-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00
Replay DrGRPO ($s = 5$)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037 [-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00
Replay DrGRPO ($s = 2$)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016 [-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00
Replay DrGRPO ($s = 2$)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047 [+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06
Replay DrGRPO ($s = 5$)	Python factors L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331 [-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24
Replay DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506 [-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26

Table 53: Local Qwen2.5-0.5B checkpoints under frozen formatting normalization. Each arrow gives original $\rightarrow$ neutral. P_8 is empirical pass@8, D_8 is verified distinct@8, and $B_8 = D_8 - P_8$. Effects are neutral minus original; brackets are pointwise 95% intervals. Local rows average the displayed number of training seeds. Five-seed intervals include seed and paired-prompt resampling; two-seed Pantry intervals condition on the two fixed checkpoints.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531 [-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688 [-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000 [-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031 [+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00
DrGRPO ($s = 5$)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025 [-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00
DrGRPO ($s = 5$)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037 [-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00
DrGRPO ($s = 2$)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016 [-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02
DrGRPO ($s = 2$)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000 [-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02
DrGRPO ($s = 5$)	Python factors L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812 [-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00
DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000 [-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00
Replay DrGRPO ($s = 5$)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019 [-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00
Replay DrGRPO ($s = 5$)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037 [-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00
Replay DrGRPO ($s = 2$)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016 [-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00
Replay DrGRPO ($s = 2$)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047 [+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06
Replay DrGRPO ($s = 5$)	Python factors L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331 [-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24
Replay DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506 [-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26

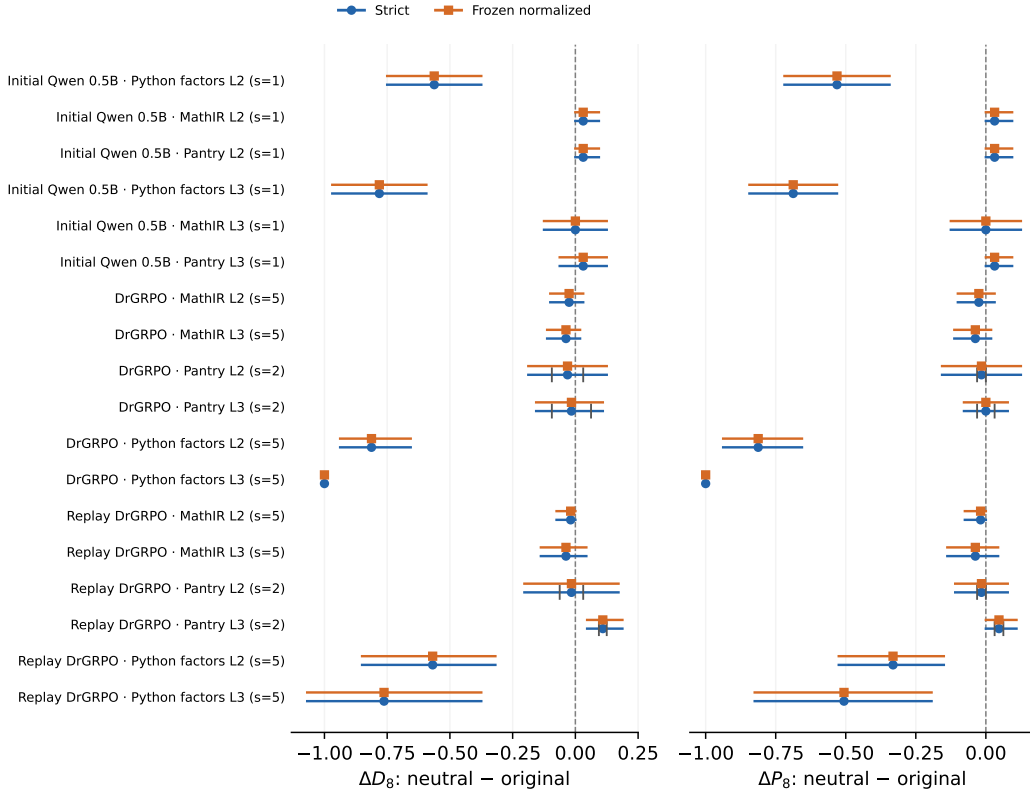


Figure 23: Prompt-hint removal effects for local checkpoints, preserving every registered domain-level cell. Blue circles use strict verification; orange squares use the same frozen formatting normalizer. Horizontal bars show the intervals described in the tables; zero is the vertical reference. Gray endpoint marks show the two individual Pantry seed effects.

Two-seed Pantry sensitivity. Two checkpoints cannot support a stable estimate of training-seed variability. The following lists both strict prompt effects; the intervals in the local table condition on these two checkpoints. DrGRPO L2 (seed 43: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); DrGRPO L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.06$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); Replay DrGRPO L2 (seed 43: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.06$; seed 46: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$); Replay DrGRPO L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.12$; seed 46: $\Delta P_8 = +0.062$, $\Delta D_8 = +0.09$).

Interpreting the intervention. The Python intervention removes an executable nested-conditional example along with divisor-testing and dispatch guidance; it also shortens the prompt. Its paired effect concerns this combined removal and does not isolate a preference cue from example or length effects. A reduction in `distinct@8` accompanied by lower `pass@8` is not by itself evidence of greater concentration among correct outputs. Here the initial model is the single fixed Qwen2.5-0.5B-Instruct checkpoint before the reported RL training; it is already instruction-tuned and has no fine-tuning-seed replication in this ablation.

Observed prompt dependence. Python shows a large loss of correctness after guidance removal. Across the five DrGRPO seeds, mean P_8 falls from .8125 to zero at Level 2 and from 1.0 to zero at Level 3; D_8 falls identically and $B_8 = 0$ in both arms. With no correct neutral DrGRPO outputs, its conditional-correct concentration is undefined. Replay DrGRPO retains neutral $P_8 = .55625$ and $.49375$ and $D_8 = .675$ and $.65625$ at Levels 2 and 3, respectively, while losing additional breadth: $\Delta B_8 = -.2375$ and $-.25625$. The initial checkpoint also loses Python correctness (Table 52). These observations identify substantial dependence on the combined Python wording intervention; a raw mode-count decline cannot be interpreted solely as a redistribution among correct answers.

Post-hoc Python failure audit. A descriptive audit of all 5,120 saved DrGRPO Python draws (five seeds, two levels, both wordings) finds that all 2,560 original outputs extract the same lambda, whose body matches the conditional example in the prompt:

```
lambda n: 2 if n % 2 == 0 else 3 if n % 3 == 0 else 5
```

Of the 2,560 neutral draws, 1,816 (70.94%) are invalid Python expressions, 59 violate other restricted-language rules, 376 return non-integers, 276 fail the proper-divisor condition, and 33 lack an extracted candidate after reaching the token limit. Thus most neutral failures involve syntax; truncation accounts for only 1.29%. The unchanged extractor, parser, and executor reproduce every stored grade and successful canonical key. This taxonomy was developed after the paired results were known and does not identify an internal training mechanism. Counts, deterministic paired examples, and source hashes are retained in `results/modebench_prompt_ablation_python_failure_diagnostic_20260911.json`.

Concentration after hint removal. Every trained MathIR checkpoint has $B_8 = 0$ under both wordings at both levels. Within each problem’s eight-draw group, every observed pair of correct outputs shares a canonical key, in every trained checkpoint–level cell: correct-pair collision is 1.0, versus the certified-support uniform reference of .2. For example, at Level 2, Replay DrGRPO retains mean $P_8 = .975$ without the order hint, compared with .99375 originally. Across its five checkpoints, the neutral arm has 4,179 colliding pairs out of 4,179 correct pairs, from 154 eligible checkpoint–problem groups; the original arm has 4,382 out of 4,382 pairs from 157 such groups. Both arms evaluate the same 32 problems at each checkpoint. This preserves the observed concentration despite removing the specified order hint; it does not measure all unseen modes or remove the other interface constraints. Pantry effects are mixed. Replay DrGRPO at Level 3 gains .109375 in D_8 , comprising .046875 in P_8 and .0625 in B_8 , but this comparison has only two training seeds. These local findings supply no completed frontier-model prompt control; that separately registered panel remains pending.

L SAMPLING-BUDGET ABLATION

L.1 PROTOCOL AND ESTIMANDS

Large-budget `pass@k` evaluation can reveal differences hidden at small sampling budgets (Yue et al., 2025). We therefore freeze a second follow-up after observing the eight-draw prompt control: within every domain-level cell we take ranks 1–16 of its original outcome-independent SHA-256 selection. The six cells retain both prompt wordings, the same mathematical problems, and the same fixed initial and trained checkpoints. Every arm receives 64 fresh draws; none of the earlier eight-draw responses enters this analysis. The initial Qwen2.5-0.5B-Instruct checkpoint has no training-seed replication. DrGRPO and Replay DrGRPO retain five matched seeds in Python and MathIR and two in Pantry. Trained Level-3 evaluation is transfer from Level-2 training.

For a complete 64-draw pool let c be its number of correct outputs and n_j its counts of distinct verified canonical keys. At $k \in \{1, 2, 4, 8, 16, 32, 64\}$, primary curves average all size- k subsets without replacement from the complete pool:

$$P_k = 1 - \frac{\binom{64-c}{k}}{\binom{64}{k}}, \quad D_k = \sum_j \left[1 - \frac{\binom{64-n_j}{k}}{\binom{64}{k}} \right], \quad B_k = D_k - P_k.$$

An infeasible numerator combination is zero. Every incorrect, empty, refused, or truncated response remains in the pool; only verifier-accepted responses contribute keys. These correlated curve points describe the retained pool, not complete unseen support. Ordered prefixes by preassigned draw index, rather than response arrival time, are a separately labeled sensitivity in the source report. $P_{64} - P_8$, $D_{64} - D_8$, and $B_{64} - B_8$ separate first-success gains from additional observed modes.

Strict verification is primary. The unchanged frozen formatter is a sensitivity, preserving every strict success and canonical key. All Python texts undergo the same serial warmed-verifier recheck; original grades and any corrections are retained. Intervals use 20,000 whole-problem bootstrap replicates (seed 20260911), paired across wordings and stratified by domain and level. Five-seed intervals additionally resample whole matched training seeds with shared prompt indices; Pantry intervals condition on its two checkpoints. Intervals are pointwise and exploratory for 16 problems per cell. Zero-width intervals reflect zero observed resampled variation, not precise evidence of equivalence.

To compare breadth at matched observed correctness, rarefy m correct draws alone:

$$R_m = \sum_j \left[1 - \frac{\binom{c-n_j}{m}}{\binom{c}{m}} \right].$$

The primary paired conditional comparison keeps only prompts with $c \geq m$ under both wordings; eligibility changes with m . Every eligible denominator is retained, and an undefined registered seed is never silently omitted from an equal-seed mean. Full-64 collision pools equal-key correct pairs within each checkpoint before taking equal-seed means.

All support counts here are certified lower bounds $L \leq M$, not exhaustive support sizes: Python uses its two externally certified witnesses, MathIR five certified modes, and Pantry its frozen certificate count. Accordingly $1/L$ is an upper reference for uniform collision over full support; $L[1 - (1 - 1/L)^m]$ is a lower reference for uniform expected breadth, not a lower bound on model breadth. Observed distinct counts may exceed L . The source report also averages this uniform breadth over $J \sim \text{Hypergeom}(64, c, k)$ to match unconditional correctness at each k .

This is a partial-panel report: the local panel is complete, while the separately registered frontier panel is omitted and the overall experiment remains incomplete.

Table 54: Eight-to-64 discovery gains under strict verification. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. $s = 1$ denotes one fixed initial checkpoint, not training-seed replication. Five-seed intervals resample seeds and paired prompts; two-seed Pantry intervals condition on the observed checkpoints and retain all paired prompt outcomes.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	-0.138 [-0.217, -0.061]
Replay DrGRPO ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	-0.497 [-0.766, -0.270]
Replay DrGRPO ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	-0.004 [-0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	-0.109 [-0.383, +0.164]
Replay DrGRPO ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [-0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	-0.008 [-0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [-0.123, +0.095]
Replay DrGRPO ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	-0.008 [-0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [-0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [-0.201, +0.392]
Replay DrGRPO ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [-0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [-0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [-0.109, +0.399]
Replay DrGRPO ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	-0.107 [-0.321, +0.098]

Table 55: Eight-to-64 discovery gains under frozen normalization sensitivity. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. $s = 1$ denotes one fixed initial checkpoint, not training-seed replication. Five-seed intervals resample seeds and paired prompts; two-seed Pantry intervals condition on the observed checkpoints and retain all paired prompt outcomes.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	-0.138 [-0.217, -0.061]
Replay DrGRPO ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	-0.497 [-0.766, -0.270]
Replay DrGRPO ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	-0.004 [-0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	-0.109 [-0.383, +0.164]
Replay DrGRPO ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [-0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	-0.008 [-0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [-0.123, +0.095]
Replay DrGRPO ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	-0.008 [-0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [-0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [-0.201, +0.392]
Replay DrGRPO ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [-0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [-0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [-0.109, +0.399]
Replay DrGRPO ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	-0.107 [-0.321, +0.098]

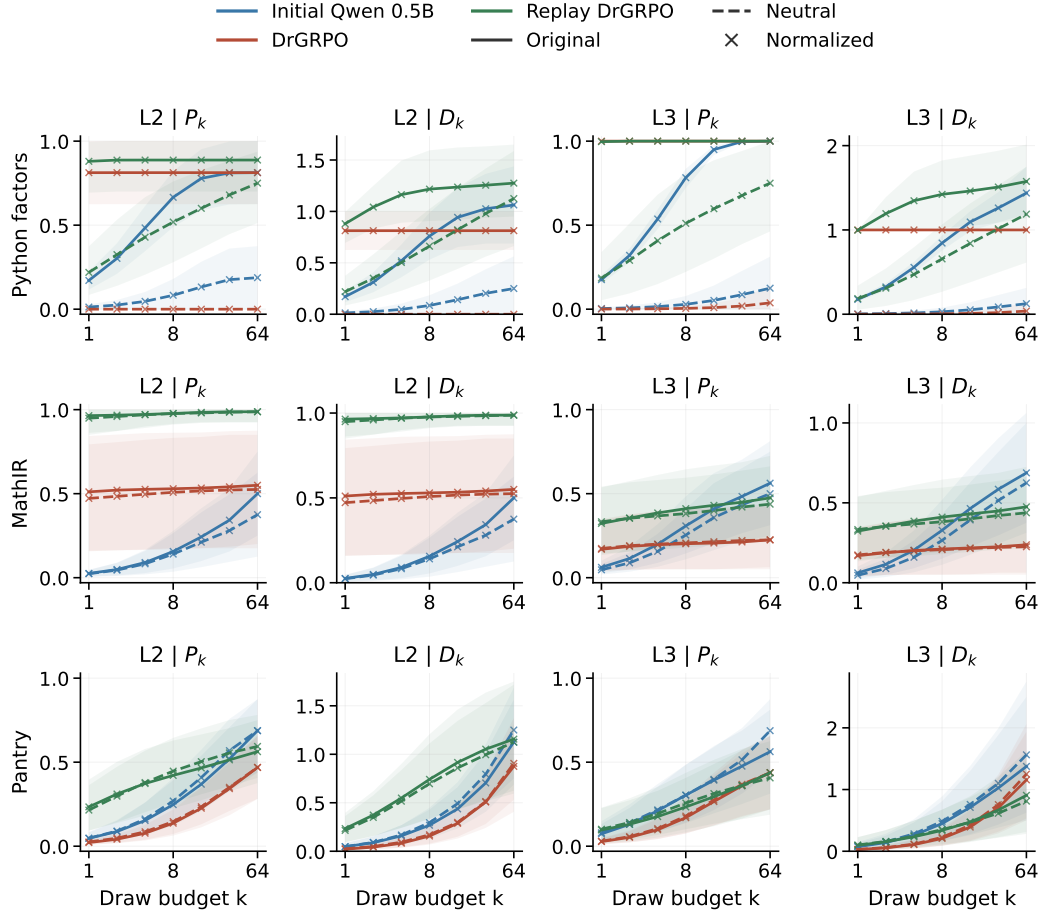


Figure 24: Fresh 64-draw discovery curves. Separate axes show P_k and D_k for each level and domain; pass probability always uses a 0–1 scale. Color identifies the checkpoint method or hosted model, solid/dashed lines original/neutral wording. Lines and shaded pointwise intervals use strict grading; crosses show the frozen-normalization sensitivity. Prefix estimates are separately retained in the report with pointwise intervals for every model and cell.

Table 56: Full-64 strict correct-pair collision. C pools correct pairs within a fixed checkpoint or deployment; trained local rows then average checkpoint rates equally across seeds. U applies the same weighting to the certified-support uniform upper reference. Eligible checkpoint–problem groups E and correct-pair counts Q are summed descriptively across checkpoints. These totals do not define the equal-seed mean; separate pooled rates and per-seed counts are retained in the report. A seed with no eligible pair leaves its all-seed mean undefined, with every individual denominator retained.

Model	Cell	C_O	C_N	U_O	U_N	E_O/E_N	Q_O/Q_N
DrGRPO	Python factors L2	1.000	—	0.500	—	65/0	130977/0
Initial Qwen 0.5B	Python factors L2	0.791	0.875	0.500	0.500	13/3	1270/24
Replay DrGRPO	Python factors L2	0.809	0.800	0.500	0.500	71/53	140722/18289
DrGRPO	Python factors L3	1.000	—	0.500	—	80/0	161280/0
Initial Qwen 0.5B	Python factors L3	0.875	1.000	0.500	0.500	16/1	1010/3
Replay DrGRPO	Python factors L3	0.805	0.797	0.500	0.500	80/52	160275/12893
DrGRPO	MathIR L2	1.000	1.000	0.200	0.200	43/42	80766/74119
Initial Qwen 0.5B	MathIR L2	1.000	1.000	0.200	0.200	3/3	64/64
Replay DrGRPO	MathIR L2	1.000	1.000	0.200	0.200	79/79	155016/151485
DrGRPO	MathIR L3	0.980	1.000	0.200	0.200	16/18	24457/25393
Initial Qwen 0.5B	MathIR L3	0.883	0.912	0.200	0.200	7/6	282/181
Replay DrGRPO	MathIR L3	1.000	1.000	0.200	0.200	34/34	46891/50301
DrGRPO	Pantry L2	0.374	0.276	0.047	0.055	9/10	101/193
Initial Qwen 0.5B	Pantry L2	0.849	0.787	0.052	0.053	7/10	212/178
Replay DrGRPO	Pantry L2	0.600	0.613	0.051	0.052	16/18	10144/8510
DrGRPO	Pantry L3	0.203	0.228	0.047	0.049	10/12	134/165
Initial Qwen 0.5B	Pantry L3	0.375	0.313	0.049	0.049	7/6	531/705
Replay DrGRPO	Pantry L3	0.636	0.666	0.051	0.052	10/11	4006/3946

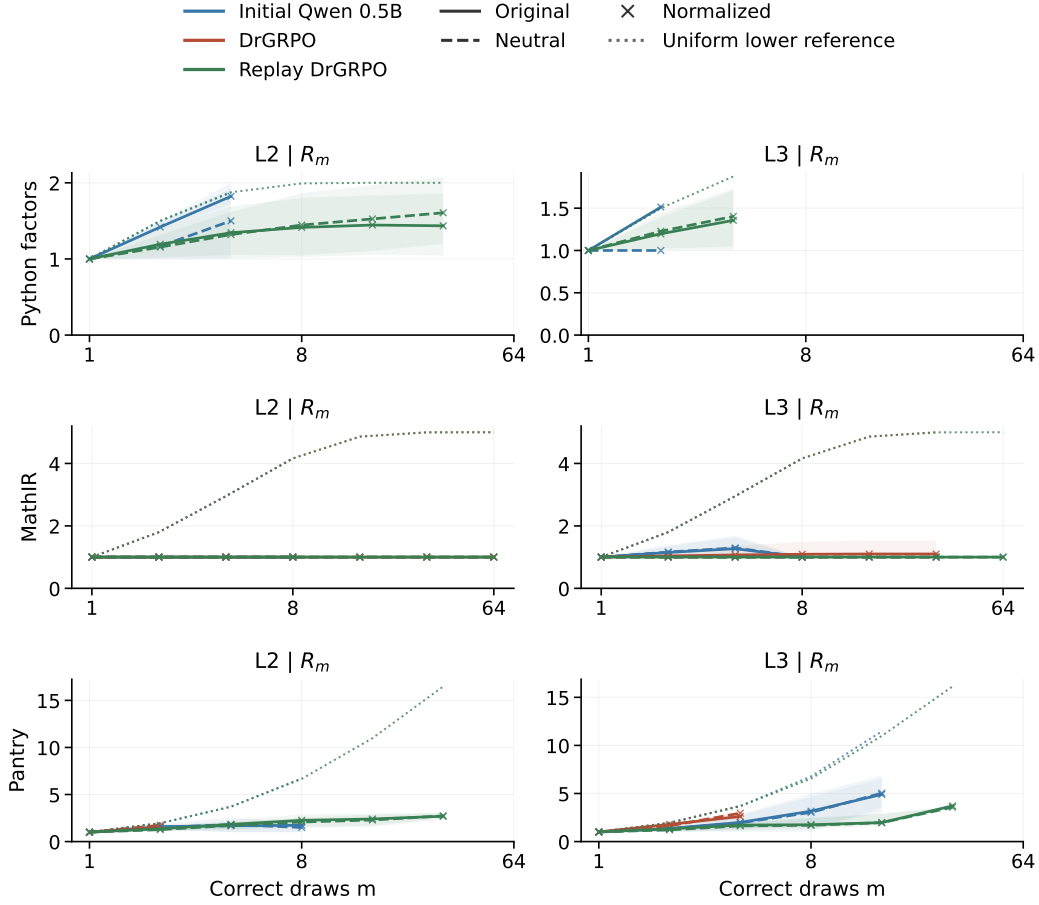


Figure 25: Breadth after exactly m correct draws, on prompts jointly eligible under both wordings. Color and solid/dashed wording styles match the discovery figure; dotted lines give method-specific uniform lower references using the same eligible prompts. Undefined means are gaps. Crosses show frozen normalization. Eligibility varies with m , so curves do not hold the eligible population fixed across the horizontal axis; the following table gives all strict joint counts for every method, level, and correct-draw budget, including groups with zero eligible prompts.

Table 57: Strict joint eligibility for a fixed correct-draw budget: both wordings must have at least m correct draws. Entries count eligible checkpoint–problem groups, summing over displayed seeds; the same 16 problems recur across checkpoints. Arm-specific eligibility at every m is also retained in the report for both prompt wordings and every registered checkpoint.

Model	Cell	$m = 1$	$m = 2$	$m = 4$	$m = 8$	$m = 16$	$m = 32$	$m = 64$
DrGRPO ($s = 5$)	Python factors L2	0	0	0	0	0	0	0
Initial Qwen 0.5B ($s = 1$)	Python factors L2	3	3	2	0	0	0	0
Replay DrGRPO ($s = 5$)	Python factors L2	59	53	44	36	28	13	0
DrGRPO ($s = 5$)	Python factors L3	3	0	0	0	0	0	0
Initial Qwen 0.5B ($s = 1$)	Python factors L3	2	1	0	0	0	0	0
Replay DrGRPO ($s = 5$)	Python factors L3	60	52	44	35	26	7	0
DrGRPO ($s = 5$)	MathIR L2	42	42	40	40	39	37	30
Initial Qwen 0.5B ($s = 1$)	MathIR L2	4	3	3	1	0	0	0
Replay DrGRPO ($s = 5$)	MathIR L2	79	79	78	76	76	75	70
DrGRPO ($s = 5$)	MathIR L3	15	14	14	14	14	14	6
Initial Qwen 0.5B ($s = 1$)	MathIR L3	8	6	6	2	0	0	0
Replay DrGRPO ($s = 5$)	MathIR L3	32	30	28	25	24	23	13
DrGRPO ($s = 2$)	Pantry L2	9	4	1	1	0	0	0
Initial Qwen 0.5B ($s = 1$)	Pantry L2	10	7	4	2	0	0	0
Replay DrGRPO ($s = 2$)	Pantry L2	17	15	12	11	10	5	0
DrGRPO ($s = 2$)	Pantry L3	13	9	4	1	0	0	0
Initial Qwen 0.5B ($s = 1$)	Pantry L3	8	6	5	4	2	0	0
Replay DrGRPO ($s = 2$)	Pantry L3	11	8	6	5	4	2	0

L.2 OBSERVED SAMPLING-BUDGET EFFECTS

The completed local panel contains 110,592 fresh responses from all 25 registered checkpoints. Each trained checkpoint is evaluated in its own domain at both levels; the initial checkpoint covers all six cells. Serial regrading corrects one original Python grade from failure to success, confirmed once in an independent warmed worker using the unchanged verifier and timeouts. The saved receipt does not identify the cause of the original discrepancy. No response is rescued by formatting normalization, so strict and normalized analyses coincide throughout this panel. The original responses and grades remain available alongside the correction audit and source identities.

MathIR: concentration largely persists through 64 draws. At Level 2, both trained methods have $B_k = 0$ at every measured budget under both wordings. Replay DrGRPO has original $P_8 = .9774$ and $P_{64} = .9875$; neutral wording gives .9767 and .9875. Its D_k equals P_k throughout, so the small late gain consists entirely of first successes. At exactly eight correct draws, both wordings have $R_8 = 1$ on the same 76 eligible checkpoint–problem groups, versus the certified-support uniform lower reference $5[1 - (4/5)^8] = 4.1611$. These groups reuse the same 16 problems across five checkpoints. At Level 3, all neutral trained groups and all original Replay DrGRPO groups also contain at most one observed correct key. There is one exception in original DrGRPO: a single problem at seed 46 contains two keys with counts 28 and 9, giving mean $B_{64} = .0125$ and $B_{64} - B_8 = .0035$. Its equal-seed correct-pair collision is .9803; the other trained MathIR cells have collision 1.0, against the uniform upper reference .2. Thus the larger budget reveals a rare additional mode without supporting an exact single-mode claim for every trained condition.

Python: prompt dependence and late discovery coexist. Original DrGRPO has unchanged $P_k = D_k = .8125$ at Level 2 and 1.0 at Level 3 from eight through 64 draws. Neutral DrGRPO has zero correct responses out of 5,120 at Level 2 and three out of 5,120 at Level 3. Each of those three successes occurs alone in its checkpoint–problem group, so Level 3 mean $P_{64} = D_{64} = .0375$ while correct-pair collision remains undefined. Replay DrGRPO behaves differently. Under original wording its pass probability is already unchanged from eight to 64, but D_k gains .0582 at Level 2 and .1529 at Level 3. Under neutral wording, P_k rises from .5186 to .7500 and from .5109 to .7500, respectively; D_k rises from .6618 to 1.1250 and from .6561 to 1.1875. These final mode counts remain below the original 1.2750 and 1.5750. The corresponding additional-mode gains are .2318 [.0574, .4565] and .2923 [.0659, .5445], with the registered seed-and-prompt intervals. Larger budgets therefore recover both first successes and additional verified alternatives in the neutral replay condition. These unconditional gains do not identify a change in the conditional distribution of correct solutions by themselves.

Pantry: eight draws understate observed discovery. Across the trained Pantry cells and both wordings, $D_{64} - D_8$ ranges from .4151 to 1.0247, with additional-mode gains ranging from .2739 to .7664. The initial checkpoint also gains .8565–1.0778 modes. In the two observed training seeds, Level 3 point estimates even change method ordering: original DrGRPO/Replay DrGRPO have $D_8 = .2075/.3381$ but $D_{64} = 1.1563/.9063$; neutral wording gives .2253/.3515 at eight and 1.2500/.8125 at 64. This is a descriptive ranking of these checkpoints, not evidence of a population-level method advantage. Pantry has only two training seeds, and fixed-correctness comparisons at larger m often have few or no jointly eligible problems. The eligibility table and undefined means remain essential to interpreting the conditional curves.

These observations support persistent concentration in several trained conditions and substantial late discovery in others. A flat finite-sample curve cannot exclude rarer unseen modes, and the result does not license a single saturation claim across domains. The new $k = 8$ points rarefy the fresh 64-draw pools on 16 problems per cell; they are not differences against the previous experiment’s 32-problem scores. The separately registered frontier panel, including GPT-5.6 Sol, is now collecting responses and supplies no completed result in this local-panel report. Cross-scale robustness of these sampling-budget patterns therefore remains an open empirical question for that panel.

M SOURCE INTEGRITY AND REPRODUCIBILITY

M.1 ADMISSIBLE SOURCES AND MISSING CHECKPOINTS

Falcon3-1B Countdown ReplayDr.GRPO seed 59 contains two complete four-draw terminal evaluation sequences with identical request identities but conflicting metrics. The integrity amendment excludes that source at every checkpoint and from every efficacy endpoint, paired contrast, and aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs: 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial intersections are descriptive and receive no confidence interval.

Trajectory records use registered sources and approved continuations or replacements, with superseded attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy and mode-count views use the same populations and gap rules. Supporting methods retain their explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the curves. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule, and source hashes.

M.2 DATA, MODELS, AND SCIENTIFIC RECORDS

The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row identities, and construction and admission records. Its loading guide and evaluation guide identify the reported splits and preserve auxiliary data separately. The model archive links each available model–domain–method–seed export to an immutable upload revision containing original weights, configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license. The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-weighting ablation, and supporting controls to their model families; the scientific-record index and restore guide identify preserved analysis inputs and SHA-256 verification procedures.

Each empirical figure has a machine-readable record identified in the figure manifest, containing its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper versions to their builders, records, and reproduction commands. Detailed source and recovery inventories remain in those records and the repository’s archived audit documents. Model availability may extend beyond the frozen analysis population; it does not change a figure’s source-admission or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories require the recorded evaluation checkpoints.

M.3 EXACT CONTINUATION AND VALIDATION

Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream state in Appendix F.4, in addition to model weights. State mismatch, non-finite values, missing completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures. Placement-only recovery may change hardware or wall time but must preserve the scientific update. Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints or silently incorporated from a newer archive.

N DISCLOSURES

AI USE STATEMENT

Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The authors reviewed and verified the resulting content and take responsibility for the final manuscript.

ETHICS STATEMENT

The benchmark and experiment-specific data are synthetic; this study collects no human-subject or personal data. More verified modes are not inherently better: a distinct execution may be risky or costly even when it passes the validator, and `distinct@K` does not measure downstream value. Systems that evaluate or optimize breadth should report correctness, invalidity, and application-specific costs alongside mode counts.

REPRODUCIBILITY STATEMENT

Claims are computed from versioned result files; missing final results and aggregate drift cause the build to fail. Appendix M records protocols, hashes, validations, and regeneration commands; Appendix D records split identities. The AI Use Statement above discloses generative-AI assistance.